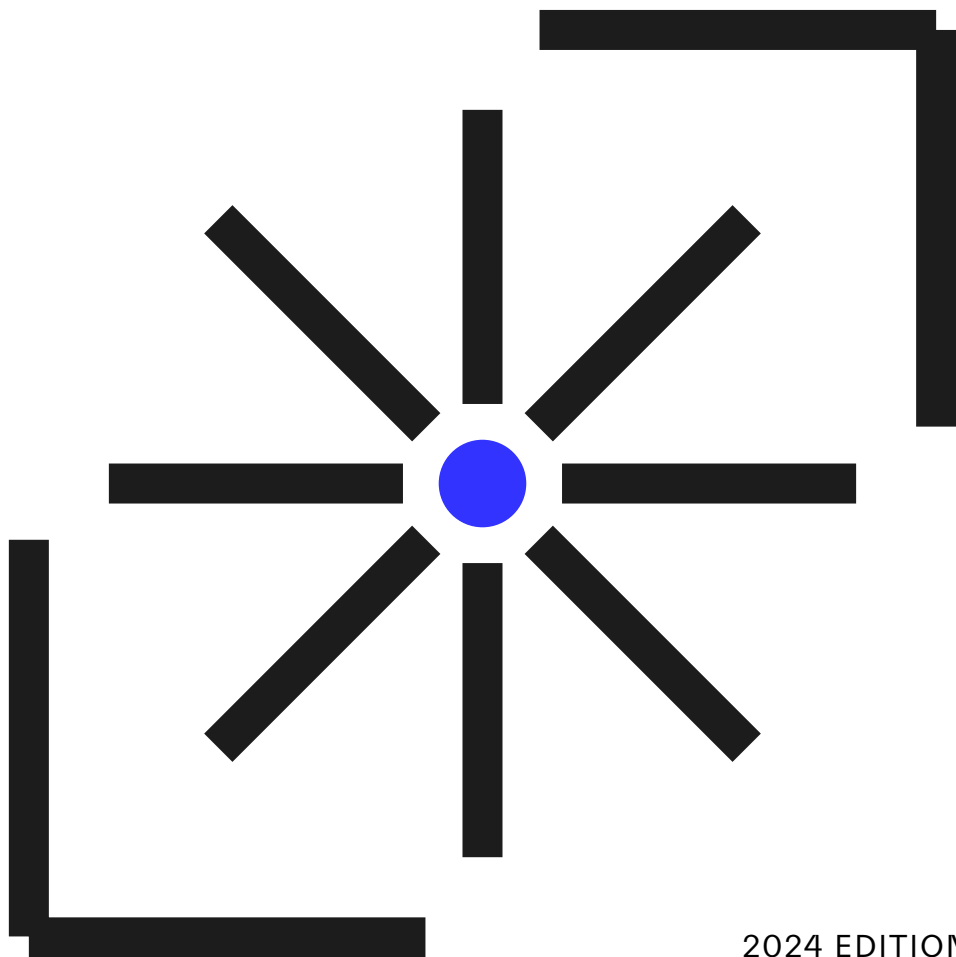
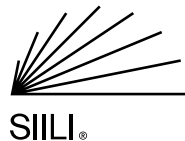


Handbook for AI-Powered Developers



2024 EDITION



This book is written by developers to developers. It is original content written by the authors together with an editorial team. We've used AI exclusively to proofread, summarize and organize text.

Thank you to everyone who contributed to making this book a reality. Your expertise, creativity, and support were invaluable.

The Authors

Handbook for AI-Powered Developers

Siili Solutions Oyj

2024 Edition

Authors

JP Heimonen

Antti Koivisto

Foreword

Tomi Pienimäki

Graphic design

Jenni Siltala

Special thanks

Marko Jaanu

Joanna Kahila

Anna Kantosalo

Jari Rangell

Valtteri Taube

Michal Wlosik

Handbook for AI-Powered Developers



Foreword 10

How to read this book 12

About Siili 14



1. Basics: Becoming AI-powered 17

Roadmap to AI-powered software development 19

Reservations and guidelines for building your roadmap 21

Towards a fully AI-powered development pipeline 23

The value proposition of getting AI-powered 26

Challenges and considerations in AI adoption 28

Short Guide to AI: Introduction 31

Context windows 33

Retrieval-augmented generation (RAG) 36

Hallucinations 38

Custom instructions and prompt engineering 39

Scenarios for custom instructions 40

Security and data protection in AI-assisted development 42

Code vs. data: A critical distinction 42

Mindset: Integrating AI in development 44

Maximizing AI usage for efficiency 44

Balancing AI use and skill development 46

AI tools in software development: A practical guide 52

From copy & paste to fully integrated 52

Prompting tools 55

ChatGPT 57

Claude 63

Developer tools	68
GitHub Copilot	70
JetBrains AI	71
Cursor	72
Amazon Q	75
SuperMaven	76
Code reviewing tools	77
CodiumAI PR Agent	78
Code Rabbit	80
Beyond prompting – software engineering agents	82
Beyond code generation with chaining and tool integration	82
Redefining development cycles	83
Challenges in adopting coding agents	85
The token economy: Cost vs. capability	87
Autonomous coding agents	88
Devin	88
OpenHands: The open-source alternative	89
Micro-agents	90
GitHub Copilot Workspaces	92
Aider	93

2. In practice: Using AI in development 97

Getting started: The ideation phase	100
Preparing context for AI assistance	105
Leveraging AI for initial ideas and architecture	107
Organize backlog with user story map	110
Build prototypes quickly	116
Bootstrapping	120
Solution structure	120
Database design	125

Product design	134
UI wireframes	135
Architectural suggestions	135
Playing devil's advocate	138
Development	143
Generating code	143
IDE assistant chat	144
External LLMs	146
Explaining	146
Code commenting	150
Refactoring	153
Fixing errors	154
Code review	158
Working with data	170
Building queries	170
Troubleshooting SQL and database issues	172
Deployment	174
Visualizing the infrastructure	174
Infrastructure as code with Terraform	176
Automating deployment with GitHub Actions	177
Scripting and tools	180
Maintenance and support	185
Scenario 1: The reservation black hole	185
Scenario 2: The great menu meltdown	186
Scenario 3: The Payment Processing Panic	187
Documentation	189
Architectural decision records (ADRs)	190
Testing and quality assurance	195
Unit tests	195
Taking testing to the next level	198
Testing with Playwright and similar tools	201
Expect the unexpected	205

Learning and skill development	206
AI as a learning accelerator	207
Brownfield considerations	214



3. Road ahead: Future trends and how to prepare 219

Decision-making in the AI era	220
Choosing the right approach	221
Trends in AI-powered software engineering	225
Evolution of LLMs	225
Agents to the rescue	228
Making agents work together	230
How to become AI-powered?	232
Cutting through the red tape	232
Fostering an AI-first mindset	233
Building an AI-ready knowledge base	233
Embracing remote and asynchronous work	234
Software engineering agents	234
Towards AI-powered development teams	236
Staying ahead with AI	238

Foreword

There's a new wave coming. In fact, it's already here, but it's not the kind of revolution that announces itself loudly with banners and slogans. This one is quieter, more subtle, yet it's changing the world faster than most realize.

Artificial Intelligence has slipped into software development processes as well, and the change it brings is sweeping. Some see it as an enhancement, others as a threat. Regardless of the view – it's simply inevitable.

Over the past 25 years, we've witnessed several significant technological disruptions, such as the rise of the internet, mobile devices, content streaming, and e-commerce. These innovations have consistently enhanced our quality of life, making it difficult to imagine returning to the way things were. Gone are the days of buying your favorite music on CDs (even AC/DC eventually embraced Spotify), or paying bills in person at the ATM.

In his book **The Coming Wave**, Mustafa Suleyman, CEO of Microsoft AI and co-founder of DeepMind, compares it to a wave. And as with any great wave, you have a choice. You can stand still and let it overwhelm you, or you can learn to ride it.

AI is woven into the very fabric of how we code, test, and maintain software. It's solving problems in ways we didn't know existed. It's accelerating timelines we thought were fixed. Today's AI tools are not just passive assistants. They are active collaborators in the coding process, capable of reviewing code, spotting bugs, generating documentation, and even suggesting new frameworks. They are augmenting human creativity, pushing the limits of what software development teams can achieve.

This book isn't just about AI in theory and how it will reshape the world.

Siili is all about making things real and thus this book is a practical guide for developers and teams who want to harness the full power of AI in their everyday work. Inside these pages, you'll find the tools and workflows that are already transforming how teams at Siili are working. We are constantly experimenting with these systems, testing their limits, and learning how to unlock their potential. We're now sharing those insights with you.

But make no mistake: this is uncharted territory, and the book offers just a snapshot. AI in software development is not a finished product. It's messy, sometimes unpredictable, and evolving faster than the policies or regulations that attempt to control it. In coming years ways of developing software will most likely change radically, we just don't know how yet. For those who are willing to embrace this wave, there are immense rewards. For those who resist, the future may look uncertain.

This book is designed to serve as a roadmap. It's not a complete guide to replacing developers with machines; it's one step on the journey of how to empower them to do more than they ever thought possible. We're not just teaching you how to use AI—we're showing you how to become AI-powered.

The wave is here. Let's ride it together.

Tomi Pienimäki

CEO of Siili Solutions

How to read this book

This handbook is designed to be as flexible as the AI tools it describes. You don't need to read it from cover to cover—unless you want to! Depending on your goals and current knowledge, you can choose a reading path that best suits your needs. Below are a few ways you might approach the book.

1. Sequential reading

If you're new to AI-assisted software development, reading from start to finish provides a comprehensive journey. You'll move from foundational concepts to advanced applications, making it ideal for those seeking a complete understanding of AI's role in software development.

2. Pick-and-choose based on need

You don't have to follow a strict order. If you're familiar with the basics or need specific information, feel free to jump to the chapters that interest you most. Each section stands alone, allowing you to access information on tools, workflows, or specific stages of development as needed.

3. Project-based approach

If you're working on a live project, focus on the chapters related to the current phase of your development. Whether you're in the requirements, coding, testing, or deployment stage, this approach helps you apply AI solutions directly to your workflow.

Conventions used in this book

Indicates code blocks

Indicates prompts for AI.



Indicates responses from AI.

TIP or NOTE

This element signifies a tip, suggestion or a general note.



About Siili

At Siili Solutions, we combine our expertise in software development with AI-driven solutions to help businesses reach their goals. We focus on practical innovation and collaboration, working closely with our clients to deliver solutions that meet their unique needs.

We specialize in AI-powered software development. We integrate AI tools throughout the entire software lifecycle—from design and coding to deployment and maintenance. Our approach brings together human insight and modern technology to create more efficient workflows and smarter outcomes.

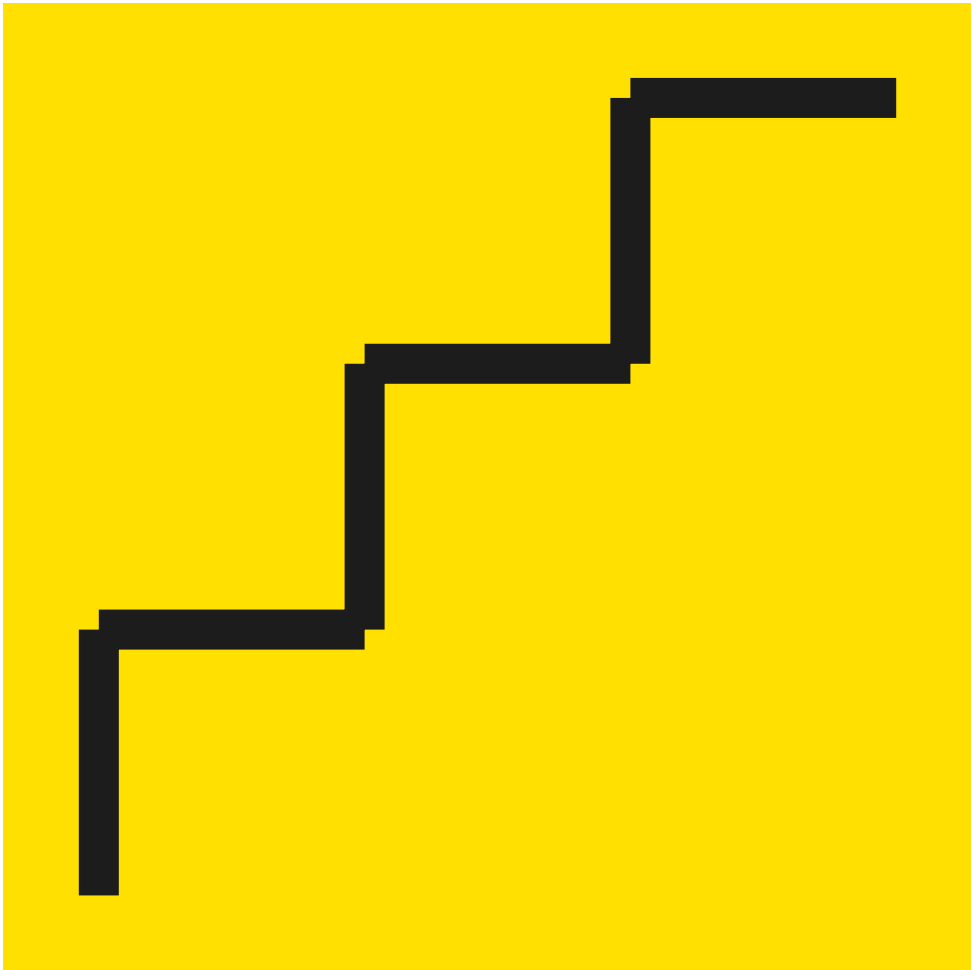
At Siili, we prioritize continuous learning and development. Our goal is to help our clients harness the power of AI to enhance their software development processes and achieve their business goals.



**MAKE
AI
REAL**

PART 1 •

Basics



Becoming AI-powered

Generative AI technologies are transforming software development. As AI tools become more sophisticated and accessible, they promise to revolutionize how we ideate, build, and maintain software products.

A key goal for using advanced tools for software development is **productivity** – the ability to create solutions faster and cheaper. There’s a growing body of research on this topic where results suggest significant improvements. For example, a recent study performed by MIT discovered an increase of 26% in software development¹. Similar results have been discovered in other studies including one we did at Siili Solutions in 2024. While this number is not yet 2- or even 10-fold, as some of the more optimistic predictions have suggested, we are certainly moving towards a significant productivity boost, especially when starting up new projects.

1) Cui, Zheyuan and Demirer, Mert and Jaffe, Sonia and Musolff, Leon and Peng, Sida and Salz, Tobias, The Effects of Generative AI on High Skilled Work: Evidence from Three Field Experiments with Software Developers (September 03, 2024). Available at SSRN: <https://ssrn.com/abstract=4945566> or <http://dx.doi.org/10.2139/ssrn.4945566>

We believe that AI can be more than a tool and will become a collaborator in the creative and technical processes of software engineering. This will transform how we do software development, from requirements gathering all the way to deployment. AI has the potential to not only enhance efficiency and improve quality, but it can also **unlock completely new possibilities in software development**.

However, this transition also brings challenges; therefore, a thoughtful approach is required to implement it.

In the following Chapters, we offer two approaches for this transition:

Roadmap: First, we share a maturity model for integrating AI into development practices. It considers the following questions: How should we change our ways as professionals and organizations? What kind of capabilities do we need to adopt?

Prospects of AI across the lifespan: We then offer an exploration of AI's potential roles throughout the software development lifecycle. We consider the following questions: What kind of tasks may be delegated fully or partially to AI tools? What needs to be designed, prototyped and rolled out to take full advantage of modern AI technology?

Our aim is to provide a comprehensive overview of how AI can be leveraged to transform software development practices and critically examine the practical considerations and challenges that come with this technological shift. Finally, we conclude this chapter with a summary discussion on challenges and considerations.

Roadmap to AI-powered software development

The first part of the transformation towards an AI-assisted software development life cycle is to recognize the new requirements for skills and organization. In the figure below, we have described this transition with a four-step maturity model framework, where each step represents the expected productivity boost and/or achieved level of automation offered by the new AI capabilities. We believe this multidimensional model serves as a useful tool to develop an AI-Powered Development Roadmap for an organization or a team looking to implement AI assisted development workflows.

The first two steps of this staircase – **Basic** and **Tailored** – focus on enabling an individual, such as a developer taking AI tools into use. The difference between the two layers resides mainly in the tooling required; imagine the Helper step as an ad-hoc smart search engine used sporadically to address specific questions or to explore new technology. Step up to Tailored level requires setting up tools that support the developer in their specific environment by creating a context for the AI tool to work in. An example tool for this level could be, for example, a private CustomGPT with a knowledge base of project context.

The next two steps move on to the domain of teamwork and integrated tools. We have named these steps **Integrated** and **Full Stream**. In these steps AI becomes more immersive and autonomous while additional tasks are delegated to it. The difference between these steps resides in

the level of automation or, more widely, ambition. AI-powered teams in the integrated level leverage AI tools as part of their daily work in single tasks, such as performing automated code reviews in isolation.

In the most ambitious full-stream state, the AI-powered steps become more integrated and reduce the need for humans in the loop significantly. Quite likely, building a complete AI development stack piece-by-piece yourself will remain unfeasible for most real-life projects and smallish software vendors, and the approach will become mainstream once the commercial offerings mature. Despite these reservations, demos like Devin and existing SaaS-style tools, such as v0 by Vercel, are taking steps in this direction.

We summarize the properties of each step along with required capabilities in the next page.

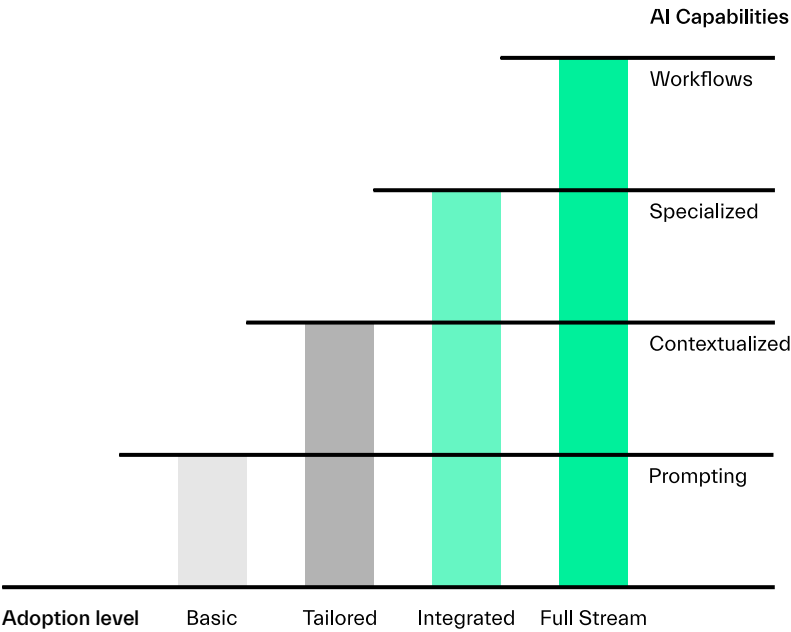


Figure 1. A four-step maturity model framework, where each step represents an expected productivity boost.

Level	Scope	Capabilities
Basic	Individual	Prompt-driven LLMs to solve problems
Tailored	Individual	Contextualized LLM/Generative AI setups with rich context, e.g. via RAG
Integrated	Team	Specialized AI-tools/agents focused on certain task, such as pull request reviews. Possibly fine-tuned models.
Full stream	Team	Integrated, orchestrated agents exchanging information covering several steps of the development stream

Figure 2. The properties of each maturity model framework step, along with required capabilities.

Reservations and guidelines for building your roadmap

Going up the AI-powered staircase requires careful planning and investment. In the following we will illustrate some of the practical challenges in implementing the changes.

Preparing special agents (sic!) for various tasks can be difficult in practice. For example, a special agent may require a special or expensive customized setup. It is important to focus on tasks where actual value and reliable outputs can be achieved. So start small and simple.

Data management of the shared context needs to be systematic. This requires that the project context submitted to agents is centrally maintained, ideally in a well-structured format in a central repository. The challenge bears a certain resemblance to the old dilemma of maintaining test databases with realistic data over time.

To illustrate the above with a simple example, consider an example testing agent that verifies your changes against a specification written in the form of a user story.

Here is an example user story benefit with an example case of developing a restaurant table reservation system (that we return to later in this book):

As a customer, I want to be able to book a table from a restaurant during the opening hours of the selected venue.
I want to be able to cancel the reservation later without needing to create a user account in the system

Quite typically, software requirements evolve over time, so in the next iteration the feature could be modified to be slightly different.

For example, let us imagine we get a change request for the user to be able to submit their allergies when making reservations. The new user story states just that

I would like to submit additional, such as allergies info when making the reservation

To be able to test the new version of our function, our AI would need to combine the previous story and the change ticket addendum. While this example might be addressable or even something you don't want to automate, it demonstrates the changing nature of the context, and importance of being realistic on how much and what you want to resolve with the help of AI.

Performance, cost and reliability issues. Adding advanced AI capabilities as part of an automated pipeline might lead to cost, performance and reliability issues. The reliability of an AI tool might not be good enough in advanced scenarios: either you get wrong answers or judgments, or the COTS model you have in the background is down when you are doing the deployment. To avoid costly mistakes, start with simple AI capabilities and review agents first in beta.

Towards a fully AI-powered development pipeline

Armed with the development roadmap illustrated in the previous Chapter, we will next investigate the prospects of AI capabilities to boost productivity and improve the lead times and quality of your development teams. We have divided these possibilities into three main development phases: **build**, **verify** and **run**.

An obvious starting point to utilize the AI capabilities is in the early stages of a software project. We refer to this phase as **build** later in this book. While using AI on an existing project is indeed possible, having the AI on board already in the requirements phase is both productive and helps in the later stages to provide context. By using natural language, diagrams and e.g. meeting transcripts, you may use AI to synthesize requirements, create user interface sketches, diagrams and other material to analyze and plan your work.

Once initial requirements and ideas are ready, you will enter the **verify** stage, where combined with the previously listed requirements, specifications and code, the changes and features that have been developed will be reviewed from several aspects, such as functionality, technology and data.

Finally in the **run** stage we deploy the verified artifacts, collect and compare data captured from the live system into reports, enhancement suggestions and feedback to be utilized in the next iteration.

We summarize this flow in the table in the next page. Each of the major stages is divided into several aspects – or AI Agents, if you will – with possible input and output material.

Ultimately these aspects may be integrated to form a Full Stream development experience. Imagine feeding the output of a Coder agent from the previous phase to a specifically instructed Reviewer agent. This could easily be done e.g. by having the Coder agent output a pull request to version control. The Reviewer agent would take the code changes and compare it with requirements (such as a user story), coding guidelines and other policies to perform a static review before any code gets accepted.

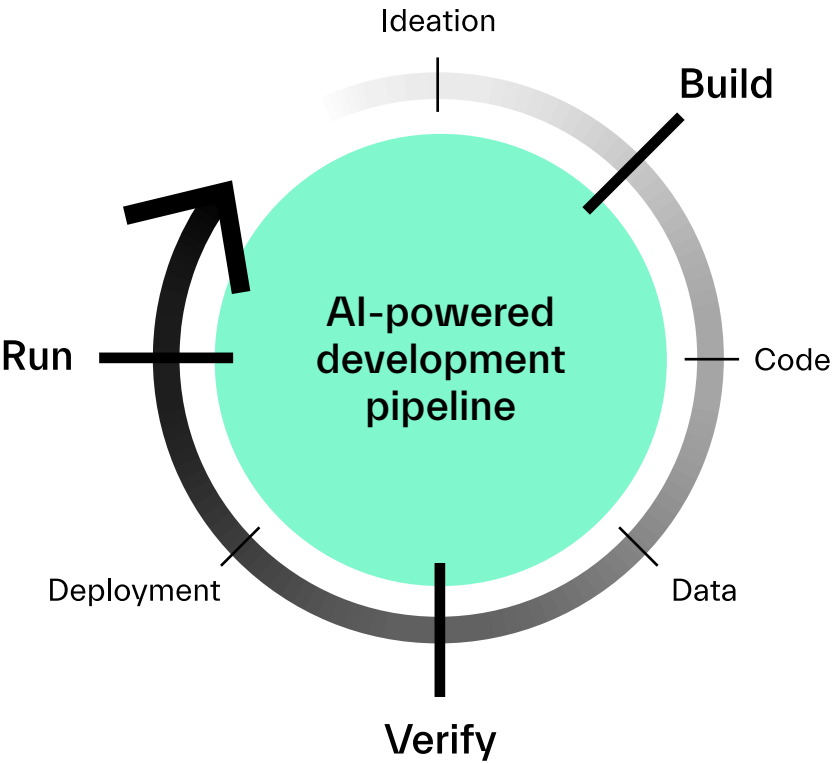


Figure 3. AI-powered development pipeline

Phase	AI Agent	Input	Output
Build	Requirements	User stories, market research, stakeholder interviews	Detailed requirements document, updated user stories, acceptance criteria
	Design	Requirements document, design scenarios, initial sketches	UX proposals, technical alternatives (adrs), prototypes, api definitions, database schemas
	Coding	Finalized requirements, existing code base, libraries	Refactored code, database modifications, automated unit tests, new pull requests
	Review	New code changes, pull requests, review guidelines	Code review approvals, change requests, improvement suggestions
Verify	Infrastructure	Infrastructure change requests, compliance policies	Infrastructure change approvals, updated ci/cd pipelines, infrastructure as code (iac) updates
	Security	Security policies, pull requests, threat models	Security audit reports, identified vulnerabilities, remediation plans
	Data	Data migration plans, database change requests, data governance policies	Reviewed data migration scripts, database integrity reports, data quality assessments
	Quality Assurance	Test data sets, defined test cases, change logs	Executed automated test cases, impact assessment reports, identified bugs
Run	Deployment	Deployment artifacts, approved infrastructure changes, operational policies	Successful deployments, deployment notifications, updated change logs, rollback plans
	Monitoring	Runtime monitoring logs, audit logs, system performance metrics	Comprehensive monitoring reports (covering data integrity, security, and system performance), incident reports, behavior analysis
	Maintenance	Maintenance logs, current code base, issue backlogs	Completed bug triages, predictive maintenance recommendations, system updates
	Feedback	User feedback, backlog items, performance reports	Iteration summaries, sentiment analysis reports, product improvement proposals

While integrating these aspects into a single automated workflow is indeed very ambitious it is certainly a path several tool vendors are actively pursuing. At the time of writing this book several aspects of it have already been partially implemented in e.g. GitHub Copilot Work-space.

We will cover several of these aspects later in this book.

The value proposition of getting AI-powered

Ever asked for a fixed quote from a software company to ship a new product? That's right. Chances are they you only get hourly rates. Similarly, getting a hard deadline is difficult.

The core reasons behind this vagueness are related risk management and the still chaotic nature of our craft. While efforts such as Agile have tried to make the loop times shorter or predictable, (and there are persistent success story rumors), the challenge remains: **are we going to invest money in something with unclear scope, price or schedule.**

Software development has been always hard, especially at scale. Advances in programming languages, frameworks, patterns, cloud and other areas have made it progressively easier to build powerful systems relatively quickly, but the basic challenges remain. This brings us to the theme of the book: will the AI address the productivity and predictability challenges of our industry?

In the diagram in the next page, we aim to answer this question by breaking down the prospects of AI-powered development when compared to the old world.

Some of the biggest, even **game-changing** benefits we currently see are perhaps modelling – data and APIs – and rapid prototyping. You can get very quick starts. On the other end, certain task types of tasks AI can already be considered helpful but still far from being a game change.

For instance, designing nice user experience, infrastructure and architecture remain areas where you can delegate some of the heavy lifting to the AI tools, but still need to do most of the stuff yourself.

The bulk of the work relies in a very traditional code-test -cycle, where AI tools are indeed very useful and can provide significant boost. Perhaps not the 10x already touted, but sometimes near that. We're yet to see if or when they turn into game changers: in some settings and limited projects they might already be that.

Use cases of AI for all the above topics are provided in Part 2 of this book.

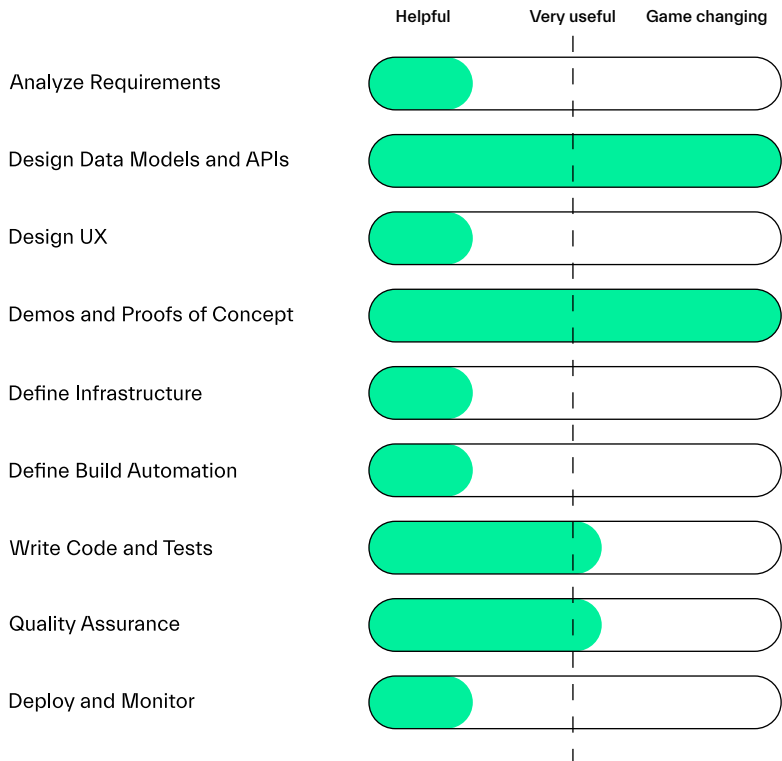


Figure 4. AI's impact across software development tasks

Challenges and considerations in AI adoption

In the previous chapter, we presented a novel framework to build an AI-powered roadmap along with a detailed list of AI prospects in the development stream. While all this is roughly doable even with the tools we have today, the field itself is evolving rapidly, especially when it comes to integrated tooling.

The main issues affecting the construction of this roadmap are:

Increased integration pushes technology in the background

We expect that AI will cover more of the traditional work phases than just coding. In a way this could be compared to making more effort towards low-code/no-code platforms.

Tool selection remains a challenge

Early adopters and pioneers aside, the expected productivity boost requires that most developers, or entire teams, adopt more AI capabilities. Considerable investment in tooling and training will be required in addition to subscription fees, and this process will also take time. Picking the correct toolsets in an unclear and rapidly changing market situation is a real challenge, if we consider the real-life practical issues when introducing any new technology or working methods.

Any new tool or software development needs to

- be stable and capable enough and
- have a reasonable market share, support and future and
- provide reasonable ROI on the required trials, trainings and integrations

Addressing these considerations is highly advisable before jumping in.

Other challenges of AI-assisted tooling include topics such as privacy and security, ethical considerations and problems that are related to the nondeterministic nature of certain AI models. We shall explore the last as it is arguably the most relevant when considering software development.

You have probably seen – and will discover later in this book – impressive examples of code generation capabilities offered today. In a real-life setting, especially over the long term, the reality is not perhaps as rosy. AI will for example hallucinate i.e. invent things out of thin air and may propose non-optimal or even outright wrong solutions. The AI's troubleshooting and proposed refactoring may not be correct and may result in hard-to-manage code that nobody understands. Similarly, while adopting a new stack just by prompting AI and letting it generate code for you opens interesting prospects, maintaining the code you do not know or understand might not be easy.

Addressing these issues takes practice and caution. You are still required on the driver's seat, and you need to pay attention to your surroundings – in a way even more than you are used to. And it is entirely possible that in certain settings and especially with skilled people AI might not produce the expected boost.

Conclusion

As we have explored briefly in this chapter, integrating AI into software development presents both exciting opportunities and significant challenges. The roadmap we presented – from Helper to Full Stream – offers a structured approach to adopting AI technologies, allowing teams and organizations to gradually increase their AI capabilities and reap the benefits at each stage.

The prospects of AI across the build, verify, and run phases of development demonstrate the potential for AI to revolutionize every aspect of the software lifecycle. From requirements gathering to deployment and maintenance, AI tools and agents can enhance efficiency, improve quality, and unlock new possibilities in software creation.

However, this transformation is not without its challenges. Tool selection, integration, and the need for new skills and organizational changes are all significant considerations. Organizations must balance the potential benefits with the practical realities of implementation especially as the

AI development field is evolving rapidly.

Ultimately, the journey towards AI-powered software development is continuous learning and adaptation.

Short Guide to AI: Introduction

AI-assisted software development represents a significant paradigm shift in computer programming. By integrating Large Language Models (LLMs) into the development process, we are witnessing a transformation in how code is written, debugged, and optimized. This handbook aims to explore the various facets of this emerging technology and its impact on software engineering practices.

To understand the role of AI in software development, it's useful to draw an analogy with traditional computer architecture:

- **LLM as CPU:** The Large Language Model functions as the central processing unit, handling complex computations and generating code based on inputs and context. Using AI to generate the computer code can be understood as a very high-level programming language – instead of typing that Java, you may just use English.
- **Context window as RAM:** The context window of an LLM can be likened to Random Access Memory (RAM). It holds the immediate context and information necessary for the model to generate relevant outputs. Just as RAM size affects a computer's performance, the size of the context window influences an LLM's capability to handle larger and more complex coding tasks.

- External knowledge as storage: When LLMs access external documentation, codebases, or use techniques like Retrieval-Augmented Generation (RAG), this process is analogous to fetching data from long-term storage, like a hard drive or solid-state drive in a computer system.

This handbook will look into these components and their interactions, exploring how they come together to create a powerful tool for modern software development. We will examine the current state of AI-assisted coding, its practical applications, limitations, and potential future developments.

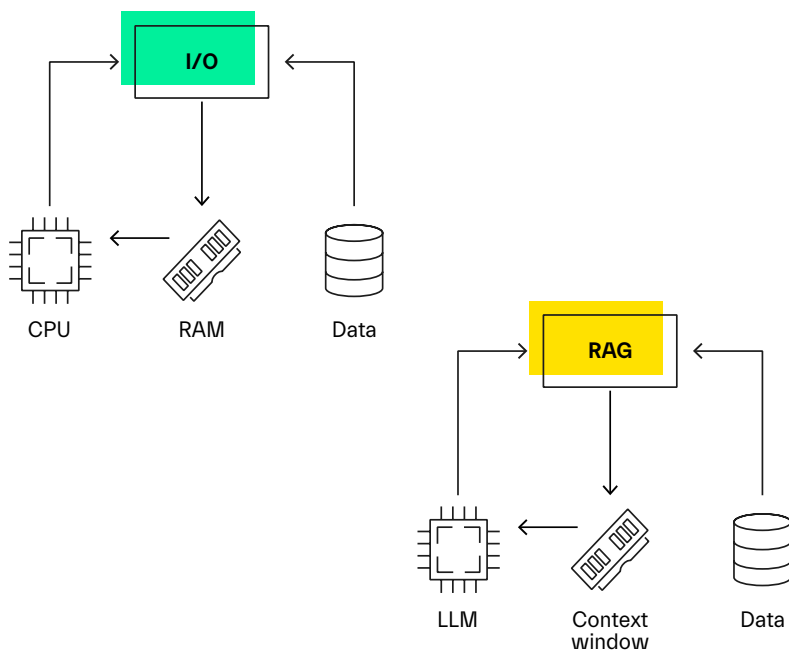


Figure 5. Analogy between traditional computer architecture and AI systems, comparing CPU/RAM to LLM/context window with RAG for data retrieval.

Context windows

Context windows in Large Language Models (LLMs) function analogously to RAM in computer systems, determining the amount of text an LLM can process simultaneously. This directly impacts the model's ability to maintain coherence and relevance in its outputs. Recent developments in context window sizes are significantly changing AI-assisted software development.

NOTE: HumanEval is a benchmark for evaluating the coding abilities of language models, measuring their performance on a set of Python programming tasks.

Key observations:

Context window growth: Context windows have expanded significantly, from GPT-4's 8.2k tokens to Gemini 1.5 Pro's 2 million tokens. This increase allows for processing larger chunks of code and documentation simultaneously.

Cost considerations: Processing entire codebases with these models can be expensive. The original GPT-4, while highly capable, costs \$30 per million tokens. However, newer models like Claude 3.5 outperform them while being ten times cheaper.

Efficient alternatives: "Small Language Models" (SLMs) and budget-friendly options are becoming more prevalent. GPT-4o mini, for instance, offers a 128k context window and a respectable 87.2 HumanEval score for only \$0.15 per million tokens.

Performance and efficiency: Claude 3.5 Sonnet currently leads in performance, surpassing the original GPT-4 with a 92.0 HumanEval score. It combines a large 200k context window with a more affordable price point of \$3 per million tokens.

Implications for Coding Assistants:

Comprehensive code understanding: Larger context windows enable coding assistants to analyze and understand more extensive codebases, potentially improving their ability to provide context-aware suggestions and refactoring recommendations.

Cost-effective development: Models like GPT-4o mini could be used for routine tasks or as efficient context retrievers for more complex models, optimizing the balance between performance and cost in development workflows.

Enhanced documentation and code generation: With models capable of processing entire projects, coding assistants can generate more accurate and context-appropriate documentation, as well as suggest larger code blocks or entire functions that fit seamlessly within the existing codebase.

Improved code review: The ability to process larger contexts allows for more thorough automated code reviews, potentially identifying issues related to overall architecture and design patterns that smaller context windows might miss.

Specialized model selection: Developers and organizations can now choose models that best fit their specific needs, balancing factors like context size, performance, and cost based on project requirements.

Outlook: As the field progresses, we can expect further advancements. The Claude 3.5 Opus model is expected to offer superior intelligence, albeit at a higher cost. This continuous evolution suggests that coding assistants will become increasingly sophisticated, capable of understanding and generating code at unprecedented scales.

Model Name	Context Window Size	Lines of Code (20k tokens = 1k lines)	Approx Input Cost per 1M Tokens	HumanEval
GPT-3.5 Turbo 16k	16k	1.6k	\$3	73.2
GPT-4	8.2k	820	\$30	88.4
GPT-4 Turbo	128k	12.8k	\$10	85.4
GPT-4o	128k	12.8k	\$5	90.2
GPT-4o mini	128k	12.8k	\$0.15	87.2
Claude 3.5 Sonnet	200k	20k	\$3	92.0
Claude 3 Haiku	200k	20k	\$0.25	75.9
Claude 3 Opus	200k	20k	\$15	84.9
LLaMA 3.1 401B	128k	12.8k	\$5.17	89.0
LLaMA 3.1 70B	128k	12.8k	\$0.88	n/a
LLaMA 3.1 8B	128k	12.8k	\$0.18	n/a
Gemini 1.5 Pro	2M	200k	\$3.5	84.1
Gemini 1.5 Flash	1M	100k	\$0.35	n/a

Figure 6. The current state of LLMs. HumanEval is a benchmark for evaluating coding abilities of language models, measuring their performance on a set of Python programming tasks.

Effects of large context windows

Large context windows allow models to ingest entire codebases, including documentation and configuration files, enabling a holistic understanding of project architecture and interdependencies. This comprehensive view empowers models to solve complex, non-isolated problems that span multiple files or modules a task that previously required breaking issues into smaller, isolated chunks.

This capability significantly reduces reliance on Retrieval-Augmented Generation (RAG) for many tasks, as models can often provide relevant information without needing to retrieve additional context dynamically. The enhanced understanding translates into improved code generation, debugging, and error analysis across the entire project. Models can propose entire functions or classes that align with the existing style and architecture, analyze errors in a broader context, and perform more thorough code reviews.

While these advantages come with increased computational costs and potential for information overload, they move us closer to AI assistants that can “understand” a project holistically, much like an experienced developer with intimate knowledge of the entire codebase.

Retrieval-augmented generation (RAG)

Retrieval-Augmented Generation (RAG) is a technique that enhances the capabilities of language models by dynamically fetching relevant information from an external knowledge base. If we extend our earlier analogy of the language model as a CPU and the context window as RAM, RAG can be thought of as loading data from storage into RAM.

While the evolution of larger context windows has made it technically possible to ingest an entire codebase in a single query, this approach is often cost-prohibitive for frequent use. Dumping the entire codebase into the context is a resource-intensive operation that should be limited to infrequent, perhaps daily, updates. For day-to-day development tasks, more selective and efficient RAG techniques are necessary.

The importance of efficient context provision

Providing proper context for language models is essential for their effective use in software development. Without adequate context, AI assistants are limited to solving problems within a narrow scope, often restricted to single files or small code snippets. This limitation can severely hamper their ability to assist with tasks that require a comprehensive understanding of the codebase.

To overcome this limitation while maintaining cost-effectiveness, several approaches can be employed by AI-powered development tools to build and provide context:

- **Currently open files:** The simplest approach is to include the content of all currently open files in the development environment. This method works well for small projects but may become unwieldy for larger codebases.
- **IntelliSense-based dependency understanding:** Leveraging the IDE's IntelliSense capabilities, this method automatically identifies and includes relevant dependencies and related files in the context.
- **Semi-automatic context building:** This approach allows the tool to suggest files for inclusion in the context, with the developer having the final say. The tool can also offer options to include only headers and public methods or the entire file content.
- **Cascade with small language models (SLMs):** In this method, an SLM first reads through the entire codebase, deciding which parts are most relevant to the current task. This curated context is then provided to the larger language model.
- **Agentic approach:** Starting with basic information (e.g., headers of all classes), an AI agent within the tool progressively explores the codebase, visiting additional files and building context as needed.

- **Vector search:** This technique involves converting code snippets or documentation into vector representations and using similarity search to find the most relevant pieces of information for a given query.
- **Graph search:** By representing the codebase as a graph (with files, functions, and classes as nodes, and dependencies as edges), this method allows the AI tool to navigate the codebase structure to find relevant context.
- **Context caching:** When similar requests are made repeatedly, context caching allows the system to reuse the cached tokens instead of reprocessing the entire context each time. This reduces the costs and speeds up response times, making it feasible to make repeated queries with large amount of code in context.

Understanding how these context-building mechanisms work can significantly enhance a developer's ability to leverage AI tools effectively. It allows developers to make informed decisions about which tools to use for specific tasks and how to structure their codebase to maximize the benefits of AI assistance. Moreover, this knowledge can help developers troubleshoot when AI tools provide unexpected or incorrect responses, as the issue may lie in the context provided rather than in the AI model itself.

Hallucinations

Hallucinations in AI occur when language models generate inaccurate or fabricated information, stemming from their probabilistic nature and inability to distinguish fact from fiction. A persistent challenge is the AI's tendency to be confidently incorrect, presenting hallucinated information with the same assurance as factual data. However, the impact of hallucinations has diminished, as modern models hallucinate less frequently than GPT-3.5 era models.

In code generation, hallucinations might manifest as non-existent methods or incorrect syntax. For example:

```
import pandas as pd

df = pd.read_csv("data.csv")
result = df.interpolate(method='cubic',
                        order=3)
print(result)
```

This subtle hallucination combines two real pandas methods, with the non-existent `order` parameter for cubic interpolation. Most IDEs would immediately flag this error, highlighting the relatively low risk of such hallucinations in practice.

The impact of code hallucinations is often less severe than in other domains. Unlike hallucinated legal clauses or medical advice, erroneous code typically triggers immediate warnings or fails fast during execution.

Emerging agentic workflows further mitigate risks by incorporating linting, compiling, and testing steps before presenting code to users. While not yet mainstream, these tools show promise in managing AI-generated code quality. Nevertheless, developer vigilance and verification of AI-generated content remain crucial in AI-assisted development.

Custom instructions and prompt engineering

Custom instructions and prompt engineering are tools for tailoring AI language models to your specific needs. Understanding when and how to use them can significantly improve your development workflow.

Everyday usage: Ad-hoc and context-free

For most ad-hoc interactions with AI assistants, complex prompt engi-

neering is not necessary or particularly useful. These are the quick, isolated questions you might ask throughout your day:

- “How do I center a div in CSS?”
- “What’s the syntax for a Python list comprehension?”
- “Explain the difference between let and const in JavaScript.”

In these cases, approach the AI as you would a knowledgeable colleague. State your question clearly, and if the first response is not quite what you need, simply rephrase or ask follow-up questions. This iterative approach is often more efficient than trying to craft the perfect prompt.

Custom instructions: Setting the stage

Paid services like ChatGPT and Claude offer the ability to set custom instructions. Think of these as setting up your development environment: you do it once (with occasional updates) to streamline all future work. They’re particularly valuable when:

- You have a **consistent working context** (e.g., specific tech stack or coding style)
- There are important **project-specific details** the AI should always know
- You frequently perform **similar types of tasks**

Scenarios for custom instructions

Project-specific assistant

Create a custom GPT or Claude project for your codebase. Include: – README contents – Project structure information – Style guides – Version and dependency details – Relevant sections of the codebase.

This creates a knowledgeable “team member” that understands your project’s context and can provide more accurate assistance. Interacting with this setup becomes part of your daily workflow, distinct from ad-hoc queries.

Specialized task helper

For repetitive tasks like updating translation files, set up custom instructions that include:

- The specific file format you use,
- Existing translations for context,
- Key terms with their required translations and
- Any project-specific translation rules.

This ensures consistency and reduces the need to repeat this information in every interaction.

The value of context

While ad-hoc queries do not require extensive context, your project-specific or specialized task interactions benefit greatly from it. The more specific and repetitive the scenario, the more valuable well-crafted instructions become.

Custom instructions bridge the gap between the AI's broad knowledge and your specific needs. They are not about automating your job but about creating a more effective tool that understands your work context.

By investing time in setting up these instructions for your common scenarios, you create a more efficient workflow. This allows you to focus on the complex, creative aspects of development where human expertise is irreplaceable while still having a powerful, context-aware AI assistant for your routine tasks.

Security and data protection in AI-assisted development

Code vs. data: A critical distinction

It's crucial to understand that code is not data in the traditional sense and does not fall under regulations like GDPR. In fact, code is often less sensitive than those responsible for it might assume. From a security standpoint, the risks associated with AI tools "seeing" your code are often overblown.

The likelihood of proprietary code being meaningfully compromised through AI tools is minimal. This paranoia echoes the unfounded fears that surrounded the move to cloud computing years ago.

The security trade-off

Security is always a compromise between protection and convenience. Prohibiting developers from using AI tools or copy-pasting code into LLMs is a poor trade-off. Such restrictions significantly hamper development speed while addressing risks that have an extremely low probability of materializing and very low impact.

The real risks

The non-deterministic nature of LLMs means it's technically possible for them to inject malicious code, but this risk is negligible – akin to a tennis ball quantum tunneling through a wall. A more significant but still negligible risk is developers finding compromised answers on

platforms like Stack Overflow or coding blogs that contain subtle but malicious code.

While it's true that AI can hallucinate and make mistakes, so can humans. The responsibility for AI-generated code lies with the development team using AI to write it. This is no different from the responsibility developers have when using any other tool or resource.

The risk of legal action resulting from using AI-generated code that someone else has copyrighted is theoretical and extremely low. OpenAI even offers indemnification protections for its API and enterprise customers.

Interestingly, many clients won't bat an eye if you copy and paste code in Slack to your teammate, even though Slack's EULA allows them to use data in training models (which Sili has opted out of). In contrast, most AI tool EULAs – especially when accessed via API – do not allow the use of your data for training.

Conclusion

The use of AI in software development doesn't introduce fundamentally new security risks. Instead, it amplifies existing concerns and challenges us to rethink our approach to code security. As with any tool, the key lies in responsible use, clear policies, and ongoing education about the real risks and benefits of AI-assisted development.

Mindset: Integrating AI in development

The bell curve meme (Figure 7.) humorously illustrates a crucial point about AI integration in software development: contrary to what cautious skeptics might believe, extensive use of AI tools like ChatGPT can lead to significant productivity gains. The meme suggests that both naive users and experienced developers end up heavily relying on AI, albeit for different reasons.

Maximizing AI usage for efficiency

The key insight is that as developers gain understanding and experience, they should aim to offload as much work as possible to AI systems. This approach, far from being detrimental, can dramatically increase productivity. The primary benefit of AI in software development is efficiency. Whether it's generating boilerplate code, suggesting complex algorithms, or rapidly prototyping ideas, AI consistently outpaces human speed in many tasks. By leveraging AI's vast knowledge and quick processing, developers can accelerate their workflow significantly.

AI as the first step

Artificial Intelligence should be the initial resource for any intellectual task in the development process. This approach might seem counterintuitive to those who believe in solving problems independently, but it's ultimately about efficiency. Even if you can solve a problem by yourself, chances are you'll spend more time and mental effort than if you had consulted AI immediately.

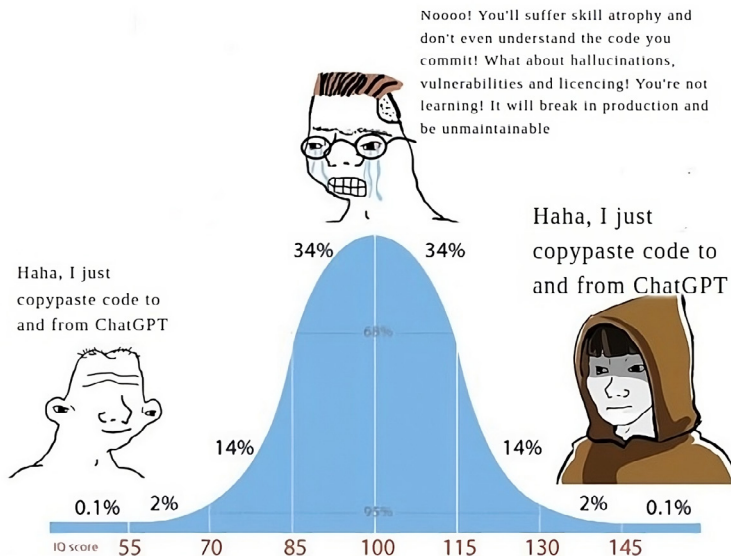


Figure 7. The bell curve meme above humorously illustrates a crucial point about AI integration in software development

By involving AI systems at the earliest stages of problem-solving, developers can quickly generate initial solutions or explore multiple approaches. If AI provides a suitable solution immediately, much time and mental effort can be saved. If not, the developer can rapidly iterate, refining the problem statement and explaining why the AI's approach is incorrect. This method allows for fast exploration of potential solutions, often avoiding time-consuming dead ends.

Developing AI-augmented skills

Effective AI integration requires skills like those of a lead developer, particularly in delegation and oversight. Viewing the AI as a capable team member allows developers to offload appropriate tasks while maintaining overall project direction and quality control. This mature integration of AI tools requires understanding both the problem domain and the behavioral patterns of Large Language Models (LLMs).

A critical soft skill for both traditional and AI-assisted development is the ability to ask for help early and often. In human teams, developers sometimes hesitate to seek assistance, fearing they might disturb colleagues or appear incompetent. This reluctance can lead to wasted hours struggling with problems that could be quickly resolved with a simple question.

AI assistants fundamentally change this dynamic. With AI, the traditional barriers to seeking help vanish entirely. AI is always available, infinitely patient, non-judgmental, and assisting you doesn't pull it away from other tasks. This means developers can and should leverage AI assistance for even the most mundane issues without hesitation.

By treating AI as an infinitely patient, knowledgeable resource – essentially, the world's most advanced rubber duck – developers can significantly enhance their productivity and problem-solving capabilities. The key is to rewire your instincts: instead of spending even five minutes puzzling over something, consult your AI assistant first.

This approach doesn't just save time on individual tasks; it accelerates the entire learning process. By getting immediate answers and explanations, developers can maintain their flow state and rapidly assimilate new information.

Balancing AI use and skill development

The goal of AI integration is not to create dependency but to use it as a catalyst for personal growth and efficiency. Mastery of this mindset allows developers to keep pace with rapid technological advancements, leverage AI to enhance their own capabilities, and focus on higher-level problem-solving and creative tasks.

Concerns about skill atrophy due to AI overreliance are often overstated and reminiscent of past arguments against calculators or internet search engines. Current AI technology, while powerful, is not yet advanced enough to completely supplant human learning and under-

standing. Even if a developer actively tried to avoid learning while using AI to solve problems, some degree of knowledge absorption would inevitably occur.

Moreover, AI serves as an exceptional learning tool, particularly when exploring new technologies or complex subjects. It can provide personalized explanations, offer diverse perspectives, and help break down complex topics into manageable chunks. This ability to rapidly access and assimilate information can accelerate skill development rather than hinder it.

The “skill atrophy” argument also overlooks the evolving nature of valuable skills in the tech industry. As AI handles more routine tasks, the most crucial skills for developers shift towards areas like system design, problem formulation, and creative solution generation – skills that are enhanced, not diminished, by effective AI collaboration.

By embracing extensive AI usage, developers can dramatically increase their productivity and focus on more complex aspects of software development that truly require human insight and creativity. The key is to understand that efficiency gained through AI use doesn’t diminish the developer’s skills, but rather allows them to apply those skills to more challenging and innovative aspects of their work.

AI across developer levels: Shattering myths and enhancing skills

The notion that AI primarily benefits junior developers is a myth ripe for debunking. While AI can swiftly solve isolated problems juniors often face, its impact reaches across all seniority levels:

- **Context complexity:** Junior issues tend to be more isolated, making them perfect AI fodder. Senior challenges often involve complex contexts – but with expanding context windows and smarter IDE assistants, AI is increasingly capable of tackling these too.

- **Adaptation advantage:** Seniors aren't inherently better at using AI. Juniors, unencumbered by years of set practices, often adapt more readily to AI-augmented workflows.
- **Productivity boost across the board:** AI amplifies productivity at all levels. For juniors, it's a low-hanging fruit. For seniors, it requires more finesse but offers substantial gains.
- **Skill amplification:** AI doesn't just solve problems; it amplifies skills at all levels, pushing everyone to elevate their game.

Level	Do more	Do less
Juniors	Rapid skill acquisition through AI Using AI for code review before PRs Sharing AI techniques with the team	Bothering seniors Blindly copy-pasting ChatGPT code Focusing on speed over learning
Mid-levels	Boosting productivity with AI assistance Tackling senior-level challenges with AI	Grunt work Sticking to familiar, routine tasks
Seniors	Delegating to AI Adopting AI and building custom tools Guiding team to use AI Guiding stakeholders to use AI Challenge your thinking	Manual work Delegating easy tasks Helping team members with easy stuff Sitting in meetings Assuming your old ways are optimal

Figure 8. While AI can swiftly solve isolated problems juniors often face, its impact reaches across all seniority levels.

Juniors

As a junior developer, AI can be a powerful ally in your learning journey. Use it to boost your understanding, improve your code, and enhance problem-solving skills. But remember, AI is a tool for growth, not a shortcut.

Here's how to make the most of it as you build your career.

- **Embrace AI but prioritize understanding:** Copy-pasting without comprehension is a recipe for disaster. Use AI-generated code as a learning tool, not a crutch.
- **AI first, humans second:** Always consult AI before asking teammates. This builds your problem-solving skills and reserves human bandwidth for complex issues.
- **Turbocharge your learning:** Use AI to understand more, faster. Ask it to explain concepts, provide examples, and challenge your assumptions.
- **Leverage AI for code review:** Before making any PRs, have AI review your solutions. This develops your critical thinking and improves code quality.
- **Share your AI learnings:** As you become proficient, share your AI techniques with the team. You might outpace seniors in AI usage, so don't hesitate to teach upwards.

Mid-levels

For developers with some experience under their belt, AI offers a unique opportunity to boost productivity and take on more complex challenges. Your focus should be on leveraging AI to enhance your output, tackle senior-level tasks, and refine your workflow. While AI can handle many routine tasks, continue honing skills that complement AI, such as system design and optimization, to stay ahead and grow in your career.

- **Boost productivity exponentially:** This is your focus. Use AI to do more, faster, and with higher quality. Your value lies in output, not just skills.
- **Push your boundaries:** Tackle senior-level challenges with AI assistance. Use it to expand your problem-solving repertoire and take on more complex projects.
- **Optimize your workflow:** Shift your workflow from think → google → AI to AI → think → google. Set time limits for problem-solving before consulting AI.
- **Elevate your skills:** While AI can handle many tasks, focus on developing skills that complement AI. This includes system design, optimization, and integrating AI into larger solutions.

Seniors

As a senior developer, delegate tasks to AI, lead AI integration efforts, and shift your focus to strategic oversight. Use AI to challenge your assumptions, bridge the gap between AI and business, and mentor others in effective AI usage. Preserve and transfer crucial knowledge by creating and maintaining AI-enhanced tools and strategies.

- **Master delegation:** Your primary focus should be delegating to AI. Use it to handle tasks you could easily do yourself, including work you'd normally delegate to juniors. There are no code monkeys in AI-powered teams.

- **Guide AI integration:** Become the AI integration strategist for your team. Develop AI-enhanced tools like Custom GPTs and review agents. Stay prepared to adopt new tools as they mature, especially agentic systems.
- **Redefine your role:** Shift from hands-on coding to strategic oversight. Focus on system architecture, AI integration strategies, and pushing the boundaries of what's possible with AI.
- **Challenge assumptions:** Use AI as your devil's advocate. Let it challenge your designs and decisions before the real world does.
- **Bridge the AI-business gap:** Educate stakeholders on AI's potential and limitations. Be prepared to tackle red tape and stakeholder paranoia about AI adoption.
- **Evolve mentorship:** Instead of teaching basic coding, guide juniors and mid-levels in effective AI use, problem framing, and solution evaluation.
- **Preserve crucial knowledge:** Develop strategies for knowledge transfer in an AI-augmented environment. Focus on building quality knowledge bases (e.g., ADRs) and use AI to reduce friction in maintaining them. Create custom AI tools that have this institutional knowledge in context.

AI tools in software development: A practical guide

This chapter examines the AI tools reshaping software development. We'll explore conversational interfaces like ChatGPT and Claude, as well as integrated coding assistants such as GitHub Copilot and JetBrains AI. Each tool offers unique features to enhance productivity across various development tasks.

From copy & paste to fully integrated

In a wider context and before jumping into details, let's explore the evolution of AI-related tools and their properties a bit. In a way, we've recognized three major steps being taken in the last few years representing the transition from first text-based offerings into fully integrated solutions where the code and underlying AI fade into the background.

This evolution is summarized in the table below, where starting with simple request-response style interaction within a browser you end up with fully integrated environment which just expects you to tell you what you want, and the technology will take care of the rest.

Let's discuss the findings of this table in the next chapters.

Task	Prompting	Developer	Integrated
Telling what to do	Input by the user. "Given this file, write unit tests"	Context-aware tool-assisted help. Add Tests-menu option on a file open in IDE.	In a feature level: "Add a new button to my order completion to send feedback about the experience"
Providing context	Upload files, images Knowledge bases RAG	Automatically the project you are working on, current file	Automatic
Verifying and applying results	Manual copy/pasting-style by you	Integrated options to review and apply proposed changes and new items	Automatic or semi-automatic

Figure 9. Starting with simple request-response style interaction within a browser, you end up with fully integrated environment.

Prompting: First conversational interfaces

The foundation of all AI tools lies in their underlying language models. Typically, you interact with them by typing your query or problem into a web portal, such as the one offered by OpenAI. These interfaces have improved rapidly and often include capabilities like data- and context management. In addition to written text, you can attach images or documents to your prompts, and even speak to AI.

To this day prompting is the primary interface most people think of when AI is mentioned when wondering about questions like:

Did you use ChatGPT to write this book?

Developer tools

Prompting with your browser gets a bit tedious when using it in daily software development tasks. Basically, you need manually transfer the AI outputs to whatever you are working with.

Luckily more integrated tools have emerged. They are typically integrated into some popular software, such as Visual Studio Code, with these tools it's way easier to interact with the models, verify and apply their recommendations to your code, files and data or may be run inside a browser where code and data is stored in the network.

These tools include products such as GitHub Copilot and Cursor.

Integrated solutions

Fully integrated, sometimes purpose-build environments take the game much further. Instead of necessarily even touching the code, the user focuses on validating the working software and modifying it directly by advising tool rather than coding, deploying and running the code by himself. Examples of integrated environment include solutions such as OpenHands, v0 by Vercel, and others.

This class of tools does indeed appear like the end game of this evolution. The current capabilities are still quite limited no matter how impressive demo's you've probably seen on the Internet, and often limited to most typical applications, such as simplistic games or CRUD-style web applications (ability to read, update, insert and delete data from your application's data model).

As you probably know, AI-powered tools are rapidly evolving and will continue to do so. The aim of this book is to give you an outlook on what's available today.

Integrated solutions are discussed later in this book in Chapters covering coding agents and in the closing chapter that discusses the future of the AI development.

Prompting tools

Prompting tools are AI-driven platforms or applications designed to generate responses or actions based on user inputs, which are typically given as prompts (questions, commands, or requests). These tools use models to understand the user's intent and provide relevant outputs.

Prompting tools are commonly used in AI systems, especially in language models, to assist with tasks such as writing, coding, data analysis, and content creation. The user inputs a prompt—such as “Write a Python function to sort a list,” and the tool generates an appropriate response or action.

It is important to note that prompting is way more powerful than searching for the same information – and it's much more than getting isolated answers to single questions. Consider prompting as discussions; whatever you said – and what AI said – will be remembered.

For example, imagine the following discussion, where you can refer to the previous answer directly by just asking more.

Write a Python function to sort a list

And we got:



```
def sort_list(input_list):  
    # Sort the list in ascending order  
    input_list.sort()  
    return input_list
```

We realize that we might need to sort also in descending order, so let's ask:

Make the sort order configurable

And AI was able to understand that it's the previous response we were referring to.



```
def sort_list(input_list,
              ascending=True):
    # Sort the list based on the ascending
    parameter
    input_list.sort(reverse=not ascending)
    return input_list
```

Prompting techniques, or prompt engineering are not in the scope of this book but will be demonstrated later in Part 2 of the book.

ChatGPT

ChatGPT, released by OpenAI in November 2022, is a conversational AI platform based on the GPT (Generative Pre-trained Transformer) language model architecture. It quickly gained widespread adoption, becoming a standard tool for various text-based tasks. OpenAI regularly updates ChatGPT, expanding its capabilities and addressing limitations.

Model Comparison

ChatGPT offers five models: GPT-4o, GPT-4o Mini, GPT-4-Turbo (now listed as legacy model), and the recently added complex reasoning models o1-preview and o1-mini.

GPT-4o leads OpenAI's models in benchmarks. However, it's losing ground to Claude 3.5 Sonnet in many tests. Despite tied scores in some areas, the consensus among users is that Claude 3.5 Sonnet is the superior model.

While GPT-4o outperforms GPT-4-Turbo in benchmarks, it's widely seen as a side grade rather than a clear upgrade. It excels in some areas but falters in others. A notable drawback is its tendency to get stuck in loops, particularly when iterating code – it may revert to original, problematic solutions even after issues are pointed out. Users also report that GPT-4o can feel more “soulless” or mechanical compared to GPT-4-Turbo.

GPT-4o Mini offers a cost and speed advantage over both GPT-4o and GPT-4-Turbo. However, these benefits mainly manifest when using the API, and there isn't much reason to use it via the ChatGPT interface.

ChatGPT's high token limits allow for extended conversations. The platform uses chat compression for longer sessions, which can result in some loss of earlier context over time

Complex reasoning models

In a recent development, OpenAI unveiled its highly anticipated o1 model, marking a significant leap in AI capabilities. This model, which had been the subject of industry speculation under codenames like Q*

and Strawberry, represents a new paradigm in machine reasoning. The o1 model's distinguishing feature is its ability to engage in deep, multi-step reasoning before producing an output. This process mirrors the deliberative thinking described in cognitive psychology, where complex problems are solved through careful consideration rather than quick, intuitive responses.

One of the most intriguing aspects of o1 is its scalable performance. Unlike traditional models where output quality is largely fixed, o1's performance can be enhanced by allocating more computational time. This flexibility allows for a trade-off between speed and quality, though currently, this balance is managed automatically by the system rather than user controlled.

Early benchmarks present a nuanced picture of o1's capabilities. While it shows improvements over its predecessors in some areas, it doesn't uniformly outperform all existing models. However, in specific domains—particularly those requiring advanced problem-solving skills—o1 demonstrates remarkable prowess. For instance, in programming competitions, o1 has shown the potential to outperform a significant majority of human participants, even achieving top-tier results in prestigious coding challenges.

The model's strengths become particularly evident in certain types of programming tasks. It excels in developing algorithms, handling complex mathematical operations, optimizing code efficiency, and tackling low-level programming challenges. These areas typically require a deeper understanding of computational principles rather than just pattern recognition or language generation.

The o1 model excels at producing longer outputs. This capability was demonstrated in its successful port of Drupal, a complex content management system, from PHP to Node.js. The model not only translated the entire codebase but also generated a comprehensive implementation plan and the necessary file structure, showcasing its ability to handle system-level programming tasks and cross-language transformations.

Codeforces Elo / percentile

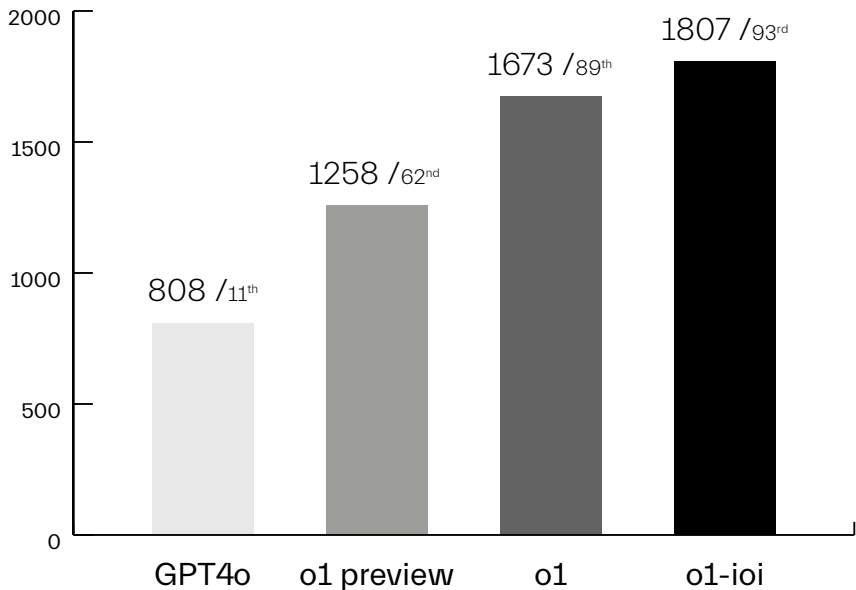


Figure 10. Comparison of Codeforces Elo ratings and percentiles for AI models

While o1 offers advanced capabilities, it comes with higher computational demands, resulting in stricter usage quotas compared to other models. Currently limited to 30 messages per week, it's positioned to remain more constrained than its counterparts. Despite showing reduced tendencies for errors and hallucinations, o1 still lacks integrated code validation tools.

For routine coding tasks, existing models like GPT-4o are often sufficient. However, o1 shines in scenarios requiring complex reasoning, particularly for tasks that are algorithmic, mathematically intensive, or require deep computational thinking. In essence, o1 is not just a more knowledgeable coder, but a more capable reasoner, making it particularly valuable for challenging programming problems that demand careful analysis and problem-solving skills.

Custom GPTs: Enhancing Development Workflows

Custom GPTs, a feature of ChatGPT, allow users to create specialized AI assistants with tailored knowledge and instructions. These customized versions of ChatGPT can be fine-tuned for specific tasks or domains, making them powerful tools for developers.

They offer a range of powerful customization features:

- **Knowledge files:** You can provide Custom GPTs with extensive knowledge files. Thanks to Retrieval-Augmented Generation (RAG), these files can be quite large without overwhelming the system. The GPT retrieves relevant information as needed, allowing for broad knowledge coverage.
- **Third party API integration:** Custom GPTs can be integrated with external APIs, authenticated via OAuth. This enables them to interact with various services and platforms, significantly expanding their capabilities and potential applications in development workflows.
- **Search and image generation and code interpreter:** Custom GPTs inherit ChatGPT's web search and image generation capabilities. However, customization options for these features are currently limited.

Repository-Specific Custom GPTs

The most potent application of Custom GPTs in development is creating one for each of your repositories. Here's how to set it up:

- Craft a prompt that outlines your code style, especially nuances that linters can't easily catch. Include a project overview, potentially auto generated using a large context window model like Gemini.
- Add READMEs, architectural documents, and business specifications as knowledge files.

- Use a script or a tool to concatenate all your code files into a single text file and add it to the GPT's knowledge base.

Unlike Claude Projects, Custom GPTs use Retrieval-Augmented Generation (RAG) instead of loading entire files into the context window. This approach efficiently uses your quota by only retrieving relevant parts, allowing it to handle larger codebases without context limitations. However, it may occasionally miss some context due to selective retrieval.

A significant limitation of Custom GPTs is the lack of API access, which prevents automatic updates when code changes are merged. This is a limitation of the Custom GPT feature itself, not related to its use of RAG. To overcome this, you can use the Assistant API, but this falls outside the ChatGPT subscription and incurs token-based charges. You'll also need to provide your own UI.

Other Custom GPT Use Cases

Custom GPTs excel at handling repetitive tasks. For instance, updating translation files:

- Provide instructions on the desired output format
- Include existing translations
- Add guidelines for translating special terms

This setup creates a specialized translator familiar with your project's nuances.

Custom GPTs shine when you need consistent, specialized assistance. By tailoring them to your specific needs, you create powerful allies in your development workflow.

Other features

- **Voice Mode:** Offers utility beyond direct coding tasks. Developers can use it for hands-free idea exploration or learning during commutes or other activities. It can also serve as a quick query tool during coding sessions, potentially reducing context switches. However, for most coding-related tasks, typing remains more efficient than speech.

- **Desktop application:** Currently Mac-exclusive, this app enhances user experience with seamless screenshot integration. While currently limited in scope, it suggests potential future developments like file system access and direct code context reading, which could significantly boost its value in development workflows.
- **Vision capabilities:** Provides practical applications in UI analysis, architectural diagram comprehension, and converting images to draft documentation. It can offer insights on design intuitiveness and streamline certain development processes. However, its visual interpretation can be unexpectedly inconsistent, sometimes failing on seemingly simple tasks while excelling at more complex ones.
- **Image generation:** Powered by DALL-E 3, this feature currently has limited applications in development. It struggles with producing accurate technical diagrams, restricting its utility. Future iterations, such as the promised GPT-4o text-to-image functionality, may address these limitations and expand its usefulness in development contexts.
- **Code interpreter:** This built-in Python sandbox is useful in various scenarios, particularly for data exploration and computations. However, perhaps a bit counter-intuitively, its value in directly assisting with code writing or complex development tasks is limited. It's more of a complementary tool for data-related tasks than a core coding assistant.
- **Web search integration:** By incorporating web search capabilities, ChatGPT enhances its ability to provide up-to-date information. This feature is particularly valuable when dealing with rapidly evolving technologies or seeking current best practices in software development.

Claude

Claude, developed by Anthropic, is a powerful conversational AI platform that has rapidly gained recognition as a formidable competitor to ChatGPT. Since its introduction, Claude has consistently pushed the boundaries of language model capabilities, often challenging OpenAI's dominance in the field.

Model Comparison

Claude offers three main models: Opus, Sonnet, and Haiku, each designed for different performance requirements and use cases.

Opus: The largest and most capable model in the Claude lineup.

Sonnet: The mid-sized model, balancing performance and efficiency.

Haiku: The smallest and fastest model, suitable for less complex tasks.

In the Claude 3.0 series, Opus clearly dominated Sonnet and Haiku in coding benchmarks. For instance, in the HumanEval benchmark, Opus scored 84.9, while Sonnet achieved 73.0 and Haiku surprisingly outperformed Sonnet with 75.9.

However, this changed significantly with the release of Claude 3.5 Sonnet. The update propelled Sonnet's performance beyond that of Claude 3.0 Opus, scoring an impressive 92.0 on HumanEval. Given its superior performance and faster processing speed, there's currently little reason to use Claude 3.0 Opus for most tasks.

The anticipated release of Claude 3.5 Opus is expected to bring another substantial leap in capabilities. If the performance gap between 3.0 and 3.5 versions of Sonnet serves as an indicator, Opus 3.5 could represent a significant advancement in AI language model technology.

Projects Feature

Claude's Projects feature serves a similar purpose to ChatGPT's Custom GPTs, allowing users to create specialized environments for specific tasks or domains. However, the two systems operate quite differently, with distinct implications for development workflows.

Knowledge Integration and Token Usage

The key difference lies in how knowledge is integrated. Claude Projects load information directly into the context window, resulting in better integration and understanding of the provided content. This contrasts with Custom GPTs, which use Retrieval-Augmented Generation (RAG) to handle larger knowledge bases.

This difference in knowledge integration affects token usage. Projects consume tokens more quickly due to full context loading, potentially exhausting message quotas faster. Custom GPTs, on the other hand, use tokens more efficiently, allowing for longer conversations with large knowledge bases.

Sharing and Management

Claude Projects can be shared, but only with other paid Claude users, limiting their collaborative potential. Custom GPTs offer broader sharing options, including with free users.

When it comes to knowledge management, Projects provide a more user-friendly experience for maintaining and updating knowledge bases. Projects still lacks API access for automatic knowledge updates, but this is a less important use case than with CustomGPTs as you wouldn't typically include the entire codebase as knowledge.

Effective Use Strategies

For developers looking to leverage Claude Projects effectively, a strategic approach is necessary:

- Create a single project with background information and code summaries as a foundation for most coding-related queries.
- For detailed code analysis, attach a concatenated text file containing relevant (but significant) parts of the codebase to your prompt. Use this approach sparingly due to high token consumption.

- Focus on key documentation and summaries rather than maintaining entire repositories as knowledge bases.

Other features

When compared to ChatGPT, Claude lacks several features present in its competitor. It doesn't offer image generation or a code interpreter, though these are rarely crucial for coding tasks. More notably, Claude is missing search functionality, which can be a significant drawback when up-to-date information is needed during development. It also lacks voice mode and a desktop application, which some users might miss for convenience.

Claude does include vision capabilities, outperforming GPT-4o in benchmarks, though not by an overwhelming margin. This feature can be particularly useful for tasks involving image analysis or interpretation in a development context.

Artifacts

Claude's most distinctive feature is its artifacts system. This system enhances code organization and versioning, providing a smoother workflow for developers. It also offers an interactive environment where Claude can generate and run React or vanilla JavaScript code directly within the chat interface. The artifacts system also allows for diagram creation using Mermaid and Lucid Charts, which can be valuable for visualizing complex systems or workflows.

The sandbox environment is restricted to a specific set of pre-installed libraries, primarily TypeScript, Tailwind, and React. It won't be able to import third-party dependencies, which limits its applications. Both the library and language support can be expected to widen, and you can work around the current limitations:

For example, if you need it to write an UI component with Material UI, you could first have it implement it using its built in UI libraries and when its looking good enough, ask Claude to port it to Material UI.

Examples of Artifact Usage

To illustrate the capabilities of Claude’s artifacts, consider these examples:

Build me an input component using React.js + typescript that picks a phone number country code and displays the flag alongside it, alongside the actual non country code phone number. Avoid external dependencies beyond your built-ins.

 Australia (+61) ▼

123456

Full number:  +61 123456

Figure 11. Input component that picks a phone number country code and displays the flag alongside it.

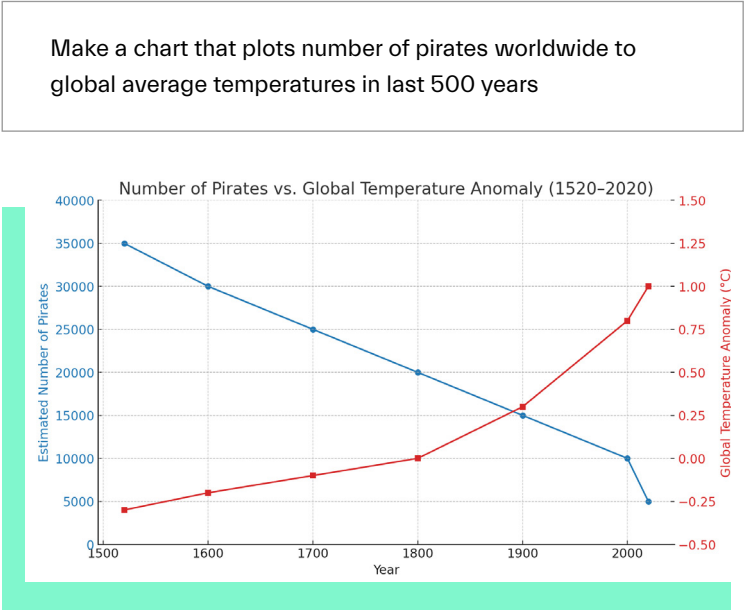


Figure 12. This chart is for entertainment purposes only. Correlation does not imply causation.

Draw me an architecture diagram using mermaid about how TempleOS works

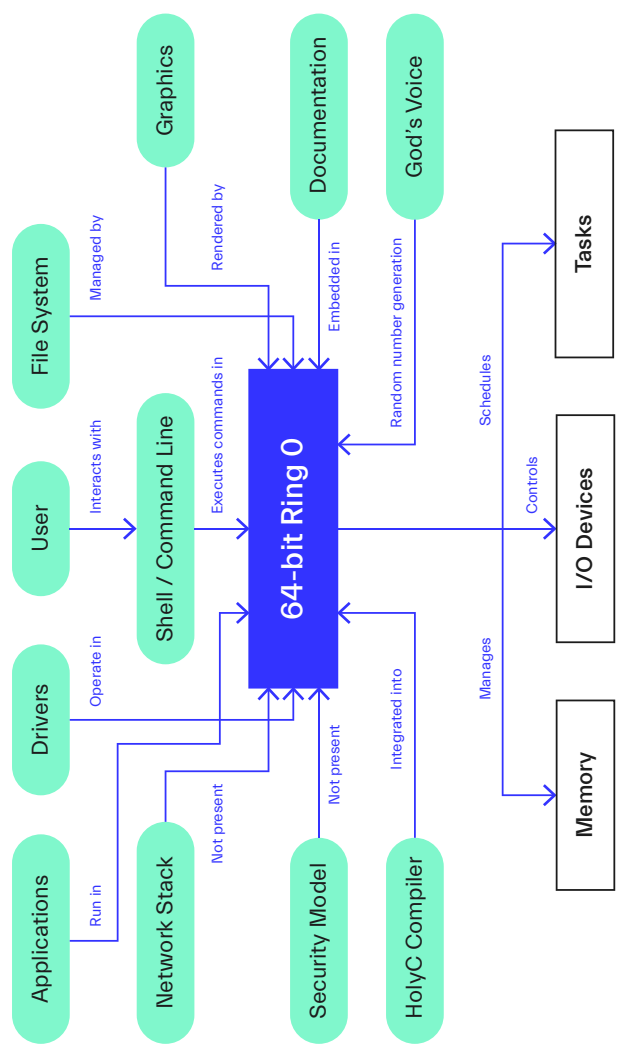


Figure 13. An architecture diagram showing how TempleOS works. Redrawn for the publication.

Developer tools

AI coding assistants like GitHub Copilot and JetBrains AI, when integrated into Integrated Development Environments (IDEs), offer a range of features that aim to streamline the coding process and boost developer productivity.

These assistants can provide real-time code suggestions as you type, often completing entire lines or blocks of code based on the context of your work. They're capable of generating function bodies, suggesting variable names, and even writing complex algorithms when given a descriptive comment or function signature. Many can also assist with code refactoring, offering suggestions to improve code structure or efficiency.

Beyond code completion, these AI tools often include features for code explanation and documentation. They can generate comments for your code, explain the purpose of a function or class, and even provide inline documentation for libraries and APIs. Some advanced integrations offer natural language interfaces, allowing developers to describe a coding task in plain English and receive relevant code snippets or explanations in return.

Error detection and debugging assistance are other key features. These AI assistants can often spot potential bugs or anti-patterns in your code, suggesting fixes or improvements. They may also help in troubleshooting by providing explanations for error messages or suggesting possible causes for unexpected behavior.

Compared to prompting interfaces, their main value comes from:

- **Being close to your workflows** and making it easy to delegate trivial coding tasks to AI. For example, you would not prompt ChatGPT to write you a normal small method, as the overhead would be too high. But with autocomplete and other close-to code functionalities, delegating to AI becomes as seamless as using the “old-fashioned” IDE autocomplete features.

- **Having the context of your codebase.** While these still mainly use vector-based RAG, you'll usually still get pretty decent answers.
- **Being able to take actions in IDE** instead of human acting as an AI secretary. Typically, the IDE assistants have been limited regarding this, but things are changing with some new features of Claude, and the bigger players are sure to catch up.

IDE assistants have become indispensable tools for developers, dramatically improving coding speed, accuracy, and overall productivity. Modern AI-powered assistants integrate seamlessly with development environments, offering capabilities like intelligent code completion, error detection, and real-time suggestions. By leveraging powerful models, they transform the way developers approach coding tasks, allowing for more efficient workflows and better code quality.

Leading IDE assistants, including Cursor, JetBrains AI, Amazon Q Developer, and GitHub Copilot, each have their unique strengths and cater to different development environments. We will analyze them to more detail in the following.

GitHub Copilot

GitHub Copilot, released in June 2021 by GitHub in collaboration with OpenAI, is an AI-driven code completion tool. It became generally available in June 2022, marking a significant step in AI-assisted software development.

Copilot employs two distinct models for different tasks. OpenAI Codex, a GPT-3 derivative fine-tuned on a vast corpus of public source code, powers Copilot's code completions and other code-related actions. While often effective for basic autocomplete tasks, it may struggle with more complex assignments. For its chat feature, Copilot leverages the more advanced GPT-4 model, enabling more nuanced interactions and explanations.

For complex coding tasks, developers might find more comprehensive assistance from standalone AI models like those powering Claude or ChatGPT.

While the exact inner workings of Copilot remain unpublished, we can infer some key aspects of its operation. Copilot heavily relies on the immediate context within your current file. It's not just pulling code out of thin air – it's analyzing what you're currently working on. While Copilot may have some ability to consider wider project context, its primary focus appears to be on the code directly surrounding your cursor. Using this local context, Copilot generates code suggestions that aim to fit seamlessly into your work.

Copilot maintains its position as a strong option, particularly for developers who need to remain within the VS Code environment. This necessity often arises from specific plugin requirements, such as certain .NET tools not yet compatible with alternatives like Cursor. Copilot's robust feature set and ongoing development ensure its continued relevance in the IDE assistant space, making it a reliable choice for many development scenarios.

JetBrains AI

JetBrains AI, launched in 2023, is the AI-powered coding assistant integrated into JetBrains' IDE suite. Unlike GitHub Copilot's reliance on OpenAI models, JetBrains AI uses a multi-model approach, including proprietary JetBrains models and those from Anthropic.

Its main advantage is tighter integration with JetBrains IDEs, enabling features like commit message generation. This integration potentially allows for a better context understanding of the entire codebase, with slightly more advanced Retrieval-Augmented Generation (RAG) capabilities.

A notable feature of JetBrains AI is its use of different models for different programming languages. This approach allows for more specialized and accurate code suggestions tailored to the specific syntax and conventions of each language. For instance, it might use one model optimized for Java development and another for Python, potentially leading to more context-aware and idiomatic code suggestions.

Performance-wise, JetBrains AI is generally on par with GitHub Copilot. While it offers unique possibilities due to its deep integration with JetBrains products, many of these are not yet fully realized. It may appear slightly less polished than Copilot in some respects, probably due to its more recent market entry.

Currently, JetBrains AI and GitHub Copilot are largely interchangeable for most coding tasks. However, JetBrains AI holds a significant advantage when working within the JetBrains ecosystem as Copilot's JetBrains integration is missing certain key features available on the Visual Studio Code-version.

For developers deeply invested in the JetBrains ecosystem, JetBrains AI emerges as the top choice. While it may not match every feature of the VS Code version of Copilot, it outperforms the JetBrains-specific Copilot implementation. The seamless integration with JetBrains tools makes it the natural choice for users committed to this IDE family, offering a familiar environment with enhanced capabilities.

Cursor

Cursor, a relatively new tool build on a fork of Microsoft Visual Studio Code, sets itself apart from other AI-powered development tools with its unique philosophy and user experience. While it shares some features with GitHub Copilot,

Cursor positions itself more as an AI-powered IDE rather than just an IDE integration. This distinction becomes apparent in how developers interact with the tool and leverage its capabilities.

Cursor has established itself as the premier IDE assistant, offering capabilities far beyond typical code completion. To maximize Cursor's potential, developers should thoroughly explore its documentation and features. This exploration involves adapting one's coding approach to leverage Cursor's strengths, particularly its semi-agentic composer feature. This powerful tool allows for building complex structures using natural language commands, fundamentally changing how code is written.

A key aspect of mastering Cursor is embracing its caret prediction and "tab-and-watch" functionality. While this new approach may initially feel unfamiliar, the resulting acceleration in coding speed and efficiency is substantial. The initial adjustment period is a small price to pay for the significant productivity gains that follow.

Key Features of Cursor include:

- **Chat:** Works similarly to Copilot's chat feature but with added model selection and quality-of-life improvements. Beta features include long context chat (add up to 10,000 lines of code by mentioning folders) and a review tab for code review on changes since the last commit.
- **Code generator:** Select and edit existing code or input new code via prompts. Easily add other symbols, files, and docs to the context when needed.

- **Composer:** Recently released feature for making changes across multiple files. Give it a detailed writeup of what you want, and it can generate substantial code chunks touching various files. In theory, it can even build entire apps in one go, though complexity may increase failure rates.
- **Context building:** All prompts default to including the currently active file. Options include using @codebase for vector-based RAG search, referring to specific symbols, files, or folders, and web references through search or direct URLs.
- **Documentation ingestion:** Provide URLs to custom documentation for Cursor to index and later tag in prompts. Especially useful for internal documentation, cutting-edge tech, or esoteric libraries.
- **Custom instructions:** Set global instructions via settings or repo-specific ones with a .cursorrules file. Specify coding styles, add context, or apply prompt engineering techniques.
- **Terminal helpers:** Press CTRL+K and prompt for shell commands instead of memorizing complex syntax. Works with most terminals, though non-bash-compliant ones may need extra prompting.
- **Fix in chat:** When errors pop up in the IDE or terminal, a button appears to attempt automatic fixes.
- **Shadow workspace:** During code generation, Cursor uses a shadow VS Code instance to run linters, check for errors, and add missing imports. This increases the odds of code working on the first try.
- **Caret prediction:** When you accept an autocomplete, Cursor will predict the next caret position. This may seem pointless on paper, but improves the workflow quite a bit and lets you just repeatedly hit tab and have Cursor predict all the changes

- **Auto remove code:** When accepting autocomplete Cursor can also remove the things you've written. This, again, might seem pointless but improves the workflow. For example, you can just write "log results" and autocomplete will remove that and replace it with an actual properly formatted logging statement.

What we've learned is that using Cursor effectively requires a mindset shift. English becomes your primary programming language, with the AI translating your intent into code. While you can use it as a standard VS Code-based IDE with AI autocomplete, diving into the documentation is crucial to grasp its full potential.

The UX might have a learning curve and could be divisive, but Cursor currently stands as one of the best AI IDE integrations available. The Composer feature pushes the boundaries towards true AI pair programming rather than just assistance.

Amazon Q

Amazon Q, released in late 2023, is Amazon's entry into AI-powered coding tools. It works with common IDEs, helping developers write code, find vulnerabilities, and make sense of existing projects. Q's tight integration with AWS services provides added value.

Amazon Q Developer shines in AWS-centric environments. Its standout feature is the ability to fine tune the assistant to a specific codebase. This customization requires additional setup effort, but the resulting contextualized assistance can be invaluable for complex AWS-based projects. For teams heavily invested in the AWS ecosystem, the initial time investment in setup can pay significant dividends in long-term productivity and code quality. Key features include:

- **Customizability:** Q adapts to unique development environments through training on company-specific codebases and private APIs.
- **Security Scanning:** Built-in vulnerability checks help maintain code security throughout the development process.
- **Language Transformation:** Specialized tools, like Java version upgrades (e.g., Java 8 to 17), streamline modernization efforts.
- **Agent Mode:** The `/dev` command activates a more autonomous assistant capable of handling complex, multi-step tasks.

Q scores an impressive 38.8% on SWE-bench verified dataset and has only very recently been dethroned from the peak position. While the coding assistant invoked by `/dev` doesn't seem to be quite as capable, it gives a hint on Amazon aiming to have it be increasingly agentic and capable of solving complex tasks.

As it stands, Q is useful but trails behind Cursor's composer feature both in capabilities and UX. The current limitations include sluggish performance and some file handling woes. It also comes as bit of a

surprise Q it won't check that the code it has written passes tests and even compiles.

These limitations are developmental rather than fundamental. As Amazon refines Q's capabilities, particularly its agent functionality, we can expect improvements in response speed, context awareness, and error resolution. While Q is not yet a perfect solution, it is already useful and worth considering if you're working heavily with AWS.

SuperMaven

SuperMaven, launched in late 2023, quickly gained traction among developers after the release of its stable version in July 2024, thanks to its unique approach to AI-assisted coding.

What sets SuperMaven apart from the current competition are performance, context window size and efficient change tracking. SuperMaven uses RNN architecture for rapid inference, making it one of the most responsive coding assistants available. Also, the latest model, "Babble," boasts a 1 million token context window, potentially allowing entire codebases to be 'seen' at once. SuperMaven also excels at tracking edit deltas within context, ensuring relevant suggestions even in large projects.

SuperMaven's autocomplete model performs similarly to GitHub Copilot but with near-instantaneous responses. While its large context window is impressive, recall may not be uniformly strong across all million tokens.

As a product, SuperMaven has been well-received for its speed and responsiveness, though some users note occasional less sophisticated completions compared to other assistants. Many see this as an acceptable trade-off given its speed and wide context window.

Noteworthy alternatives

Two noteworthy developments warrant attention. Google's Gemini code

assistants claim to maintain entire codebases in context, likely utilizing their Gemini Flash model. While promising, this technology currently requires significant setup and hasn't yet reached full maturity. Its potential, however, is significant, particularly for large-scale projects.

SuperMaven is taking a different approach, pioneering a non-transformer-based architecture. This innovation promises rapid autocompletes and extensive context windows capable of ingesting entire codebases. While still emerging, SuperMaven's novel approach could potentially reshape the field of IDE assistance, offering new possibilities for code generation and comprehension.

Code reviewing tools

AI-assisted code review tools are emerging as a promising technology in software development, offering potential improvements to traditional review processes. These tools focus on analyzing pull requests, not entire codebases, and are beginning to show real utility in streamlining code review workflows.

Unlike linter tools that catch syntax errors, AI review tools aim to provide higher-level insights. They act as additional, rapid reviewers that can spot issues before human eyes see the code or catch problems that humans might overlook. While they don't significantly boost expertise, they do offer efficiency gains by handling routine checks quickly.

Key advantages of AI-assisted code review include automatic summarization that provide more insight into review comments, common error pattern recognition and generally more comprehensive analysis on the code. Generally automated code review tools often catch issues which are often missed by human reviewers and apply coding standards and best practices uniformly across all reviews.

Furthermore, by handling routine checks and providing clear, actionable feedback, AI tools can smooth the review process for humans,

potentially decreasing code review turnaround time.

Code review tools can be easily integrated into existing development workflows through GitHub or GitLab integrations. They operate alongside human reviewers, providing additional insights without disrupting established processes.

This chapter introduces two prominent AI code review tools: CodeRabbit and CodiumAI PR Agent. We'll examine their core features and how they function within the code review process, giving developers a clearer picture of how these AI assistants can complement human expertise in real-world scenarios.

CodiumAI PR Agent

CodiumAI PR Agent is an AI-powered tool designed to enhance the pull request process. It operates through a set of customizable commands, allowing users to invoke specific AI-powered functions during code review. The tool aims to streamline various aspects of PR management, from description generation to test creation.

Key features of CodiumAI PR Agent include:

- **/describe**: Automatically generates a comprehensive PR description, including title, type, summary, code walkthrough, and labels. This can save time in PR preparation and provide consistency in PR documentation.
- **/review**: Offers adjustable feedback about the PR, highlighting potential issues, security concerns, and estimated review effort. This feature can help prioritize review tasks and identify critical areas for human review.
- **/improve**: Provides code suggestions for enhancing the PR. This can help developers refine their code before or during the review process.

- **`/ask`**: Allows users to pose free-text questions about the PR, with the AI providing answers. This feature can help clarify aspects of the code changes without needing to directly consult the PR author.
- **`/update_changelog`**: Automatically updates the `CHANGELOG.md` file with PR changes, ensuring documentation stays current with code changes.
- **`/similar_issue`**: Retrieves and presents issues like the current PR, which can help in identifying recurring problems or related work.
- **`/add_docs`**: Generates documentation for changed methods, functions, or classes, promoting better code understanding and maintenance.
- **`/generate_labels`**: Creates custom PR labels based on user-defined guidelines, which can aid in PR categorization and workflow management.
- **`/analyze`**: Enables interactive generation of tests, docs, and code suggestions for changed components, offering a comprehensive review of PR changes.
- **`/custom_prompt`**: Generates improvement suggestions based on specific guidelines defined by the user, allowing for tailored code review processes.
- **`/test`**: Generates unit tests for selected components based on PR code changes, potentially improving code coverage and quality.
- **`/checks`**: Provides feedback and analysis for failed CI jobs, helping to quickly identify and address integration issues.

- **`/find_similar_component`**: Retrieves similar code components from within the organization's codebase or from open-source repositories, which can aid in maintaining coding standards or identifying potential code reuse.

CodiumAI PR Agent is available both as a self-hosted solution and as a pro version, offering flexibility in deployment options. This command-driven approach allows for targeted use of AI in the code review process, potentially streamlining workflows by providing specific assistance as needed

Code Rabbit

CodeRabbit is an AI-driven platform focused on optimizing code review processes within the software development lifecycle. It leverages machine learning to assist developers in reviewing, testing, and improving code quality.

At its core, CodeRabbit performs intelligent code reviews, analyzing bugs, performance issues, and adherence to best practices. The platform offers high customizability, allowing teams to configure detailed review parameters for specific parts of their repository. CodeRabbit enhances the code review process with several key features:

- **Automated CI/CD pipeline** integration
- **Learning capability** to remember and apply project-specific guidelines
- **Interactive communication** in pull requests, simulating human reviewer interactions
- **Pull request summarization** and context provision from related issues

- **Integration with external sources** like JIRA for comprehensive change tracking
- **Generation of code change diagrams**
- **Customizable review configurations** for different parts of the codebase

The platform integrates with both GitHub and GitLab, including self-hosted instances, fitting into existing workflows. It also incorporates built-in linters and static analysis tools to provide a comprehensive code review solution.

Beyond prompting – software engineering agents

Coding agents represent a significant evolution in AI-assisted software development. While coding assistants help developers write code, agents aim to implement code with minimal human intervention. The distinction between assistants and agents isn't binary, but rather a spectrum of increasing autonomy.

The primary goal of coding agents is to eliminate repetitive manual work from the development process. This doesn't mean removing humans from the equation; instead, it's about freeing developers from being AI secretaries. Ideally, agents should handle predictable parts of the software engineering process autonomously, including interacting with development tools like linters and debuggers.

By automating routine tasks, coding agents allow developers to focus on higher-level decision-making, creative problem-solving, and complex architectural challenges. The result is a more efficient development process where human expertise is applied where it's most valuable.

Beyond code generation with chaining and tool integration

Coding agents augment the capabilities of models by two primary methods: chaining smaller tasks together, potentially with feedback

capabilities, and via integrating with development tools.

Agents don't just write code once. They create code, review it, test it, and then improve it. It is possible to have this cycle to be repeated until the code meets the required standards.

Furthermore, agents can be integrated to specialized tools, such as testing frameworks, linters, and version control systems. They apply these tools as needed throughout the development process.

Combining chaining and tool integrations enable agents to take the game way beyond simple code generation. Ultimately, they can manage entire development workflows, from initial coding to testing and refinement with little to no human intervention.

Redefining development cycles

Software development typically involves two types of cycles:

- **Inner cycle:** The rapid iteration of writing code, testing, and debugging.
- **Outer cycle:** Higher-level tasks like design decisions and architecture planning.

Coding agents are reshaping this paradigm by taking over much of the inner loop, freeing developers to focus more on the outer loop. This shift fundamentally alters how developers work and interact with their projects.

In practice, this means less time spent as an “AI secretary.” No more endless copying and pasting of generated code, followed by reporting compilation errors or test failures back to the AI. Instead, developers initiate tasks with high-level descriptions or specifications and then monitor the agent’s progress through automated logs or dashboards.

When the agent encounters issues it can't resolve, it flags them for human intervention, providing context and attempted solutions. This approach allows developers to step in only for complex problems that genuinely require human insight, typically related to the outer loop of development.

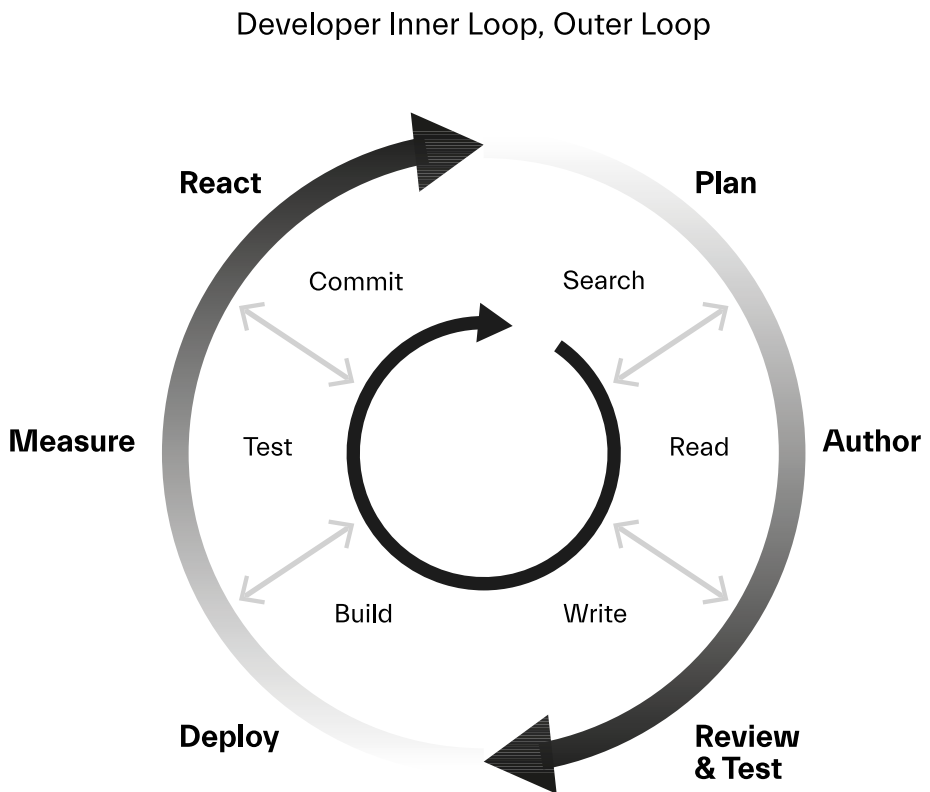


Figure 14. The two cycles of software development: an inner cycle of rapid coding, testing, and debugging, and an outer cycle focused on design and architecture planning.

Challenges in adopting coding agents

While coding agents show promise, they face significant hurdles that need to be overcome for widespread adoption. These challenges span from fundamental issues of code comprehension to practical matters of integration and cost.

Code location and context

The most critical challenge for coding agents is understanding where and how to make changes in complex codebases. This “code location” problem is even more acute for agents than for assistants, as agents need comprehensive knowledge of all relevant code sections to operate autonomously.

Current approaches like vector search fall short, unable to provide the nuanced understanding required for complex software engineering tasks. While modern LLMs boast impressive context windows, capable of ingesting sizable code chunks or even entire repositories, this approach comes with prohibitive costs. Even with context caching reducing expenses by a factor of 10, the price remains too high for practical, agentic usage at scale.

Plumbing and tool integration

The second major hurdle is “plumbing” - the intricate code and processes required to integrate agents with development tools and workflows. Current agents often rely on executing shell commands, a method that works to some degree but is prone to errors.

This is a similar problem that Infrastructure as Code (IaC) aims to address for the cloud. Instead of knowing the exact sequence of commands, the developer defines the end state and leaves the tedious error parsing and change management to the IaC engine.

Debugging presents another challenge. Agents typically resort to print line debugging instead of leveraging more sophisticated debugging tools. They also lack access to standard IDE features, making seemingly trivial tasks like renaming a method with dependencies surprisingly

difficult. Agents miss out on the benefits of auto-imports, linter auto-fixes, and other productivity tools that human developers take for granted.

In summary, the ability to debug the systems to the degree the developers can do it remains a hard challenge for agents or AI-powered development in general. The current AI tools excel in static code generation and analysis, whereas the runtime side – debugging and testing live systems – still requires a human in the loop.

Sub-Agent Interaction

Coordinating multiple specialized agents introduces additional complexities to the software development process. One of the primary challenges is passing the necessary context between sub-agents. As each agent focuses on specific tasks, ensuring they all have the relevant information without overwhelming them with unnecessary data is a delicate balance.

Error recovery in a multi-agent system presents another significant hurdle. When an error occurs, identifying its source and implementing corrective measures across multiple agents can be complex. This complexity increases with the number of agents involved and the intricacy of their interactions.

Perhaps the most challenging aspect is dynamically altering approaches when simple fixes fail. This requires a level of adaptability and coordination that current systems struggle to achieve. Agents must not only recognize when their current strategy is ineffective but also collaboratively develop and implement new approaches without disrupting the overall development process.

The token economy: Cost vs. capability

Despite these challenges, the economics of coding agents remain promising. Data from OpenHands CodeActAgent v1.8 on the SWE-bench lite test illustrates this.

At \$4–8 per successful fix, agents are often more cost-effective than human developers for many tasks. Lightweight models like GPT-4o-mini, at just \$0.01 per attempt, allow for low-risk problem-solving attempts. Even when agents don't fully resolve an issue, they often make progress that human developers can build upon.

One effective strategy is to treat these agents like junior developers, assigning them simpler tasks. This approach leverages their cost-effectiveness while working within their current capabilities.

As AI models continue to improve, we can expect to see better performance at lower costs, gradually making coding agents more viable for a wider range of development tasks. However, overcoming the fundamental challenges of code comprehension, tool integration, and inter-agent coordination remains crucial for realizing the full potential of coding agents in software development.

Autonomous coding agents

Devin

Devin, unveiled in early 2024, initially made waves by solving 13.86% of issues in the SWE-bench dataset, surpassing previous benchmarks. While newer models have since approached 20% success rates, Devin's development has continued apace.

Devin was met with mixed reactions from the tech community. While some hailed it as a breakthrough, others expressed skepticism about its real-world applicability. Microsoft's acquisition of Devin's creators lent credibility to the project. Industry speculation suggests that Microsoft intends to integrate Devin into Visual Studio Code, potentially granting it access to a wide array of IDE features.

Devin's architecture consists of three key components:

- A planner agent maintaining a task list
- Coding agents responsible for code generation
- CLI agents executing code and managing files

Though not yet generally available, Devin is evolving from a proof-of-concept into a more polished product. Its core architecture remains built around a planner agent, coding agents, and CLI agents, but recent updates have significantly expanded its capabilities.

Key improvements include the addition of secrets sharing, user access to Devin's workspace and browser, and the introduction of playbooks for specifying workflows and repetitive tasks. Devin now features a project knowledge base and integrations with platforms like Slack and GitHub.

Notably, Devin can now autonomously reply to GitHub PR comments, implementing requested changes without direct human intervention. It also automatically builds its knowledge base from chat interactions and supports rollbacks and state saving. The introduction of "Multi-

Devin” allows for parallel task execution, potentially boosting overall efficiency.

These enhancements, coupled with ongoing improvements in speed, cost-effectiveness, accuracy, and reliability, suggest that Devin is inching closer to practical application in real-world development environments.

OpenHands: The open-source alternative

OpenHands, originally known as OpenDevin, emerged as the open-source community’s response to closed-source autonomous coding agents. While its capabilities have expanded beyond its initial focus on Python, it remains a work in progress, striving to match the functionality of its proprietary counterparts.

OpenHands now supports multiple programming languages and can handle simple project creation tasks. However, its practical utility in professional settings remains limited. The project faces several challenges common to autonomous coding agents, including efficiently navigating large codebases, integrating with existing development tools, and coordinating multiple specialized agents for complex tasks.

OpenHands’ performance varies significantly depending on the language model used. Larger models like GPT-4 or Claude-3.5 improve success rates but increase operational costs and reduce processing speed. Conversely, smaller models like GPT-4-mini reduce costs substantially but at the expense of lower success rates. This trade-off between performance and resource usage remains a key consideration for potential users.

Recent funding has injected new resources into the project, though the scope of work remains daunting compared to available resources. The open-source nature of OpenHands presents both challenges and opportunities. It evolves rapidly with frequent updates and improvements,

Disclaimer: Siili is loosely involved in OpenHands development

allowing developers and companies to directly contribute to shaping the tool's future.

OpenHands distinguishes itself in two key areas. First, its open-source nature allows for transparency, customization, and community-driven development. Second, unlike some proprietary solutions, it can be deployed and experimented with immediately, enabling early adopters to gain hands-on experience with autonomous coding agents.

As the project matures, it has the potential to become a powerful, accessible alternative to closed-source autonomous coding solutions. The open-source community's collective efforts and imagination will play a crucial role in determining OpenHands' future capabilities and impact on AI-assisted software development.

Micro-agents

Micro-agents by Builder.io are designed to enhance AI-assisted coding by providing reliable and high-quality code generation. Unlike traditional AI tools that often produce code requiring extensive debugging, Micro-agents use unit tests as guardrails to ensure that generated code meets specified requirements. This iterative approach involves generating unit tests based on natural language descriptions, writing code to pass these tests, and automatically refining the code until all tests pass.

The philosophy of pragmatic expectations

Micro-agents in AI-assisted coding embody a “minimum viable functionality” approach. They focus on making changes to single files, following a simple workflow:

- **Write** the test
- **Iterate** until functionality is achieved.

This pragmatic strategy aims to sidestep the complexities of full-scale coding agents by narrowing the scope to achievable goals. It's an acknowledgment that while AI has potential in software development, a targeted, limited approach may yield more immediate benefits. By

lowering expectations and focusing on specific, manageable tasks, micro-agents try to bridge the gap between ambitious AI projects and practical, day-to-day coding needs.

Current implementation and focus

Currently, micro-agents operate primarily within the TypeScript ecosystem, with an emphasis on Node.js and front-end projects. This focus raises questions, particularly when considering the application of Test-Driven Development (TDD) to front-end code.

The decision to target front-end-oriented TypeScript projects seems counterintuitive. Unit tests for front-end components often pale in comparison to visual regression testing on platforms like Storybook. This choice appears to apply to the tool in an area where its strengths are least likely to shine.

A more natural fit might have been domains where TDD excels: back-end services, data processing, or areas where output is primarily functional rather than visual.

Usability and future potential

In their current state, micro-agents hover just below the threshold of practical usability. However, the core concept holds promise. To realize its potential, a micro-agent system should ideally be:

- Language agnostic
- Significantly more mature
- Integrated seamlessly into IDEs.

With refinement and a pivot towards more suitable applications, micro-agents could evolve into a valuable developer tool. While not yet fully realized, they are an intriguing approach to AI-assisted coding that, if properly developed, could significantly enhance productivity without replacing human expertise.

GitHub Copilot Workspaces

GitHub Copilot Workspaces is an AI-powered development environment that extends GitHub Copilot's capabilities to cover the entire software development lifecycle. It integrates closely with GitHub repositories and tools, enabling developers to move from initial idea to working code using natural language inputs. The environment supports real-time collaboration, code generation, and iterative development directly within a shared workspace. As of now, Copilot Workspaces is still in technical preview, with general availability expected to be announced in November 2024, according to industry rumors. The workflow in Copilot Workspaces follows these steps:

- **Task initiation:** The process begins when a developer inputs a task description using natural language. This description serves as the starting point for the AI to understand the developer's intentions and goals.
- **Specification generation:** Based on the task description, the AI generates a detailed specification. This specification includes an overview of the task, an assessment of the current state of the codebase, and a list of relevant files and dependencies. Developers can review and edit this specification, ensuring that it accurately reflects their requirements before proceeding.
- **Plan creation:** Once the specification is finalized, Copilot Workspaces creates a plan of action. This plan outlines the specific changes needed to accomplish the task, including which files need to be modified, what new files should be created, and the nature of the changes to be implemented. As with the specification, developers can review and adjust this plan to ensure it aligns with their vision.
- **Implementation:** After the plan is approved, the AI proceeds to make the proposed changes to the codebase. It generates new code, modifies existing files, and implements the changes as outlined in the plan.

- **Pull request generation:** In the last step, Copilot Workspaces creates a pull request having all the implemented changes. This pull request is then ready for review and potential integration into the main codebase.

As a technical preview, it has not been widely evaluated in real-world scenarios. Early reviews present a mixed picture, suggesting that while the tool offers some value, it has yet to fully realize its potential. A significant limitation is the absence of a feedback loop – the AI does not verify the functionality of the generated code before presenting it to the user. This oversight means that developers still need to thoroughly review and test the AI-generated code, potentially reducing the efficiency gains the tool aims to provide.

The ultimate success of GitHub Copilot Workspaces will depend on how effectively it addresses these limitations and fulfills its promise of comprehensive AI-assisted development.

Aider

Aider is an AI coding tool that integrates with local git repositories, enabling efficient pair programming with large language models. It stands out for its unique approach to context management and automation potential.

Repo graph: Aider's edge

Aider's key feature is its use of a concise map of the entire git repository. This map includes important classes and functions with their types and call signatures, helping Aider understand the code it's editing and its relationship to the rest of the codebase.

The repo map is optimized using a graph ranking algorithm, treating each source file as a node and creating edges between files with dependencies. This allows Aider to select the most relevant parts of the codebase that fit into the LLM's context window.

This approach is the main secret sauce behind Aider's impressive performance on SWE Bench.

Practical use and automation

Aider works with the most popular programming languages and can connect to various LLMs, though it performs best with GPT-4o and Claude 3.5 Sonnet. It can edit multiple files simultaneously and allows users to edit files in their preferred editor while chatting.

A standout feature is Aider's scriptability, which enables automation of complex coding tasks. Here's an example of using Aider to migrate Python 2.x code to Python 3.x:

```
from aider.coders import Coder
from aider.models import Model
import os

def migrate_to_python3(directory):
    model = Model("gpt-4o")
    py_files = [f for f in os.listdir(directory) if f.endswith('.py')]

    for file in py_files:
        file_path = os.path.join(directory, file)
        coder = Coder.create(main_model=model, fnames=[file_path])
        coder.run("Migrate this Python 2.x code to Python 3.x. "
                  "Update syntax, use print function instead of statement, "
                  "update division behavior, use modern string formatting, "
                  "and make any other necessary changes for Python 3 compatibility.")
```

Usage

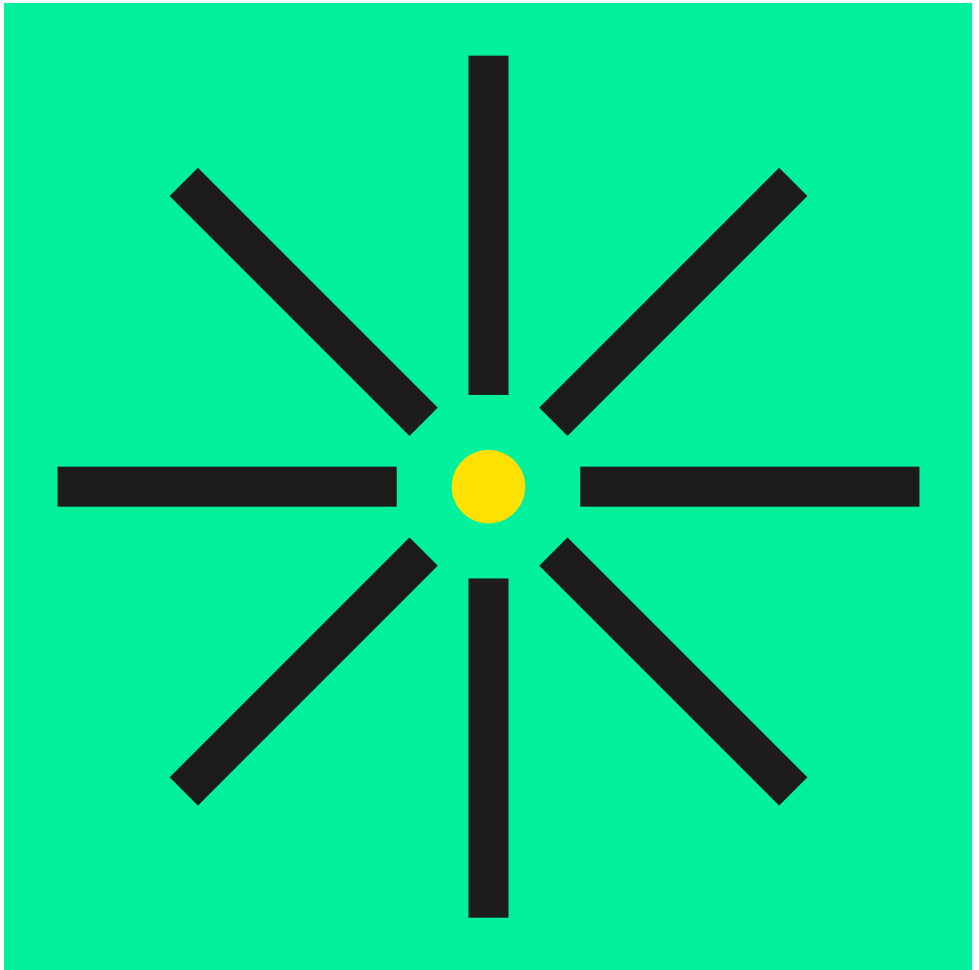
```
migrate_to_python3("/path/to/your/python2_project")
```

This script automates the process of updating multiple Python files from 2.x to 3.x syntax, demonstrating Aider's potential for large-scale code modifications.

While Aider is already useful, future IDE integrations could further enhance its capabilities, making it an even more integral part of the coding workflow.

PART 2 •

In practice





Using AI in development

This section explores the practical applications of AI tools in software development. Rather than guessing on AI's potential to replace developers, we focus on how current AI technologies can enhance and streamline the development process.

Generic goals of AI Integration are:

- **Enhanced productivity:** Accelerate development by using AI for code generation, testing, and bug fixing.
- **Foster innovation:** Use AI as a collaborative tool to uncover new approaches and identify gaps in requirements or implementation.
- **Improved Quality:** Employ AI for thorough code reviews, bug detection, and security threat identification.
- **Reduced cognitive load:** Delegate routine tasks to AI, allowing developers to focus on complex problem-solving and creative aspects of development.

- **Expanded technical capabilities:** With AI aid, developers can more confidently approach projects involving unfamiliar languages or frameworks.

In this section, we will give concrete examples of how to maximize the value of current AI tools throughout the software development lifecycle. To illustrate these applications, we'll use a hypothetical project as a consistent reference point.

Introducing iEat – a custom-built, SaaS restaurant table reservation system. iEat serves as a contextual glue to show applications of AI in real-world work, ranging from first concept development through to long-term maintenance.

By exploring these use cases, you'll gain practical insights into integrating AI tools into your development workflow, potentially transforming your approach to projects and expanding your technical capabilities.

The use cases are divided into the following phases, which follow a typical path from idea to production and maintenance.

Ideation	How can you use AI to gather and analyze requirements?
Bootstrapping	How to get a quick start for your project?
Product Design	How to have the first UX for your solution? What options do you have for the architecture?
Development	Use AI to generate, explain and refactor and commit your code.
Working with Data	Turn AI into your query assistant and figure out what is wrong with the data.
Deployment	Build application infrastructure and automation with AI
Maintenance & Support	How to find issues in a running system
Documentation	How to generate and write documents without drawing and writing them? How to turn your whiteboard drawings into digital format?
Testing and Quality Assurance	Use AI to write unit tests, test data and mocks for you
Learning and Skill Development	Turn the AI into your personal, free and patient tutor
Brownfield Considerations	A brief overview and ideas how to use AI in an old (brownfield) project.

Figure 15. A path from idea to production and maintenance.

Getting started: The ideation phase

Getting started

All projects start with a kickoff. Typically, the amount of explicit up-front requirements and material exists varies, but luckily for our iEat, there aren't any beyond the first idea.

In the following, we will show you ways to use AI to get on track in the first stages of the project. We conclude with a fast, up & running prototype to confirm our approach to iEat early on.

Collect and refine ideas

Collecting ideas, requirements, constraints and other issues to build a first roadmap for any project is something you can utilize LLMs in several ways.

Workshop Organizer: Create an agenda with tasks and goals

For instance, just to get things rolling, prompt something like this:

We're building a new restaurant table reservation system called iEat and are organizing the first ideation workshop to collect ideas and requirements for the software. Propose a workshop agenda.

And take it from there.

Ideation: Generating ideas for project features and improvements

Commercial AIs “know” pretty much about everything out there. Let’s see what kind of requirements list we get for iEat:

What kind of cool ideas you’d propose for a table booking system?

Even the first output is surprisingly to the point!



Cool Ideas for iEat Table Booking System

AI-Powered Reservation Management

- **Dynamic Table Allocation:** Use AI to optimize table allocation based on real-time data, such as booking trends, customer preferences, and table turnover rates.
- **Predictive Analytics:** Predict busy periods and suggest reservation times to customers, improving the dining experience and maximizing restaurant capacity.

Personalized User Experience

- **Customer Profiles:** Maintain detailed customer profiles, including dining preferences, past reservations, special occasions, and dietary restrictions.
- **Custom Recommendations:** Provide personalized dining recommendations based on customer preferences and past behavior.

Seamless Integration

- **POS Integration:** Integrate with Point-of-Sale systems to

streamline operations, update table statuses in real-time, and enhance customer service.

- **Third-Party Services:** Allow integration with popular third-party platforms like Google Maps, Yelp, and OpenTable for a broader reach.

Evaluate and refine requirements

Now that we have a list of key features to be done first in a plain text file, let's see if AI can suggest something more:

Consider this list of requirements for our system. Improve the requirements for clarity, suggest more features and correct errors, if any. Requirements for iEat

iEat shall be able to manage table bookings for several restaurants. Using iEat, customers may use a web interface to see available restaurants, when they are open, and make a reservation at a certain table and time. Customers shall also be able to submit custom requests and define their allergies when making a reservation. iEat shall by default prevent duplicate bookings by the same customer. Customers shall be able to cancel their reservations using information they received when they placed their booking.

The response had a nice categorization along with typo and language improvements.



The most interesting part was the suggested other features that was taken from general context, not something we inputted. Indeed, these kinds of features ought to be considered!

- **User account management:** Customers shall have the

option to create an account, allowing them to view their reservation history, save preferred restaurants, and manage their personal information.

- **Real-time notifications:** iEat shall send real-time notifications via email or SMS to confirm reservations, remind customers of upcoming bookings, and inform them of any changes or cancellations.
- **Waitlist feature:** If a desired time slot is fully booked, customers shall have the option to join a waitlist and be notified if a table becomes available.
- **Review and rating system:** Customers shall be able to rate and review their dining experience post-visit, helping other users make informed decisions.
- **Restaurant management interface:** Restaurants shall have access to a management dashboard to: View and manage all incoming reservations. Update table availability and operating hours. Access customer requests and allergy information. Generate reports on booking trends and customer feedback.
- **Integration with payment systems:** Allow optional pre-payment or reservation deposits, integrating with popular payment gateways to secure bookings.
- **Mobile application:** Develop a mobile app for iOS and Android platforms to provide a more seamless booking experience for customers on the go.
- **Analytics and insights:** Provide restaurants with analytics and insights about customer preferences, peak booking times, and other useful metrics.

Summarize material

In today's workplace, we're inundated with information from meetings, emails, instant messages, MIRO boards, Confluence, Teams channels, and other document-sharing tools. This flood of information can quickly become overwhelming. Summarizing meeting transcripts alone can feel like a daunting task.

This is where AI truly excels: summarizing data. Imagine you have dozens of transcribed interviews, workshop meeting notes, and Post-it boards that need to be synthesized into a presentation. Here's a sample workflow to manage this:

- **Collect and concatenate:** Gather all the material and combine it into a single text file.
- **Upload and summarize:** Upload the text file to your language model (LLM) and request a summary.
- **Refine as necessary:** Adjust your prompt as needed to address specific needs, such as detecting and resolving conflicts in the information.
- **Download and prepare:** Download the summary document and use it as a foundation for your presentation.

Using AI to summarize data not only saves time but also helps ensure that key insights are captured effectively and efficiently.

Preparing context for AI assistance

As we transition from the ideation and requirements gathering phase to active development, it's crucial to set up our AI assistant with comprehensive context. This preparation ensures that the AI can provide relevant, project-specific aid throughout the development process. For our iEat project, we'll configure Claude with essential information to maximize its utility.

Key components of AI context

- **Project requirements:** Upload a concise document outlining the core functionalities, user stories, and technical specifications for iEat. This forms the backbone of the AI's understanding of project goals.
- **Communication summaries:** Provide summarized transcripts from key meetings, email threads, and chat logs (Teams, Slack) that capture important decisions, discussions, and evolution of ideas. This gives the AI insight into the project's history and rationale behind certain choices.
- **Team structure:** Include a brief overview of team members, their roles, and areas of expertise. This allows the AI to tailor its responses to the proper skill level and responsibility area of each team member.
- **Project overview:** Supply a high-level description of iEat, including its purpose, target market, and any unique selling points or challenges. This context helps the AI understand the broader implications of its suggestions.
- **Technical stack:** Detail the chosen technologies, frameworks, and development methodologies. This ensures the AI's recommendations align with your technical ecosystem.
- **Project timeline:** Share key milestones and deadlines to help

the AI prioritize its aid and understand time constraints.

- **Known constraints:** Outline any budget limitations, regulatory requirements, or other constraints that might affect development decisions.

Setting up Claude

To implement this context setup in Claude, we'll use its project feature. This allows us to maintain a persistent knowledge base that Claude can reference throughout our interactions. As we will be using this project quite a lot, it makes sense to adjust custom instructions a bit to avoid unnecessary politeness:

**No small talk, rambling, leadups, conclusions or especially apologies. just be very direct and get straight to the point.
When writing code ensure it's clean and elegant**

By investing time in this initial setup, we have created a powerful, project-aware AI assistant. This contextualized AI can offer more pertinent suggestions, ask more insightful questions, and generally provide higher-quality assistance throughout the development lifecycle of iEat.

Remember to update this context as the project evolves, ensuring that Claude always has the most current information to work with. This dynamic approach to AI context management will help in maintaining the relevance and accuracy of the AI's contributions to your development process.

Setting up Cursor

In this project, we primarily utilize Claude. While Cursor, our development environment of choice, offers the option to include project-specific information through a `.cursorrules` file, it's important to note that this feature has different use cases and limitations compared to Claude. When using `.cursorrules`, it's advisable to be concise and focus on

essential coding details rather than including extensive amounts of information. This approach helps maintain efficiency and relevance in your development process.

Good examples include:

- Favor named exports for components.
- Use the “function” keyword for pure functions.
- Atomic reusable components should go to `src/components`, and strictly view specific under `src/view/my-view/`

Leveraging AI for initial ideas and architecture

With our AI assistant primed with project context, we can harness its capabilities to spark initial ideas and architectural proposals. This approach interweaves AI’s vast knowledge base with human ability, fostering a dynamic decision-making process.

The collaborative dance between AI and human insight begins with the AI generating proposals based on project requirements. We then engage in a constructive debate, challenging the AI’s viewpoints to expose potential blind spots or weaknesses. This AI-generated foundation and later critique serve as fuel for team discussions, where collective human expertise comes into play.

As ideas are refined and decisions take shape, we may find ourselves at a crossroads. In such instances, we can turn back to our AI partner for more perspectives or arbitration, helping to break deadlocks and move the project forward.

For our iEat project, we’re employing this method to navigate crucial initial tech decisions. By using AI in this way, we’re able to explore a wider range of possibilities and potential pitfalls, all while maintaining human oversight and ultimate decision-making power. This collaborative approach sets the stage for a robust, well-considered technological foundation for iEat.

Give me a first proposal for technologies used and general architecture.

And in a few seconds, we have a solid proposal for the core technology applicable in iEat.



Certainly! I'll provide you with an initial proposal for the technologies and general architecture for the iEat project based on the information provided. This proposal aims to address the requirements and challenges while using the expertise of the team members mentioned.

Eat Technology Stack and Architecture Proposal

1. Technology Stack

Backend

Primary language: Node.js

Framework: Express.js

Database: PostgreSQL (for relational data) and MongoDB (for flexible document storage)

ORM: Sequelize (for PostgreSQL)

API: RESTful API with GraphQL for complex queries

Authentication: JWT (JSON Web Tokens)

Real-time communications: Socket.io

Frontend

Framework: React.js

State management: Redux

UI component library: Material-UI

Styling: Styled components

...

The technology choices were a decent first guess, but we weren't quite

happy with them yet. So according to our preferences, we asked for modifications:

Some alterations: - prefer Prisma over Sequelize - just restful API, no GraphQL - no need for real time communication for now, or a mobile app. MVP scope is web only - Zustand instead of Redux for state management, tailwind for styling - GitHub actions for CI/CD - As for architecture, your current solution is splitting things into way too small nanoservices, keep it sane and maybe let's go with a monolithic approach for the MVP

Tirelessly polite as ever, answer started with:



Thank you for the feedback. I appreciate the direction towards a more streamlined approach for the MVP. Let's revise the proposal based on your input. ... revised plan along with a breakdown of what was changed. ...

Now that we have a document describing the technologies used and architectural decisions, we can add it to our Claude project.

Organize backlog with user story map

In previous chapters, we roughed out the initial features for the iEat system. Now it's time to organize the backlog with the team, turning our loose collection of ideas into a coherent plan.

First things first: we need a solid set of user stories. These short, simple descriptions of features from the user's perspective form the backbone of our map.

Write user stories for this project, considering obvious stuff such as registering.

Here are user stories for the iEat project, including registration and other core functionalities:



Customer User Stories

- As a customer, I want to register an account so that I can manage my reservations and preferences.
- As a customer, I want to search for available restaurants by cuisine, location, or date/time.
- As a customer, I want to view restaurant details, menus, and availability before booking.
- As a customer, I want to make a reservation for a specific date, time, and party size.
- As a customer, I want to specify dietary restrictions or special requests when booking.
- ...

The AI's first output gives us a good starting point, but it's crucial to

review and refine. We noticed stories about mobile apps and payments – features we’re not tackling in our MVP. Time for a quick course correction.

We’re not doing the mobile app for now, also payments are not in scope yet. In general, you should be more focus on MVP functionality, but aim to cover the cases needed t here 100%.

Which yields:



Understood. I’ll revise the user stories to focus on the MVP functionality, removing mobile app and payment-related stories. Here’s an updated set of user stories:

...

A picture is worth a thousand words, especially when you’re trying to get the whole team on the same page. Let’s see how AI can help us visualize our user stories.

Visualize the user stories as mermaid diagram on a user story map.



Certainly. I’ll create a Mermaid diagram to visualize the user stories for the iEat MVP.

(a decent graph but visualized in a way that doesn’t fit this book or a PowerPoint slide)

The AI’s first attempt at visualization was subpar. It gave us a complex graph that, while informative, wasn’t quite what we needed for a clear, high-level view. Time to nudge it in the right direction.

Adjust the visualization so that it's not a graph but the features are otherwise grouped, so that it works better as a picture.

User stories are great, but they need to turn into actionable tasks. Enter Jira tickets – the bread and butter of many a development team's workflow.

Populate Jira tickets out of these & the requirements. Use epics, user stories and tasks. Also include things that are purely technical and not directly parts of the user stories given. You should include priority, workload, and a short description. Output it in csv so I can easily import it to Jira

Despite how much like or not about JIRA, Claude didn't hesitate to jump right in:



Certainly. I'll create a CSV format output with Jira tickets based on the user stories and requirements, including epics, user stories, and tasks. I'll also add some purely technical items and include priority, workload (in story points), and a short description for each item.

```
Type,Summary,Description,Priority,Story
Points,Epic Link
Epic,User Management,Manage user accounts
and authentication,High,,
Epic,Restaurant Management,Handle restau-
rant registration and information,High,,
Epic,Reservation System,Manage the core
reservation functionality,High,,
Epic,Review System,Handle customer re-
views and ratings,Medium,,...
```

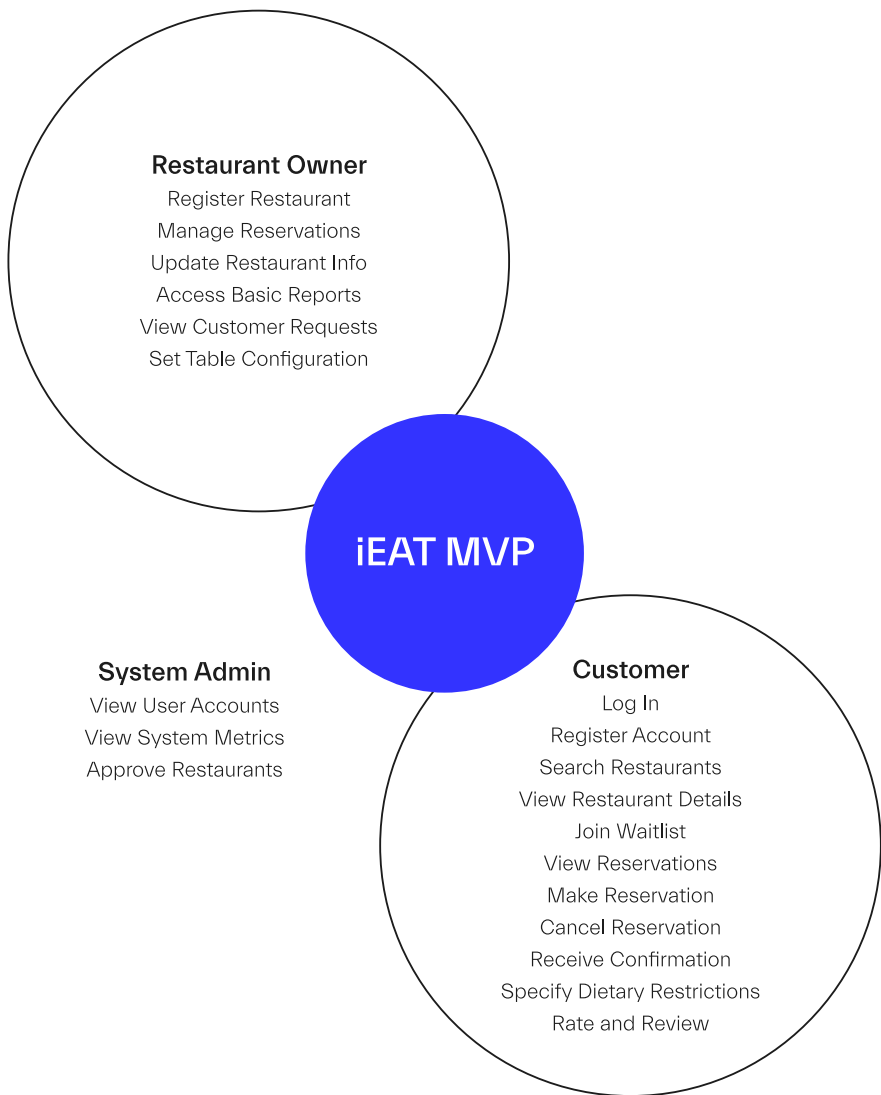


Figure 16. Visualized user stories. Redrawn for the publication.

With this CSV output, we've got a Jira-ready breakdown of our project, complete with epics, user stories, and tasks. It even includes behind-the-scenes technical work that doesn't directly map to user stories but is crucial for making the magic happen.

Finally, we need a way to showcase our plan to the higher-ups – you know, the ones who control the purse strings but might not know their APIs from their elbows.

Build a visualization of the tasks that I can show to my pointy haired boss (PHB) using tool of your choice.

In addition to being familiar with Dilbert characters, AI was smart enough to propose a first timeline for the project.



Certainly! I'll create a visualization of the tasks using a Gantt chart, which is an effective way to show project timelines and dependencies to management. This type of chart is particularly useful for displaying the structure and timeline of a project to stakeholders like your "pointy haired boss".

Voila! A Gantt chart that even the most technically challenged manager can grok. It lays out our project timeline, dependencies, and milestones in a visually appealing format. Just the thing to get buy-in from the corner office crowd.

iEAT MVP Project Timeline

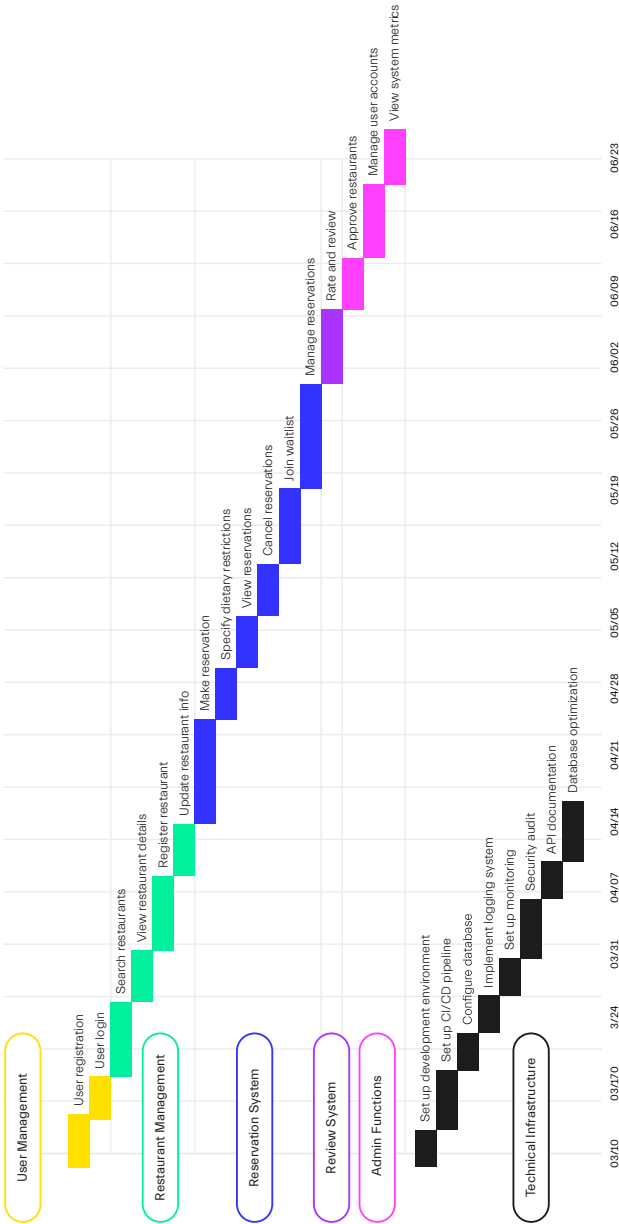


Figure 17. A Gantt chart showing project timeline, dependencies, and milestones. Redrawn for the publication.

Build prototypes quickly

Prototypes offer a subset of your system's functionality that can be tested, refined, and used to gather valuable feedback. With AI and modern web technologies, we can now create functional prototypes quickly. Let's explore how we built a working prototype for iEat, focusing on the core reservation features.

We started by asking our AI assistant to create a basic prototype using Claudes built in libraries such as Tailwind. To conserve our Claude usage quota, we started a new chat. But as the Claude project we're using already has plenty of info on the project, we don't really need to explain anything to it.

Build a functional prototype of our frontend. Login etc. can be ignored, to showcase its MVP functionality. Focus on the basics. Use placeholder data & implement it so that it can be run directly from Claude artifacts.

And in an instant:



Certainly. I'll create a basic prototype of the iEat frontend using React and Tailwind CSS, focusing on the core MVP functionality. This will include restaurant listing, reservation booking, and a simple reservation management view.

Next, we requested improvements to the UI and more fields for the reservation form.

This is great but could you make the UI prettier, and in the reservation form. Add at least name, phone number, and allergies. Add placeholder image for the restaurants.

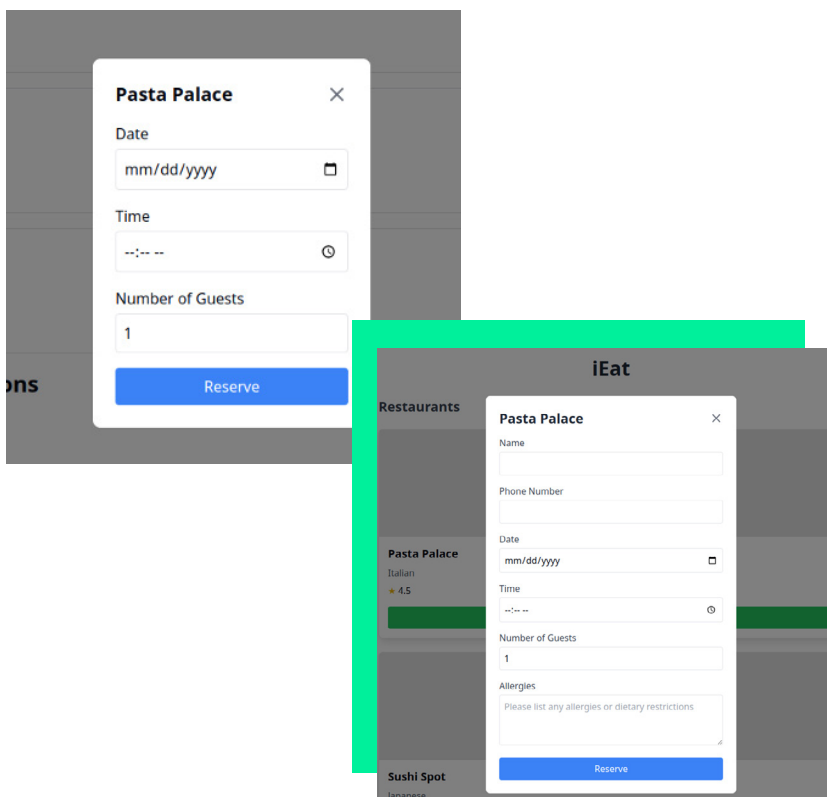


Figure 18. Two AI-generated iEAT prototypes.
The second prototype represents the core MVP functionality.

While the prototype still doesn't really meet the high standards we set before the launch of the program, this demo certainly demonstrates the speed you may add features with simple prompts.

This refined prototype now represents the core MVP functionality we're aiming for. It's at a stage where we can start asking crucial questions like, "Is this aligned with our vision?" or "Does this meet our basic requirements?" This visual representation allows stakeholders to provide

concrete feedback and helps to identify any gaps in our initial concept.

Given that our prototype is heading in the right direction, it makes sense to use the code behind it. Rather than treating this as just a visual mockup, we can extract and refine the underlying code, giving us a head start on our actual front-end development.

As we'd already decided the programming language and some parts of the stack we are going to use, let's give this a shot:

Ok close enough. Now could you extract the code in a more usable form?

Use typescript everywhere. Use Zustand for state management Have the actions be placeholders and have it return some placeholder data instead of actual API calls. Write custom hooks to access Zustand store.

Split things into smallish files and ensure cleanliness in general. (doesn't need to run in Claude anymore)

Also, while you're at it add routing and whatnot.

And now we're getting somewhere! While comparable results for typical front-end projects may be generated with standard scaffolding tools, such as create-react-app, Claude seems perfectly capable of doing all that along with the project-specific items in place!



Certainly. I'll restructure the code using TypeScript, Zustand for state management, and split it into smaller files for better organization. I'll also add basic routing. Here's a more organized and scalable structure for the iEat frontend.

```
src/
├── components/
│   ├── RestaurantCard.tsx
│   ├── ReservationForm.tsx
│   └── ReservationList.tsx
├── pages/
│   ├── Home.tsx
│   └── Reservations.tsx
├── store/
│   ├── useRestaurantStore.ts
│   └── useReservationStore.ts
├── types/
│   └── index.ts
├── hooks/
│   ├── useRestaurants.ts
│   └── useReservations.ts
├── App.tsx
└── index.tsx
```

Now, let's implement each file:

... (pretty solid implementation for each of these) ...

What we've accomplished here is remarkable. Not only have we created a visually representative mockup of our MVP, but we've also jumpstarted our frontend codebase. The restructured code provides a solid foundation for further development. The kicker? This entire process – from the first concept to the working prototype and structured codebase – took just a matter of minutes, rather than the days or even weeks it might have taken using traditional methods. This efficiency allows us to iterate quickly, gather feedback early, and accelerate our development cycle significantly.

Bootstrapping

After the first roadmap has been laid out, potentially with the help of a prototype, it's time to establish the foundation for the project. While basic scaffolding tools for most stacks are valuable, sometimes you might choose to bootstrap the project manually to minimize dependencies or undesired features.

Solution structure

Getting the major components right at the start of a project is challenging and often prone to failure. The organization of the system in the codebase and during runtime, as well as the internal and external interfaces (APIs), has a significant impact and can be difficult to change later. For example, breaking down a large module into separately hosted processes can be complex if there is a high degree of coupling.

If we look from the developer perspective, most IT systems have the following major layers in one form or another:

- **Storage:** Where the data resides “at rest” and how it's organized, partitioned, etc.
- **Business logic:** How is the data accessed and what kind of logic (transactions, chains) and rules are needed?
- **Control and security:** How users of the data are authenticated and what functions and data they are allowed to use.

- **Presentation and UX:** How the data is presented to the user. How do the users navigate and access it?
- **Integrations:** How the system exchanges data/actions with different systems.

Another viewpoint to consider is technology:

- **Runtime infrastructure:** Runtime infrastructure: How exactly is the system run—in the cloud, on Kubernetes, or in a virtual machine (VM)??
- **Development infrastructure:** What kind of tools and frameworks are being used?
- **Operations:** How is the system managed, configured and observed once it's up and running?

Layers above naturally have their own substructures which may or may not be visible to other layers, such as Onion architecture internally for a module (or even between modules/processes), microservices (or monolith) and so forth.

How ever we address these requirements while keeping different concerns separate enough, the end-result is a multidisciplinary effort. In this chapter, we focus on the application layer: how to organize our application into a manageable entity.

Now that we've reached the build phase, major things such as core architecture have already been decided and we can focus on smaller scale problems. For iEat, the remaining question is to decide on the UI framework.

For a developer, the solution structure is manifested by how the source codes are partitioned during development in the source tree and repositories. Furthermore, in runtime, by how many modules there need to be and how they interact.

Let's use Claude to set up a basic repository structure for our iEat project. We'll use the project context we set up earlier, which includes our requirements and tech stack.

First, we'll ask for the basic file and folder structure:

Provide me with the basic file and folder structure for a monorepo, including IaC and GitHub actions. Exclude frontend from it.

And we got:



Here's a basic file and folder structure for a monorepo, excluding the frontend, and including IaC and GitHub Actions:

```
ieat-monorepo/  
├── .github/  
│   └── workflows/  
│       ├── ci.yml  
│       └── cd.yml  
├── apps/  
│   └── backend/  
│       └── src/  
└── ...
```

The proposed structure looks solid, covering our backend, infrastructure, and CI/CD needs. But why stop at a mere outline when we can get Claude to do the heavy lifting?

Let's ask it to generate a bash script that will create this structure for us:

Good enough. Since it is 2024, I am not going to write any of these, could you write me a bash script that creates the structure and populates the files to sane defaults.

Always willing to work, we delegated yet another boring and tedious task to whom it belongs, i.e., the computer:



Certainly. I'll create a bash script that generates this structure and populates key files with basic content.

```
#!/bin/bash
# Create the main directory
mkdir -p ieat-monorepo
cd ieat-monorepo

# Create directory structure
mkdir -p .github/workflows apps/backend/src/{controllers,models,routes,services,utils} apps/backend/tests packages/{common/src,database/{migrations,seeds}} infrastructure/{terraform,scripts} docs

# Create files
...
```

The script Claude provided is a good start, but as with any AI-generated code, it's crucial to review and adjust. Upon inspection, we noticed a few omissions:

You missed out on couple of files. also, the backend part is supposed to use ts, not js

Quickly admitting his errors, Claude came back with a solid correction. Details of this discussion were omitted.



You're right, I apologize for those oversights. Let's correct the script to use TypeScript for the backend and include the missing files.

TIP: Sometimes LLM chats, especially the long ones, can get off track. Launch a fresh one if you feel you're not getting anywhere, and you might get better responses. Also, aim to be concise and direct..

This **iterative workflow** we just had with Claude displays how AI can accelerate the first setup of a project. It's worth noting, however, that human oversight is still crucial. AI can miss nuances or make assumptions that don't align with project specifics.

The final script gives us a solid foundation to build upon. It sets up our monorepo structure, including placeholders for our Node.js backend (now correctly using TypeScript), Terraform configurations for infrastructure, and GitHub Actions for CI/CD.

With our basic structure in place, we're now ready to dive deeper into the core components of our application. The logical next step is to focus on our data layer – the backbone of any robust application. In the next section, we'll explore how to use AI assistance in designing and implementing our database schema, ensuring it aligns with the specific needs of our iEat project.

Database design

When it comes to database design, AI can significantly accelerate the process. Instead of starting from nothing, we can leverage our AI assistant to generate a comprehensive entity map based on our project requirements and tech stack. Let's walk through this process step-by-step.

Given our tech stack and requirements, could you draw out a comprehensive map of all our database entities.

Happy to take the role of the database engineer, Claude got back to us quick with a nice ERD.



Certainly. I'll create a comprehensive entity relationship diagram (ERD) for the iEat database based on the given requirements and tech stack. This diagram will show the main entities, their attributes, and relationships.

The initial proposal is good, but it's crucial to review and refine based on our specific needs. In our case, we noticed that the handling of allergies needed adjustment:

Good, but in the UI, we let users to just elaborate on their allergies and special requests instead of using a predetermined list or anything, so allergies probably should not be distinct DB entries.

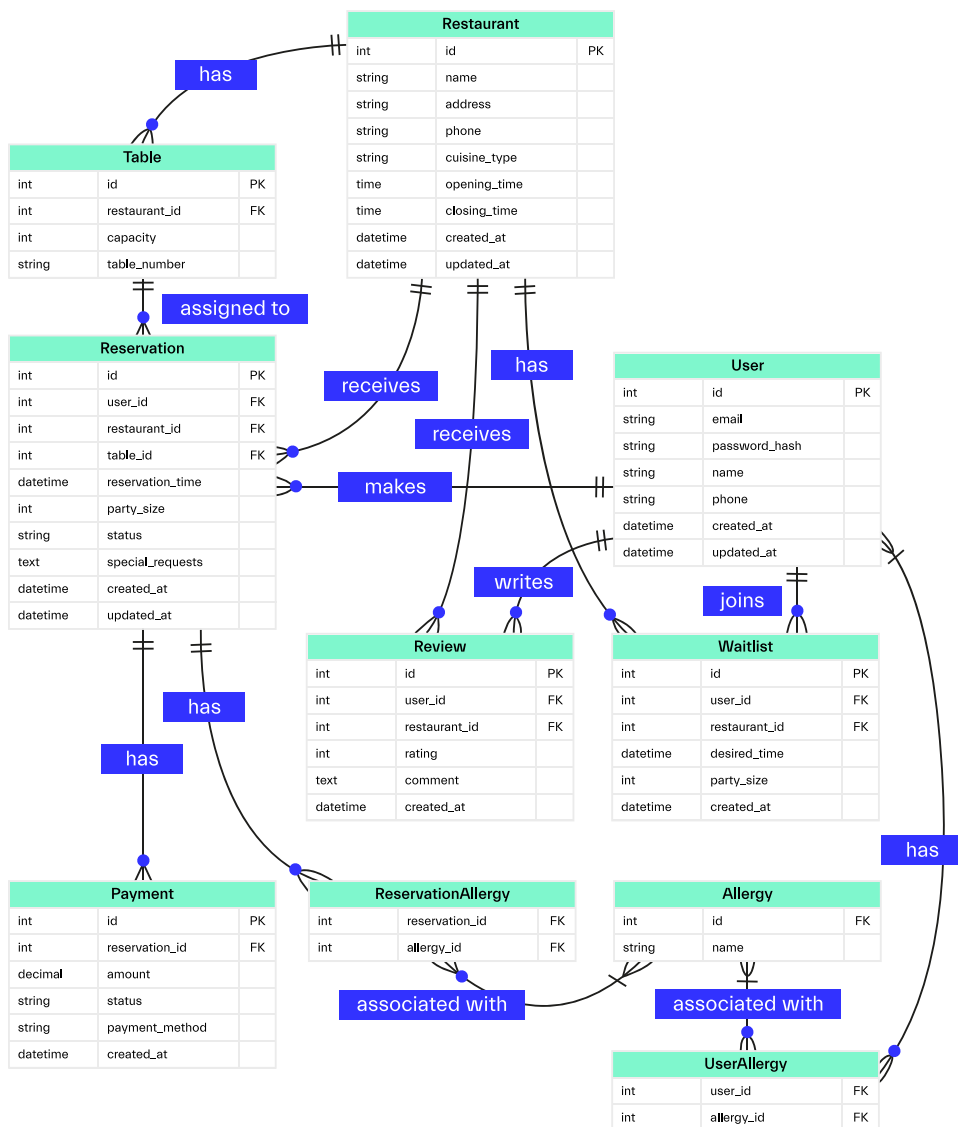


Figure 19. First outcome of the comprehensive map of all our database entities. Redrawn for the publication. We noticed that the handling of allergies needed adjustment.

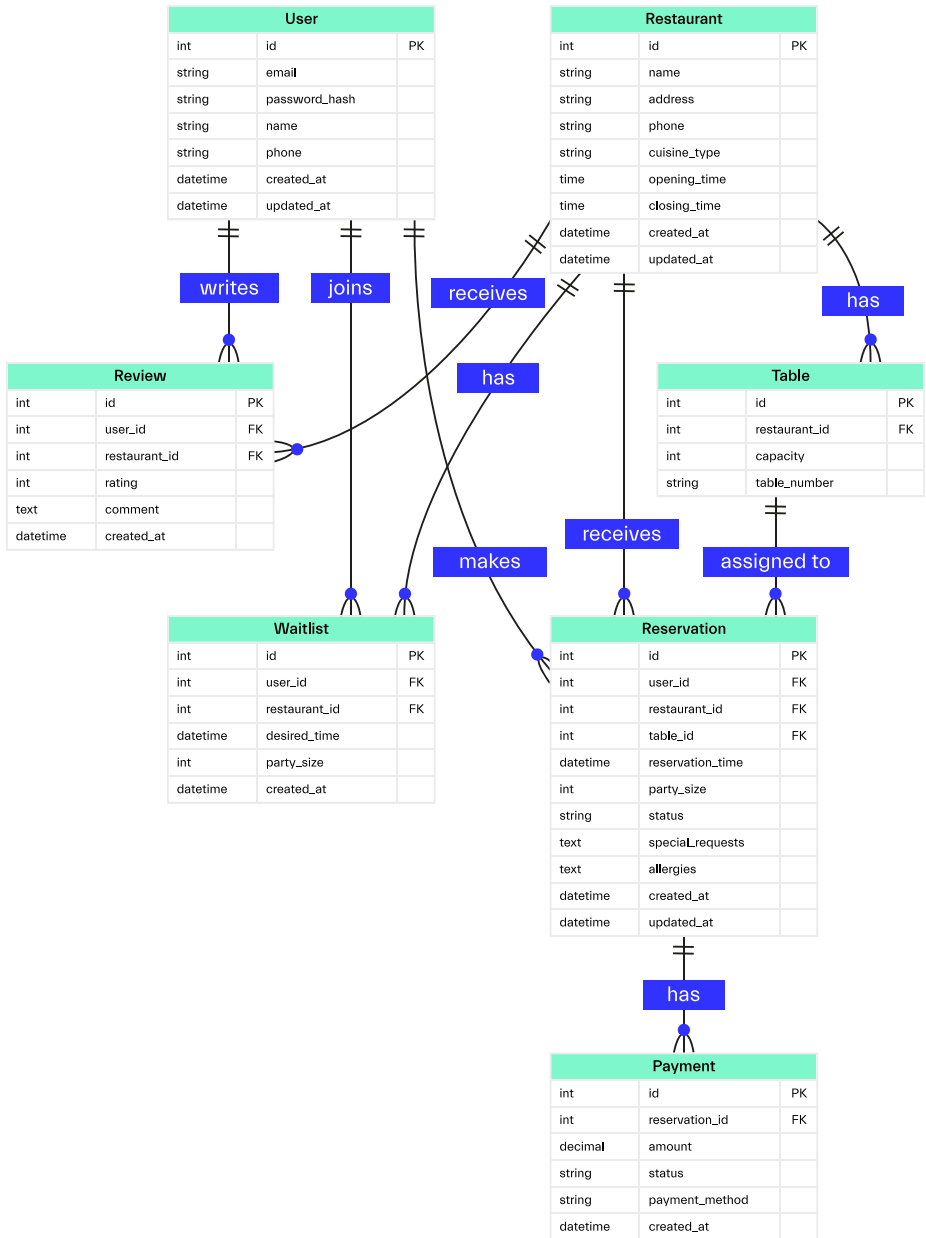


Figure 20. Second outcome - a better, revised model by Claude. Redrawn for the publication.

With our refined entity map in place, we can now ask Claude to generate the SQL schema:

Ok this is solid now. Now write me all the SQL spells to set it up.

The capabilities of this Data Engineer were not limited to drawings (sometimes referred to as hand-waving), but he turned out to be skilled in SQL, too:



Certainly. I'll provide you with the SQL statements to create the database schema based on our updated ERD.

```
-- Create User table
CREATE TABLE "User" (
    id SERIAL PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    name VARCHAR(255) NOT NULL,
    phone VARCHAR(20),
    created_at TIMESTAMP WITH TIME ZONE
    DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE
    DEFAULT CURRENT_TIMESTAMP
);

-- Create Restaurant table
CREATE TABLE Restaurant (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    address TEXT,
    phone VARCHAR(20),
    cuisine_type VARCHAR(100),
    opening_time TIME,
```

```

        closing_time TIME,
        created_at TIMESTAMP WITH TIME ZONE
        DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP WITH TIME ZONE
        DEFAULT CURRENT_TIMESTAMP
    );
    ...

```

Here's where AI really shines. With a single prompt, we can generate not just the database schema, but also the associated API, ORM, and validation schemas:

Now while you're at it, I want you to

Create rest APIs for basic CRUD operations we have.

Do ORM mapping for them.

Build zod schemas for them that we can reuse in frontend.

Aware of iEat, we got our ORM (Object-to-relational mapping) set up in a breeze:



I'll create the REST APIs, ORM models, and Zod schemas for our basic CRUD operations, adapting them to your monorepo structure. We'll focus on the User, Restaurant, and Reservation entities as examples.

Let's start with the ORM models using Prisma:

```

...
model User {
  id          Int    @id @default(autoincrement())
  email       String  @unique

```

```

    password    String
    name        String
    phone       String?
    createdAt   DateTime @default(now())
    updatedAt   DateTime @updatedAt
    reservations Reservation[]
    reviews     Review[]
    waitlists   Waitlist[]
  }

```

The Zod schemas weren't a problem either:



Now, let's create Zod schemas in the common package:

```

import { z } from 'zod';

export const UserSchema = z.object({
  id: z.number().int().positive(),
  email: z.string().email(),
  name: z.string().min(1),
  phone: z.string().optional(),
  createdAt: z.date(),
  updatedAt: z.date(),
});

```

And finally, a simple RESTful API to serve our objects all the way to the future customer of the iEat-powered restaurant was handed out to us just like that:



Now, let's create the REST API routes and controllers:

```

import express from 'express';
import restaurantRoutes from './restaurantRoutes';
import reservationRoutes from './reservationRoutes';
const router = express.Router();

```



```

router.use('/restaurants', restaurantRoutes);
router.use('/reservations', reservationRoutes);

...

const prisma = new PrismaClient();
export const createUser = async (req: Request, res: Response) => {
  try {
    const userData = UserSchema.parse(req.body);
    const user = await prisma.user.create({ data: userData });
    res.status(201).json(user);
  } catch (error) {
    res.status(400).json({ error: 'Invalid user data' });
  }
};
...

```

Why stop here? Let's try the same this with the front-end, creating an abstraction layer for API calls:

Write an API layer for frontend that abstracts the api calls.
Use Zod to validate the responses.

This was not a problem either:



Certainly. I'll create an API layer for the frontend that abstracts away the API calls and uses Zod for response validation. We'll use Axios for making HTTP requests and create a separate file for each entity.

```

...
export const reservationApi = {
  create: async (reservationData:
Omit<Reservation, 'id' | 'createdAt' |
'updatedAt'>): Promise<Reservation> => {
    const response = await api.post('/
reservations', reservationData);
    return ReservationSchema.parse(re-
sponse.data);
  },

  get: async (id: number): Promise<Reser-
vation> => {
    const response = await api.get(`/res-
ervations/${id}`);
    return ReservationSchema.parse(re-
sponse.data);
  },

```

And finally, we can set up state management – you know the fun part with all SPAs (Single Page Apps):

Build Zustand stores that use these APIs.

And we got:



Certainly. I'll create Zustand stores that utilizes the API layer we've built. These stores will manage the state for users, restaurants, and reservations, and use the API functions to fetch and update data.

```

import create from 'zustand';
import { Restaurant, restaurantApi } from
'../api';

```

```

interface RestaurantStore {
  restaurants: Restaurant[];
  currentRestaurant: Restaurant | null;
  isLoading: boolean;
  error: string | null;
  fetchRestaurants: () => Promise<void>;
  fetchRestaurant: (id: number) => Promise<void>;
  createRestaurant: (restaurantData:
Omit<Restaurant, 'id' | 'createdAt' |
'updatedAt'>) => Promise<void>;
  updateRestaurant: (id: number, restaurantData: Partial<Restaurant>) => Promise<void>;
  deleteRestaurant: (id: number) => Promise<void>;
}

```

In conclusion, the journey above process shows the power of AI-assisted development. With just a few prompts, we've went from defining a basic application concept to a working full-stack solution, complete with:

- A comprehensive Entity Map (ERD)
- SQL Schema
- REST API endpoints for CRUD operations
- ORM models
- Zod schemas for validation
- A front-end API abstraction layer
- Zustand stores for state management

While there's still room for iteration and refinement, we've made remarkable progress in a short time. This approach allows developers to focus on high-level design decisions and project-specific requirements, while the AI handles much of the boilerplate and implementation details.

Product design

Big decisions made early in a project often prove challenging to change later, primarily due to time constraints. Even with AI speeding up and improving our development workflow, this reality remains unchanged.

Typical “hard-to-change” design decisions include:

- **Overall usability and UI logic:** The fundamental concept and logic of the user interface are critical for user experience and are difficult to change once established.
- **Main design pattern:** The architectural pattern used to organize the internal workings of the system can have far-reaching implications for flexibility and scalability.
- **Runtime and hosting environment:** Choosing the runtime, hosting environment, and programming languages influences performance, compatibility, and future development.
- **Database schema:** The structure of the database affects data management and retrieval efficiency, making significant changes complex and time-consuming.

Each of these decisions depends on several factors but can be validated and explored using AI tools before fully committing to them.

UI wireframes

Creating a prototype or even a wireframe mockup illustrating the flow of actions can greatly facilitate discussions with system users. These approaches are well within the capabilities of AI tools. Sometimes, skipping the intermediate design tool altogether might be a practical strategy, though using tools like Figma to design the system's appearance remains a solid option.

We previously demonstrated the prototyping approach in Build Prototypes Quickly.

Now, let's take a different approach by asking for a graphical breakdown of the user interface flow and main data components:

Create a diagram depicting the flow of actions for the iEat reservation and cancellation process as a wireframe. Use mermaid diagrams.

This request resulted in a diagram that provides a solid starting point for discussing the user action flow.

Architectural suggestions

Designing software architecture from the key viewpoints is a team effort. While letting AI design your architecture might sound wild, do not rule it out as a “whisperer” immensely knowledgeable about this, but as somebody who might also be very foolish at times.

Defining architecture may be approached from several angles. Typically, the following viewpoints require some attention:

- Solution
- Technology
- Infrastructure
- Security

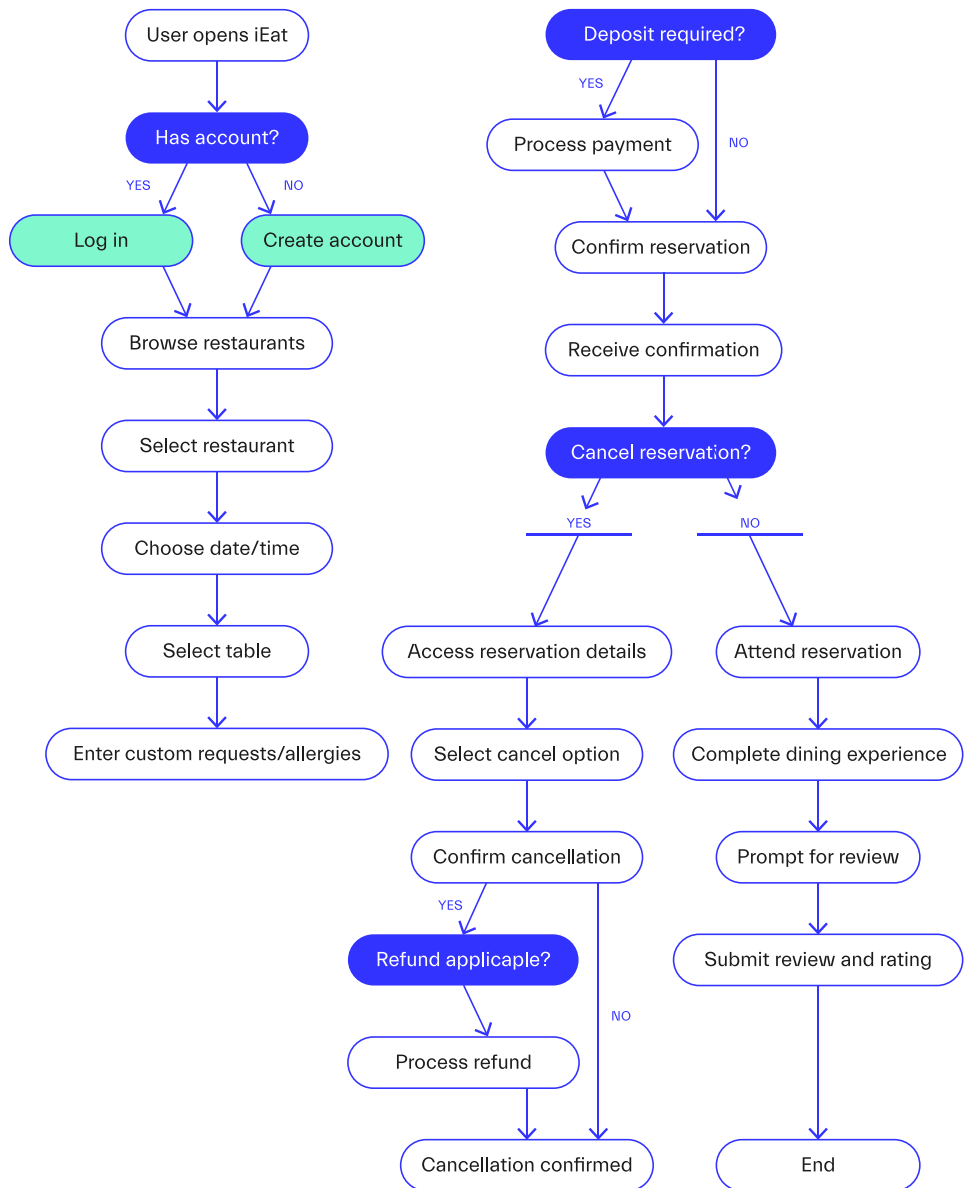


Figure 21. A graphical breakdown of the user interface flow and main data components. This provides a solid starting point for discussing the user action flow.

- Data
- Integrations
- Development

Let's try using AI from to discuss architecture with us and start with solution architecture – you know, designing the basic system structure from code point of view could be.

We're now ready to design the basic solution structure for the iEat system. We are very unlikely to build any mobile apps, so a web interface is enough. Keep the system relatively simple and use the same tools and frameworks across the stack. Suggest basic solution architecture.

Ignore the previously set technology choices, and think if from scratch, there is no need to explicitly avoid them. Justify your choices.

Think in steps and provide analysis before jumping into technological decisions.

The first response was quite lengthy and omitted for clarity. When asked for a compact summary for it we get:



The proposed iEat solution adopts a monolithic architecture with a React and TypeScript frontend, Node.js and Express backend, and PostgreSQL database.

This design emphasizes simplicity and consistency across the stack, using JavaScript/TypeScript throughout. Key technologies include Prisma for ORM, Passport.js with JWT for authentication, Socket.io for real-time notifications, and Stripe for payments.

The architecture separates concerns into distinct components (user interface, reservation management, notifications, etc.) while keeping a unified codebase, allowing for easier development, debugging, and future scalability.

Playing devil's advocate

When you have your first iteration of your design and architecture ready, it's usually a clever idea to stop for a second and either review or discuss the merits and places for improvement before starting full steam with your sprint 1.

One role, often played by the slightly grumpy senior developer is to play the devil's advocate to validate the design.

For instance, for iEat, let's question the SPA-based approach.

What options would you consider for the UI layer. Would a SSR based be a quicker start and keep the overall solution simpler?

While the SSR vs SPA debate is certainly out of scope of this book, LLM was able to point out a typical shortcoming where SEO (Search Engine Optimization) might be difficult with a pure SPA-based solution that dynamically retrieves data from a dedicated endpoint making the content inaccessible for search engine indexers. This might be a key point to consider as we're dealing with a consumer-targeted system, but certainly not the only one.



Recommendation for iEat MVP

Consider adopting a hybrid approach using Next.js:

- **Quicker Start:** Next.js provides a robust starting point with built-in SSR capabilities.
- **Simplicity:** Pages that don't require heavy interactivity (e.g., restaurant listings) can use SSR for faster initial loads.
- **Flexibility:** Interactive components (e.g., booking interface) can utilize client-side rendering for a smoother user experience.
- **Futureproofing:** Next.js allows easy transition to a more complex app as iEat grows, without major architectural changes.

This approach balances the need for a quick start with the potential for future scalability and enhanced user experience.

Due to the models being trained to be as helpful and polite as possible, they tend to be overly agreeable and adapt to your views instead of holding their ground. This can be easily mitigated and even over-adjusted by prompting.

What options would you consider for the UI layer. Would a SSR based be a quicker start and keep the overall solution simpler? Wouldn't Next.js give us the best of both worlds? Function as a very stubborn and confrontational senior developer and try to hold your ground with the initial plan.

Quickly catching the tone with the stubborn senior, Claude gets critical:



Look, I hear you bringing up Next.js, but let's cut to the chase

– we don't need it. Here's the deal:

- **Overkill for our needs:** Next.js is packed with features we simply won't use. We're building a straightforward reservation system, not complex e-commerce platform. Why bloat our project with unnecessary overhead?
- **Learning curve:** Sure, Next.js uses React, but it's still another framework to learn. We'd be slowing down development while the team gets up to speed on Next.js specifics.
- **We're not chasing SEO:** Next.js's SSR capabilities are great for content-heavy sites that need search engine optimization. That's not us. Our dynamic content doesn't benefit much from SSR.
- **Unnecessary complexity:** Next.js's routing and build system adds another layer of complexity we don't need. Our current setup with React Router is more than sufficient.
- **Performance is already solid:** With proper code splitting and lazy loading in our React app, we're already covering our performance bases. The marginal gains from Next.js's optimizations aren't worth the trade-offs for our use case.
- **Flexibility:** Sticking with our current React setup gives us more control over our build process and deployment. We're not locked into Next.js's opinions on how things should be done.

Bottom line: Next.js is a solution to problems we don't have.



We've got a plan that works, a team that knows how to execute it, and a timeline to meet. Adding Next.js or any other SSR tool at this point is just scope creep that'll slow us down. Let's focus on building the product instead of chasing shiny new tech, all right?

While you certainly need to challenge the plans that AI has, it is also very useful to turn the tables and **have AI challenge yours**. Among real people, there is always some amount of group think, and many people struggle with criticism. As it's been since the dawn of the computers, many engineers regard their favorite technology, programming language, front end stack, database or whatever as the **best solution for everything** making exploring alternatives mentally impossible.

AI does not have these traits and can be almost as brutal of a critic as you. So having flame wars with AI instead of the guy in the next cubicle, you will save a lot of sanity points. Don't we all have that guy somewhere who thinks Kubernetes is the best thing since sliced bread and is **very** persistent about it?

Kidding aside, let's have Claude challenge our first technical plans.

Take our technical plans and play devil's advocate with them. Be as brutal and harsh as possible and try to shoot everything down, keeping in mind that this is the best place to catch any / all flaws we might have.

Given permission to speak it's mind out, Claude didn't save any shots:



Here's a critical analysis of the technical plans for iEat: <Long list of criticism, ranging from "meh", to "we can live with this" to "ok, we probably need to consider this">

Most stinging parts were reasonable and to the point.

- Zustand is relatively new and unproven compared to Redux or MobX, and it has limited ecosystem and community support.
- Stateless nature of JWT makes token revocation difficult.
- No consideration for handling conflicts, overbookings, or complex scheduling scenarios
- No plans for multi-language or multi-currency support, limiting global scalability.
- No mention of load testing or stress testing.
- PostgreSQL lacks built-in horizontal scaling capabilities.
- Critical for a reservation system to have instant updates.

TIP: Remember that the big LLMs probably ‘know’ more than you do. Using AI to verify your choices and to explore alternatives is certainly superior to googling or stackoverflowing the same.



Development

Generating code

Perhaps the most obvious use case for LLMs is code generation. You may generate code from scratch or change some existing code. Tools at your disposal include direct prompting, sometimes as simply as by asking explicitly for something to be generated.

Completions

Autocomplete in modern IDEs shines in handling trivial coding tasks swiftly and effortlessly. While it's not doing anything you couldn't manage yourself, it saves keystrokes and mental bandwidth. The AI models powering these features differ from chat interfaces, optimized for frequent, cheap, and low-latency predictions. This trade-off results in less capability for complex reasoning but excellent performance in routine coding tasks. Autocomplete typically relies on limited context – the current file, open tabs, and project dependencies – contributing to both its speed and limitations.

Contrary to some advice, writing comments before code to guide autocomplete often leads to subpar results. This approach tends to describe the “how” rather than the “why,” cluttering your codebase with obvious remarks. A well-crafted method signature should provide sufficient context for both AI and human readers, communicating purpose and expected behavior without redundant comments.

Consider this example of autocomplete in action:

```
export const createUser = async
(req: Request, res: Response) => {
  try {
    const userData = UserSchema.
parse(req.body);
    const user = await prisma.user.cre-
ate({ data: userData });
    res.status(201).json(user);
  } catch (error) {
    res.status(400).json({ error:
'Invalid user data' });
  }
};
```

In this case, autocomplete excels at filling in the standard implementation details. It correctly infers the need to extract the ID from the request parameters, query the database, and handle both successful and error cases. This shows how autocomplete can streamline routine coding tasks, allowing developers to focus on more complex aspects of their work.

IDE assistant chat

IDE integrations using more sophisticated AI models offer capabilities that go beyond simple autocomplete. These tools can search through your entire codebase to bring relevant files into context. This broader perspective allows for more intelligent suggestions and a deeper understanding of your project structure.

These advanced integrations employ smarter models than traditional autocomplete, making them particularly useful for bread-and-butter coding tasks. They excel at generating boilerplate, implementing common patterns, and even suggesting entire function implementations

based on context and naming conventions.

One of the key advantages of these IDE-integrated tools is the improved user experience compared to copying and pasting between your editor and external AI chatbots. This seamless integration makes it easier to adjust your workflow and leverage AI more aggressively in your development process. You're more likely to ask for AI aid when it's just a keystroke away, rather than a context switch to another application.

So, for example, instead of finding the piece of code in question and copy pasting it to another window, you may just highlight the part you want and have that included in the query context directly. In the example below, we've selected a line of code and started invoking CoPilot's building snippet for "fix".

While these advanced IDE assistants offer significant benefits, it's worth noting that most still require manual interaction to apply changes. They typically suggest code or actions but leave the final decision and implementation to the developer. This approach keeps a balance between AI aid and human control over the codebase.

Looking ahead, we're seeing the emergence of more agentic workflows, as highlighted by tools like Amazon Q developer agent or Cursor's new composer feature. These point towards a future where AI assistants can take on more complex, multi-step tasks with minimal human intervention.

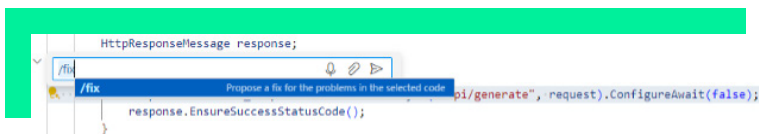


Figure 22. In the example above, we've selected a line of code and started invoking CoPilot's building snippet for "fix".

External LLMs

When it comes to code generation, external Large Language Models (LLMs) like ChatGPT and Claude truly shine in complex scenarios. These tools are most valuable when you're facing tasks that go beyond simple autocomplete – the kind where you'd typically want to sit down with a colleague and hash out ideas.

External LLMs excel at generating larger code structures, entire functions, or even full modules. They can consider broader contexts and requirements, making them ideal for tasks that require a deeper understanding of the project's architecture and goals.

One of the key advantages of using custom GPTs or Claude Projects is the ability to provide more comprehensive context. You can feed these systems not just code snippets, but entire codebases, along with requirements and user stories. This broader perspective allows the AI to generate code that's more aligned with your project's specific needs and conventions.

For particularly complex code generation tasks, external LLMs offer the luxury of a vast context window. You can paste in large chunks of your existing codebase, giving the AI a holistic view of your project's structure and style. This proves invaluable when you need to generate code that integrates seamlessly with your existing systems.

Explaining

Reading code written by others, or even code you wrote yourself a while ago, can be tedious—especially when there is confusing naming and insufficient comments. Instead of spending a lot of time parsing the code, ask AI to do it for you!

In this example, we're using Claude, but you could also do this directly from IDE by highlighting the code and using the `/explain` feature.

Give me a brief rundown on what's happening here:

```
router.post('/reservation', async (req,
res) => {
  const { restaurantId, userId, date, time,
partySize } = req.body;

  const lockKey = `reserva-
tion:lock:${restaurantId}:${date}:${-
time}`;
  const lockValue = uuidv4();

  try {
    const lock = await redis.set(lockKey,
lockValue, 'NX', 'PX', 10000);
    if (!lock) {
      return res.status(409).json({ er-
ror: 'Concurrent booking attempt' });
    }

    const availableTables = await pris-
ma.$queryRaw`
      SELECT t.id, t.capacity
      FROM "Table" t
      LEFT JOIN "Reservation" r ON t.id =
r."tableId"
      AND r.date = ${date}
      AND r.time = ${time}
      WHERE t."restaurantId" = ${restau-
rantId}
      AND t.capacity >= ${partySize}
      AND r.id IS NULL
      ORDER BY t.capacity ASC
      LIMIT 1
    `;
  }
```

```

    if (availableTables.length === 0) {
      return res.status(400).json({ error: 'No available tables' });
    }

    const table = availableTables[0];

    const reservation = await prisma.reservation.create({
      data: {
        userId,
        restaurantId,
        tableId: table.id,
        date,
        time,
        partySize,
        status: 'CONFIRMED',
      },
    });

    await redis.publish('reservations',
      JSON.stringify(reservation));

    const pipeline = redis.pipeline();
    pipeline.zadd(`restaurant:bookings:${restaurantId}`, Date.now(), reservation.id);
    pipeline.hincrby(`restaurant:stats:${restaurantId}`, 'total_bookings', 1);
    await pipeline.exec();

    res.json(reservation);
  } catch (error) {

```

```

        console.error(error);
        res.status(500).json({ error: 'Internal server error' });
    } finally {
        const script = `
            if redis.call("get", KEYS[1]) ==
ARGV[1] then
                return redis.call("del", KEYS[1])
            else
                return 0
            end
        `;
        await redis.eval(script, 1, lockKey, lockValue);
    }
});

```

Yields:



This code snippet is a route handler for creating a restaurant reservation. Here's a brief rundown of what's happening:

It's a POST route for '/reservation' that handles reservation requests.

- It uses Redis for distributed locking to prevent concurrent booking attempts for the same time slot.
- It queries a database (likely PostgreSQL, given the use of Prisma) to find an available table that meets the party size requirements.
- If a suitable table is found, it creates a new reservation in the database.

- It publishes the reservation details to the Redis channel for potential real-time updates.
- It updates some statistics in Redis about the restaurant's bookings.
- Finally, it releases the distributed lock using a Lua script to ensure atomicity.

The code employs several technologies and concepts:

- Express.js (or similar) for routing
- Redis for locking and pub/sub
- Prisma ORM for database operations – SQL raw queries for complex table availability checks.

This implementation aims to manage concurrent requests safely and maintain consistent booking data across a distributed system.

Using explaining features might reveal gaps in your logic, too! Use it before you commit.

Code commenting

Code comments, when used effectively, enhance code readability and maintainability. AI can assist in creating meaningful comments while encouraging better coding practices.

Docstrings

Docstrings are the primary tool for code documentation. They describe the purpose, parameters, and return values of functions, classes, and modules. Unlike inline comments, docstrings are accessible through introspection, helping both developers and users. Writing and maintaining docstrings can be tedious. AI-assisted development tools like GitHub Copilot streamline this process. Most IDE assistants now offer

a `/doc` command that generates docstrings based on function signatures and content.

```
/doc
router.post('/reservation', async
(req, res) => {
...
/**
 * Express router for handling
 * reservation requests.
 *
 * @remarks
 * This router handles the creation of
 * reservations for a restaurant.
 * It uses Redis for locking and Prisma
 * for database operations.
 *
 * @example
 *
 * const router = require('./reservation-
 * Router');
 * app.use('/api/reservation', router);
 *
 */
```

Inline comments

Inline comments can help decipher complex code, whether written by others or your past self. AI can explain confusing code blocks or suggest proper comments. However, this is often a stopgap solution.

Add comments.

And Claude filled in the blanks for us:



```
router.post('/reservation', async (req,
res) => {
  ...
  router.post('/reservation', async (req,
res) => {
    // Extract the necessary data from
the request body
    const { restaurantId, userId, date,
time, partySize } = req.body;

    // Generate a unique lock key and
value for the reservation
    const lockKey = `reserva-
tion:lock:${restaurantId}:${date}:${-
time}`;
    const lockValue = uuidv4();

    try {
      // Set the lock in Redis with a
10-second expiration time
      const lock = await redis.
set(lockKey, lockValue, 'NX', 'PX',
10000);
      if (!lock) {
        // If the lock couldn't be
acquired, return a 409 Conflict error
        return res.status(409).json({
error: 'Concurrent booking attempt' });
      }
    }
    ...
  }
}
```

Needing to explain code with comments usually shows that the code needs refactoring. As Robert C. Martin states, “The proper use of

comments is to compensate for our failure to express ourselves in code.”

Refactoring

Constant refactoring not only keeps code clean and efficient but covers much of what you do daily as a developer: much of the work we do daily is not building something entirely new. AI tools can streamline this process, but they’re most effective when combined with traditional methods and human judgment.

AI excels at code cleanup, especially with clear instructions about desired outcomes, coding styles, and preferred patterns. For repetitive tasks, consider creating a custom GPT or Claude project with your preferences embedded in the prompt.

```
Refactor this to align with clean code principles.  
router.post('/reservation', async (req, res) => {  
  ...
```

And the result indeed was an improvement on what we had:



```
const router = express.Router();  
const prisma = new PrismaClient();  
const redis = new Redis();  
  
const LOCK_EXPIRATION = 10000; // 10  
seconds  
  
router.post('/reservation', async (req,  
res) => {  
  try {  
    const reservationData = extractReser-  
vationData(req.body);
```

```

        await processReservation(reservation-
Data, res);
    } catch (error) {
        handleError(error, res);
    }
});

function extractReservationData(body) {
    const { restaurantId, userId, date,
time, partySize } = body;
    return { restaurantId, userId, date,
time, partySize };
}

```

Despite AI's power, don't neglect traditional IDE refactoring tools. Renaming a method used across multiple files, for instance, is difficult for AI but trivial with built-in IDE features.

Complex refactoring lacks a universal solution. It typically involves a mix of manual analysis, IDE tools, AI-assisted transformations, and creative problem-solving. Use AI for generating ideas and handling repetitive tasks, IDE tools for precise changes, and your expertise for overall direction and quality control.

Fixing errors

AI can significantly streamline the debugging process.

The default strategy is simple: copy-paste the error message and relevant code into an AI assistant and ask for a fix. This often yields quick results for straightforward issues.

Please fix this:

```

async function getReservationsWithMenu-
Items(restaurantId: number) {

```



```

    try {
      const reservations = await prisma.
reservation.findMany({
  where: { restaurantId },
  include: {
    user: true,
    restaurant: {
      include: {
        menuItems: true
      }
    }
  }
});

    return reservations.map(reservation
=> ({
  id: reservation.id,
  userName: reservation.user.name,
  restaurantName: reservation.restau-
rant.name,
  menuItems: reservation.restaurant.
menuItems.map(item => item.name)
}));
  } catch (error) {
    console.error('Error fetching reser-
vations:', error);
    throw error;
  }
}

```

```

TypeError: Cannot read properties of un-
defined (reading 'map')
    at /path/to/your/file.ts:20:59
    at Array.map (<anonymous>)
    at getReservationsWithMenuItems (/

```

```
path/to/your/file.ts:18:25)
    at processTicksAndRejections
(node:internal/process/task_queues:95:5)
```

By giving the relevant segment to Claude with the explanation gave us a solid fix candidate:



It seems the error is occurring because `reservation.restaurant.menuItems` is undefined. This could happen if the `menuItems` relation is not properly loaded or if some reservations don't have associated restaurants or menu items. Let's modify the function to handle this case more gracefully:

```
async function getReservationsWithMenu-
Items(restaurantId: number) {
  try {
    const reservations = await prisma.
reservation.findMany({
  where: { restaurantId },
  include: {
    user: true,
    restaurant: {
      include: {
        menuItems: true
      }
    }
  }
});

    return reservations.map(reservation
=> ({
  id: reservation.id,
  userName: reservation.user?.name ??
'Unknown User',
  restaurantName: reservation.restau-
```

```
rant?.name ?? 'Unknown Restaurant',
    menuItems: reservation.restaurant?.
menuItems?.map(item => item.name) ?? []
    ));
    } catch (error) {
        console.error('Error fetching reser-
vations:', error);
        throw error;
    }
}
```

NOTE: A similar feature is easily accessible to GitHub Copilot users, who can use the `/fix` command directly in their IDE without the need for copy-pasting. While this can be efficient, complex issues often benefit from more iterative approaches. Starting with `/fix` and then moving to a more detailed conversation with an AI assistant often works well.

Context is crucial for complex errors. Provide all relevant logs, error messages, and code files to the AI. With large context windows, especially in models like Claude, you can input substantial code chunks for analysis when the error source isn't obvious.

Be aware that AI models can sometimes get stuck in loops, proposing previously unsuccessful approaches. If this happens, start a new chat with a summary of failed attempts. This reset often leads to fresh perspectives.

For esoteric or internal issues where general AI struggles, consider building a custom GPT or Claude project with relevant documentation in its context. If this doesn't help, or isn't worth the effort, you may need to fall back on traditional debugging methods. It might feel archaic, but as a professional, you should be capable of solving issues without AI aid when necessary.

Code review

While automated code review tools like CodeRabbit and CodiumAI PR Agent streamline the review process, AI can provide an additional layer of insight, acting as a second pair of eyes on your code with minimal friction. This “four-eyes” principle can significantly enhance code quality and catch issues that might otherwise slip through.

For quick improvements on individual files, you can use GitHub Copilot’s @workspace feature, which provides rapid suggestions. If you need a more comprehensive review, you can opt for a deep dive by pasting code snippets into AI assistants like Claude or ChatGPT, allowing for a more thorough analysis and feedback on your code.

To maximize the effectiveness of AI-assisted code review, consider setting up a custom AI environment:

- Create a Claude project or custom GPT **specifically for code review**.
- **Include your team’s coding standards**, style guides, and best practices in the AI’s context.
- **Attach your linter rules** to ensure AI suggestions align with your automated checks.
- **Specify areas of focus** that go beyond what linters typically catch, such as architectural concerns or performance optimizations.

This customized setup allows the AI to provide more relevant and project-specific feedback.

Examples

AI excels at finding logical errors and edge cases that human reviewers might overlook. In this example, we’ll see how an AI assistant can spot subtle bugs in a TypeScript Express route handler for updating menu items.

Check this code for any bugs:

```
app.put('/menu-items/:id', async (req:
Request, res: Response) => {
    const { id, name, description, price,
available, errorMessage } = validateAnd-
GetMenuItemParams(req);
    if (errorMessage) {
        return res.status(400).json({ error:
errorMessage });
    }

    try {
        const menuItem = await fetchMenu-
Item(id);
        if (!menuItem) {
            return res.status(404).json({ er-
ror: 'Menu item not found' });
        }

        const updatedItem = {
            ...menuItem,
            name: name || menuItem.name,
            description: description || menu-
Item.description,
            price: price || menuItem.price,
            available: available === undefined
? menuItem.available : available,
            lastUpdated: dateUtils.utcnow()
        };

        const result = await updateMenu-
Item(updatedItem);
        return res.status(200).json(result);
    } catch (error) {
        console.error('Error updating menu
```

```
item:', error);  
    return res.status(500).json({ error:  
    'Internal Server Error' });  
  }  
});
```

And some indeed were found:



After reviewing the code, there are a couple of potential issues to consider:

Unintended overwriting of false values: The code uses the logical OR operator (||) to update fields. This can lead to unintended behavior if you want to explicitly set a field to a false value (like an empty string or 0).

For example:

```
name: name || menuItem.name,  
description: description || menuItem.de-  
scription,  
price: price || menuItem.price,
```

If a user wants to set the name to an empty string or the price to 0, this code will keep the old value instead.

Inconsistent handling of boolean available: The available field is handled differently from the others:

```
available: available === undefined ?  
menuItem.available : available,
```

This correctly allows setting available to false, but it's inconsistent with how other fields are handled. It might be confusing for developers maintaining the code.

Potential for unnecessary updates: The code always updates

the `lastUpdated` field and calls `updateMenuItem`, even if no fields have changed. This could lead to unnecessary database operations.

Lack of input sanitization: While the code assumes that `validateAndGetMenuItemParams` does its job correctly, it's worth noting that there's no explicit sanitization of inputs in this snippet. Depending on how `updateMenuItem` is implemented, this could potentially lead to security issues.

No validation of updated fields: The code doesn't re-validate the updated fields. If `validateAndGetMenuItemParams` only validates the input and not the resulting object, there could be issues if, for example, combining the new and old values results in an invalid state.

To address these issues, you might consider:

- Using a different method to update fields that allows explicit setting of false values.
- Consistently handling all fields, including `available`.
- Only updating fields that have changed.
- Adding explicit input sanitization if not handled by `validateAndGetMenuItemParams`.
- Re-validating the final object before calling `updateMenuItem`.

As shown above, AI can quickly identify issues like the inability to set fields to empty values due to the use of the logical OR operator. It's worth noting that while the AI's response was quite verbose, it found

the intended bug along with other legitimate concerns, such as potential unnecessary updates and lack of input sanitization. As a reviewer, you have the flexibility to focus on the most relevant points and ignore others as needed.

Security vulnerabilities are another area where AI can offer valuable insights. Let's examine how an AI assistant can find and suggest fixes for potential security issues in a Node.js login route.

Scan this code for vulnerabilities:

```
app.post('/login', async (req: Request, res: Response) => {
  const { username, password } = req.body;

  if (!username || !password) {
    return res.status(400).json({ error: 'Username and password are required' });
  }

  try {
    const query = 'SELECT * FROM users WHERE username = $1';
    const result = await pool.query(query, [username]);

    if (result.rows.length === 0) {
      return res.status(401).json({ error: 'Invalid credentials' });
    }

    const user: User = result.rows[0];
    const isValid = await bcrypt.compare(password, user.password);

    if (!isValid) {
      return res.status(401).json({ error: 'Invalid credentials' });
    }
  }
});
```



```
const token = jwt.sign({ id: user.id, username: user.username
}, JWT_SECRET, { expiresIn: '1h' });
res.json({ token });
} catch (error) {
  console.error('Error logging in:', error);
  res.status(500).json({ error: 'Internal server error' });
}
});
```

And just like with the finding bugs-scenario above, we indeed got good observations what we need to analyze:



After analyzing the code, I've identified several potential vulnerabilities and security concerns:

- **SQL Injection:** While the code uses parameterized queries (\$1), which is good practice, it's worth double-checking that the pool.query function properly handles parameter sanitization.
- **Timing Attack Vulnerability:** The error messages for invalid username and invalid password are identical, which is good. However, the time taken to respond might differ between these two cases, potentially allowing timing attacks.
- **Weak Password Policy:** There's no apparent check for password strength. This could lead to weak passwords in the system.
- **Token Security:** The JWT token contains the user's ID and username. While not inherently a vulnerability, it exposes potentially sensitive information.
- **Token Expiration:** The token expires in 1 hour. Depending

on the application's security requirements, this might be too long.

- **Missing Rate Limiting:** There's no apparent rate limiting, which could make the endpoint vulnerable to brute-force attacks.
- **Error Handling:** The catch block logs the full error, which might expose sensitive information in logs if not properly sanitized.
- **HTTPS Not Enforced:** The code doesn't ensure the request is made over HTTPS, potentially exposing credentials if used over an insecure connection.
- **Missing HTTP Headers:** Important security headers (like X-XSS-Protection, X-Frame-Options, etc.) are not set in this endpoint.
- **Lack of Input Validation:** There's minimal input validation (only checking if username and password exist). More thorough validation could prevent potential attacks.
- **JWT Secret Security:** The code uses a JWT_SECRET. Ensure this is stored securely and not hard-coded.
- **Password Storage:** While bcrypt is used (which is good), ensure the salt rounds are sufficient (typically 10 or more).
- **Session Management:** There's no apparent session invalidation mechanism. Consider implementing a way to invalidate tokens on logout or password change.
- **Verbose Error Messages:** The error messages, while not overly specific, could potentially be made even more

generic to avoid leaking information about the system's internals.

- **To enhance security**, consider addressing these points, implementing additional security measures like rate limiting and comprehensive input validation, and following OWASP security best practices.

In this case, the AI found security considerations, including the intended vulnerability (potential timing attacks). It also highlighted other critical issues such as the lack of rate limiting, potential for SQL injection (though mitigated by parameterized queries), and missing HTTPS enforcement.

This comprehensive analysis shows the AI's ability to provide a thorough security review, often catching issues that might be overlooked in a manual review process.

Beyond finding bugs and vulnerabilities, AI can also suggest improvements to make your code more elegant, readable, and maintainable. Let's look at how an AI assistant might suggest refactoring an Express.js endpoint.

Review this code for general elegance:

```
app.post('/book', (req: Request, res: Response) => {
  const { userId, restaurantId } = req.body;

  if (!userId || !restaurantId) {
    return res.status(400).json({ error:
      'User ID and Restaurant ID are required'
    });
  }
});
```

```

    }
    const user = users.find(u => u.id ===
userId);
    if (!user) {
        return res.status(404).json({ error:
'User not found' });
    }

    const restaurant = restaurants.find(r
=> r.id === restaurantId);
    if (!restaurant) {
        return res.status(404).json({ error:
'Restaurant not found' });
    }

    if (restaurant.availableTables <= 0) {
        return res.status(400).json({ error:
'No tables available for booking' });
    }

    if (user.bookings.length >= 3) {
        return res.status(400).json({ error:
'User has reached the maximum number of
bookings' });
    }

    restaurant.availableTables--;
    user.bookings.push(restaurantId);

    // Update the restaurant and user in
their respective arrays
    restaurants = restaurants.map(r => r.id
=== restaurantId ? restaurant : r);
    users = users.map(u => u.id === userId
? user : u);

```

```
res.json({ message: 'Table booked successfully', user, restaurant });
});
```

For the general elegance or aesthetics, there were also several remarks:



Focusing on the elegance of implementation, here are some observations and suggestions for improvement:

Early Returns: The code uses early returns for error cases, which is good for readability and reduces nesting.

Repetitive Pattern: There's a repetitive pattern for checking existence and availability. This could be abstracted into helper functions for more elegance.

...

A more elegant implementation might look like:

```
app.post('/book', async (req: Request,
res: Response) => {
  const { userId, restaurantId } = req.
body;

  if (!userId || !restaurantId) {
    return res.status(400).json({ error:
'User ID and Restaurant ID are required'
});
  }

  try {
    const user = await findEntity(users,
userId);
    const restaurant = await findEnti-
ty(restaurants, restaurantId);
```

```

        if (!user || !restaurant) {
            return res.status(404).json({ error: 'User or Restaurant not found' });
        }

        const { updatedUser, updatedRestaurant } = bookTable(user, restaurant);

        await updateEntity(users, updatedUser);
        await updateEntity(restaurants, updatedRestaurant);

        res.json({ message: 'Table booked successfully', bookingId: generateBookingId() });
    } catch (error) {
        res.status(400).json({ error: error.message });
    }
});

```

These suggestions for improving code structure, readability, and modularity showcase how AI can contribute to maintaining high code quality standards. By offering insights into better coding practices, AI can help developers continually improve their skills and produce more maintainable code.

Committing code

Writing good and descriptive commit messages is often seen as a nuisance. In certain settings, where for example work ticket references are considered enough you might not need to bother that much with writing, granted all tickets are accurately described and easily accessible via git.

Fortunately, tools, such as Copilot can analyze your uncommitted changes and make quick summaries that help the maintenance and e.g. RCA analysis later. In case you have pull requests (PR) in your workflow, well written commit messages translate into less writing when making the PR.

Create a commit message.

And again, we were able to save some time by handing out the tedious writing task to AI.



Fix typo in `cancelReservation` method and improve error handling

- Corrected variable name from `reservationIdW` to `reservationId`.
- Enhanced error messages for better clarity.

In a real world setting you might need to be a bit more verbose and to include things like related ticket numbers and some reasoning behind the fix.

Working with data

SQL is well understood by all major LLMs. You can easily generate a query from the schema definition, and as these tasks are reoccurring it makes sense to add your definitions to a Custom GPT or a Claude Project.

Building queries

During the first phases of the iEat system, we had one feature request to order the available times visible in the booking UI in a way that encourages us to reserve a table to typically silent hours.

We figured that this could be most easily done with SQL queries.

Create a select query for available times that orders results in a way that times typically unreserved are shown first.

And got a reasonable starting point as a response. Coming up with a similar from scratch would have needed considerable effort.



```
WITH time_slots AS (  
  SELECT generate_series(  
    restaurant.opening_time::time,  
    restaurant.closing_time::time,
```



```

        '30 minutes'::interval
    ) AS slot_time
    FROM restaurant
    WHERE restaurant.id = :restaurant_id
),
reservation_counts AS (
    SELECT
        DATE_TRUNC('hour', reservation_
time) +
            EXTRACT(MINUTE FROM reserva-
tion_time)::int / 30 * INTERVAL '30 min'
    AS slot_time,
        COUNT(*) AS reservation_count
    FROM reservation
    WHERE restaurant_id = :restaurant_id
    AND DATE(reservation_time) = :target_
date
    GROUP BY 1
)
SELECT
    ts.slot_time::time AS available_time,
    COALESCE(rc.reservation_count, 0) AS
popularity
FROM
    time_slots ts
LEFT JOIN
    reservation_counts rc ON ts.slot_time
= rc.slot_time
WHERE
    NOT EXISTS (
        SELECT 1
        FROM reservation r
        JOIN "Table" t ON r.table_id =
t.id
        WHERE r.restaurant_id = :restau-
rant_id

```

```

        AND DATE(r.reservation_time) =
:target_date
        AND r.reservation_time = ts.slot_
time
        GROUP BY t.restaurant_id
        HAVING COUNT(*) >= (SELECT
COUNT(*) FROM "Table" WHERE restaurant_id
= :restaurant_id)
    )
ORDER BY
    popularity ASC,
    available_time ASC;

```

Troubleshooting SQL and database issues

Widely speaking AI tools have gotten already so good, that they should be your first go-to place to figure out why an error occurred. This applies also to data and SQL domain. Parsing query errors and database errors is no exception.

AI tools have become remarkably proficient, making them the ideal starting point for diagnosing errors. This principle extends to the data and SQL domain as well.

Parsing query errors and database issues is no exception. Here are some effective workflows:

- **Error interpretation:** Paste the query with its error message into your LLM for a quick explanation.
- **Automated fixes:** Ask the AI to correct the error – you might be pleasantly surprised by its capabilities.
- **Query explanation:** For complex queries you don't fully understand, let the AI break it down for you.

- **Query optimization:** Request the AI to improve or simplify a query that's slow, hard to maintain, or difficult to read.
- **Performance analysis:** Share the query's execution plan with the AI. Ask it to interpret the plan and suggest optimizations based on the data flow and bottlenecks it identifies.
- **Index recommendations:** Provide the AI with table schemas and query patterns. It can often suggest appropriate indexes to improve performance.
- **Query rewriting for specific databases:** If you're migrating between database systems (e.g., MySQL to PostgreSQL), ask the AI to rewrite your queries for optimal performance on the target system.
- **Execution plan comparison:** When you have multiple query versions, share their execution plans with the AI. It can compare them and explain the trade-offs between different approaches.
- **Data modeling advice:** Describe your data structure and common queries to the AI. It can suggest schema improvements or normalization techniques to enhance overall database performance.
- **Temporal query analysis:** For queries that perform differently at various times, provide the AI with query logs and performance metrics. It can help identify patterns and suggest time-based optimizations.

Be careful to paste or include any real data to your prompts unless you know that's allowed in your assignment.

Deployment

This chapter will walk you through a practical example of deploying a cloud infrastructure using modern tools and practices. We'll start with a high-level diagram and end with a fully automated deployment pipeline. As with earlier examples, we're using our Claude Project for it, and we don't need to provide it with much context.

Visualizing the infrastructure

The first step in any deployment process is to clearly understand and visualize the infrastructure. Here, we'll use a Mermaid diagram to illustrate our cloud setup. This visual representation serves as a blueprint for our next steps.

Graph out our cloud infrastructure with mermaid.

This diagram provides a clear overview of our infrastructure components and their relationships. It's a valuable tool for communication among team members and stakeholders, ensuring everyone is on the same page before diving into the practical details of implementation.

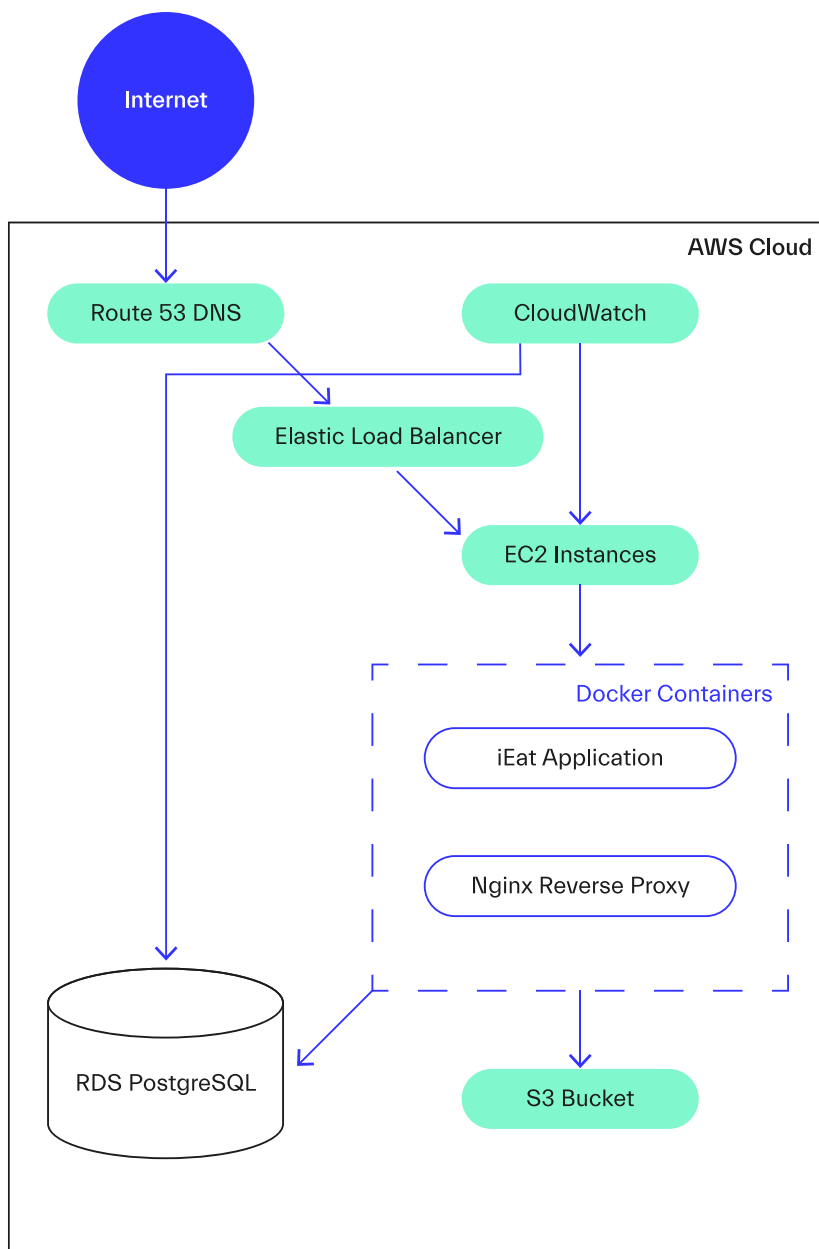


Figure 23. An overview of our infrastructure components and their relationships.

Infrastructure as code with Terraform

With our infrastructure visualized, the next step is to translate this diagram into code. Enter Infrastructure as Code (IaC), a practice that allows us to manage and provision our infrastructure through machine-readable definition files. We'll use Terraform, a popular IaC tool, to define our cloud resources.

Write me terraform scripts to set it up. Keep it clean and split it into multiple files.

Widely used language as Terraform did not pose any challenges for AI, either:



Certainly. I'll create a set of Terraform scripts to set up the infrastructure we've diagrammed. I'll split the configuration into multiple files for better organization and maintainability.

.. multiple code files omitted ...

These Terraform scripts set up the basic infrastructure for the iEat application on AWS. The configuration is split into multiple files and modules for better organization:

- **main.tf**: The main configuration file that sets up the provider and calls the other modules.
- **variables.tf**: Defines the input variables for the configuration.
- **modules/networking**: Sets up the VPC, subnets, and related networking components.
- **modules/ec2**: Configures the EC2 instances, Auto Scaling Group, and Load Balancer.

- `modules/rds`: Sets up the RDS PostgreSQL database.
- `modules/s3`: Creates an S3 bucket for static assets.
- `modules/cloudwatch`: Sets up CloudWatch logs and a basic dashboard.
- `modules/route53`: Configures the Route 53 DNS settings.

While generating IaC code without understanding the target platform is certainly possible, **it might be risky and/or expensive**. So, review everything before committing anything, and have somebody in the loop to review the things for you.

Automating deployment with GitHub Actions

The final piece of our deployment puzzle is automation. We'll use GitHub Actions to create a CI/CD pipeline that automatically deploys our infrastructure whenever changes are pushed to our repository. This setup includes some other features like Slack notifications for errors and production deployments.

Write to me GitHub actions for deployment. Merges on main should deploy it to staging & for prod deployments there should be a manual trigger. As we have a monorepo make sure BE gets deployed if and only if there are changes & likewise on FE.

All deployment errors should trigger a notification our slack channel (endpoint in github secrets), as well as successful prod deployments.

And here's a snippet on what we got:



```
name: iEat Deployment
on:
  push:
    branches:
      - main
  workflow_dispatch:
    inputs:
      environment:
        description: 'Environment to
deploy to'
        required: true
        default: 'production'
        type: choice
        options:
          - staging
          - production
jobs:
  check_changes:
    runs-on: ubuntu-latest
    outputs:
      backend_changed: ${ steps.filter.
outputs.backend }
      frontend_changed: ${ steps.filter.
outputs.frontend }
    steps:
      - uses: actions/checkout@v3
      - uses: dorny/paths-filter@v2
        id: filter
        with:
          filters: |
            backend:
              - 'backend/**'
            frontend:
              - 'frontend/**'
```



```

deploy_backend:
  needs: check_changes
  if: needs.check_changes.outputs.back-
end_changed == 'true'
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v3
    - name: Set up Node.js
      uses: actions/setup-node@v3
      with:
        node-version: '16'
    - name: Install dependencies
      run: cd backend && npm ci
    - name: Run tests
      run: cd backend && npm test
    - name: Deploy to staging
      if: github.event_name == 'push'
      run: |
        # Add your staging deployment
        commands here
        echo "Deploying backend to
        staging"
    - name: Deploy to production
      if: github.event.inputs.environment
        == 'production'
      run: |
        # Add your production deployment
        commands here
        echo "Deploying backend to
        production"
  ...

```

This workflow automates our deployment process, reducing the chance of human error and ensuring consistency across deployments. The added Slack notifications keep the team informed of important

events without requiring constant manual monitoring.

What we've accomplished with just a handful of prompts is working application code, required runtime infrastructure, automated the delivery with CI/CD, and even created the database schema without typing a single line of them ourselves. It's some settings all of this could be very well automated: here's we've worked directly with an LLM as we could've perhaps collaborated with a colleague over Slack.

In the next, we'll explore other aspects such as tooling, testing and documentation.

Scripting and tools

Above, we've demonstrated the capabilities of off-the-shelf AI tools to generate pretty much everything you've used to author yourself.

Furthermore, we present two additional topics that are not often shipped with the product but are utilized either internally by developers or by the ops team: maintenance tools (scripts) and SQL Queries and their optimization.

For instance, those who have worked e.g. with sh or Powershell scripts will find ChatGPT to be a natural fit for the task.

Let's consider we'd like to build a periodical database cleanup script to clean up old records from our database. Building a Node.js typescript software for this would certainly be possible, but we considered that to be an overkill.

Instead, we asked Claude:

Build a script that can be deployed to a container to launch a SQL script that cleans up reservations older than three months. The container should be run daily in AWS as a scheduled task.

What we got as a response was pretty much exactly what we expected with little need for tweaking.



Certainly. I'll create a script that can be deployed in a container to clean up reservations older than three months and provide instructions on how to set it up as a scheduled task in AWS.

```
import os
import psycopg2
from datetime import datetime, timedelta

# Database connection parameters
DB_HOST = os.environ.get('DB_HOST')
DB_NAME = os.environ.get('DB_NAME')
DB_USER = os.environ.get('DB_USER')
DB_PASS = os.environ.get('DB_PASS')

def cleanup_old_reservations():
    # Calculate the date 3 months ago
    three_months_ago = datetime.now() -
timedelta(days=90)

    # SQL query to delete old reservations
    delete_query = """
DELETE FROM reservations
WHERE reservation_date < %s;
"""
```

```

conn = None
try:
    # Connect to the database
    conn = psycopg2.connect(
        host=DB_HOST,
            database=DB_NAME,
            user=DB_USER,
            password=DB_PASS
        )
    cur = conn.cursor()

    # Execute the delete query
    cur.execute(delete_query, (three_
months_ago,))

    # Commit the transaction
    conn.commit()

    # Print the number of deleted
rows
    print(f"Deleted {cur.rowcount}
old reservations.")

    # Close the cursor
    cur.close()
except (Exception, psycopg2.Databa-
seError) as error:
    print(f"Error: {error}")
finally:
    if conn is not None:
        conn.close()

if __name__ == "__main__":
    cleanup_old_reservations()

```

Now, let's create a Docker file to containerize this script:

```
... code omitted ...
```

Create a requirements.txt file with the following content:

```
... code omitted ...
```

Now, let's create an AWS ECS task definition to run this container as a scheduled task:

```
... code omitted ...
```

To set this up as a daily scheduled task in AWS:

Build and push the Docker image to Amazon ECR:

```
docker build -t cleanup-reservations .
docker tag cleanup-reservations:latest
<your-ecr-repository-url>/cleanup-reservations:latest
docker push <your-ecr-repository-url>/
cleanup-reservations:latest
```

Create an ECS cluster (if you don't already have one):

```
aws ecs create-cluster --cluster-name
ieat-cleanup-cluster
```

Register the task definition:

```
aws ecs register-task-definition --cli-
input-json file://cleanup-task-defini-
tion.json
```

Create a CloudWatch Events rule to schedule the task:

```
aws events put-rule --name daily-clean-
up-rule --schedule-expression "cron(0 1 *
* ? *)"
```

Add permissions to the CloudWatch Events rule to run the ECS task:

```
aws events put-targets --rule
```

```
daily-cleanup-rule --targets "Id"="1", "Arn"="arn:aws:ecs:<your-region>:<your-account-id>:cluster/ieat-cleanup-cluster", "RoleArn"="<your-cloud-watch-events-role-arn>", "EcsParameters"="{ "TaskDefinitionArn"="arn:aws:ecs:<your-region>:<your-account-id>:task-definition/cleanup-reservations:1", "TaskCount"=1, "LaunchType"="FARGATE", "NetworkConfiguration"="{ "awsvpcConfiguration"="{ "Subnets"=[ "<your-subnet-id>" ], "SecurityGroups"=[ "<your-security-group-id>" ], "AssignPublicIp"="ENABLED" } } }"
```

Make sure to replace all placeholder values (like `<your-ecr-repository-url>`, `<your-db-host>`, etc.) with your actual AWS resource values.

This setup will run the cleanup script daily at 1:00 AM UTC. You can adjust the cron expression in step 4 to change the schedule as needed.

Maintenance and support

Potential use cases for AI tools to help you in maintenance and operations of a software are numerous and have been used behind the scenes for instance in security scene for quite some time.

In this guide we only scratch the surface, but from a developer perspective, we highlight some of the most obvious in the following scenarios.

Scenario 1: The reservation black hole

Problem: Customers report that their reservations are disappearing into thin air. The system shows no record of bookings made just minutes ago.

- **Log Analysis:** Feed the relevant logs from the reservation system. Ask it to find patterns in failed reservations and highlight any common factors.

Analyze these logs from our reservation system. Look for patterns in failed reservations and highlight any common factors.

- **Database Query Assistance:** If you suspect a database issue, AI can help craft and refine SQL queries to investigate the problem.

I need to query our reservations table to find discrepancies. Can you help me write a SQL query that compares successful reservations against our audit log?

- **Code Review:** Ask AI to review the reservation handling code, looking for potential race conditions or data consistency issues.

Review this code snippet from our reservation handler. Look for potential race conditions or data consistency issues that could cause reservations to disappear.

Scenario 2: The great menu meltdown

Problem: Restaurant menus are displaying incorrect prices, missing items, or even dishes from other restaurants.

- **Data integrity check:** Ask AI to analyze a sample of the menu data to spot inconsistencies or corruption.

I'm providing a sample of our menu data. Can you analyze it for inconsistencies, particularly focusing on price discrepancies and mismatched restaurant-dish pairings?

- **API troubleshooting:** If the issue is related to the API serving menu data, AI can aid in debugging the API calls and responses.

Here are the API calls and responses for our menu service. Can you identify any anomalies or potential points of failure?

- **Caching logic review:** Ask AI to review the caching logic for menu data, looking for potential issues that could lead to stale or incorrect data being served.

Examine this caching logic for our menu service. Are there any flaws that could result in serving stale or incorrect menu data?

Scenario 3: The Payment Processing Panic

Problem: Payments are failing intermittently, with some users being charged multiple times while others can't complete transactions at all.

- **Error log interpretation:** Feed the error logs from the payment processing system and ask for a summary and potential root causes.

Analyze these error logs from our payment processing system. Summarize the types of errors we're seeing and suggest potential root causes.

- **Transaction flow analysis:** Ask AI to review the transaction processing flow, looking for points of failure or inconsistencies.

Here's a diagram of our payment processing flow. Can you identify any potential points of failure or inconsistencies that could lead to our current issues?

- **Code diff analysis:** If the issue started after a recent deployment, ask AI to review the code diff and identify potentially problematic changes.

Here's the code diff from our last deployment. Can you identify any changes that might be causing our payment processing issues?

- **Test case generation:** Use AI to generate comprehensive test cases for the payment system to prevent future issues.

Based on our current payment processing issues, can you generate a set of test cases to help us prevent similar problems in the future?

By using AI tools in these crisis scenarios, developers can quickly gather insights, identify potential causes, and develop solutions. While AI won't solve the problems automatically, it can significantly speed up the troubleshooting process and provide valuable perspectives that might be overlooked in the heat of the moment.

Remember, the key to effective AI-assisted crisis management is to provide as much relevant context as possible. This includes logs, code snippets, system architecture diagrams, and detailed descriptions of the problem. The more information you provide, the more targeted and useful AI aid can be.



Documentation

Documentation is vital in software development, offering key insights into a project's structure, functions, and use. While context is important, AI can significantly improve documentation even with minimal input.

A practical approach to documentation involves jotting down first thoughts or bullet points, regardless of quality, then using AI to transform this rough draft into polished technical writing. This method is effective because it lowers the barrier to starting documentation, turns stream-of-consciousness notes into coherent technical content, and proves useful even without extensive project context.

When more context is available, such as project specs and tech stack details, AI can produce even more exact documentation. This is particularly helpful for projects using standard solutions, where AI can use common practices to fill in gaps.

For smaller documentation tasks, providing the full codebase to an AI writing assistant can essentially cover all documentation needs within that scope. However, when dealing with larger projects where the entire codebase exceeds the AI's context window, using class and method signatures instead of full code can effectively mitigate this limitation while still providing sufficient context for comprehensive documentation generation.

Architectural decision records (ADRs)

Architectural Decision Records (ADRs) serve as a valuable tool for documenting significant project decisions. While the process of creating ADRs can be time-consuming, AI can streamline this task considerably. By inputting meeting logs or relevant discussions from platforms like Slack, AI can generate well-structured ADR entries.

This approach not only reduces the effort needed to maintain ADRs but also eases the creation of a more comprehensive knowledge base. For instance, a custom Slack bot could be developed to append troubleshooting solutions directly to a wiki using simple slash commands, thereby continuously enriching the project's documentation.

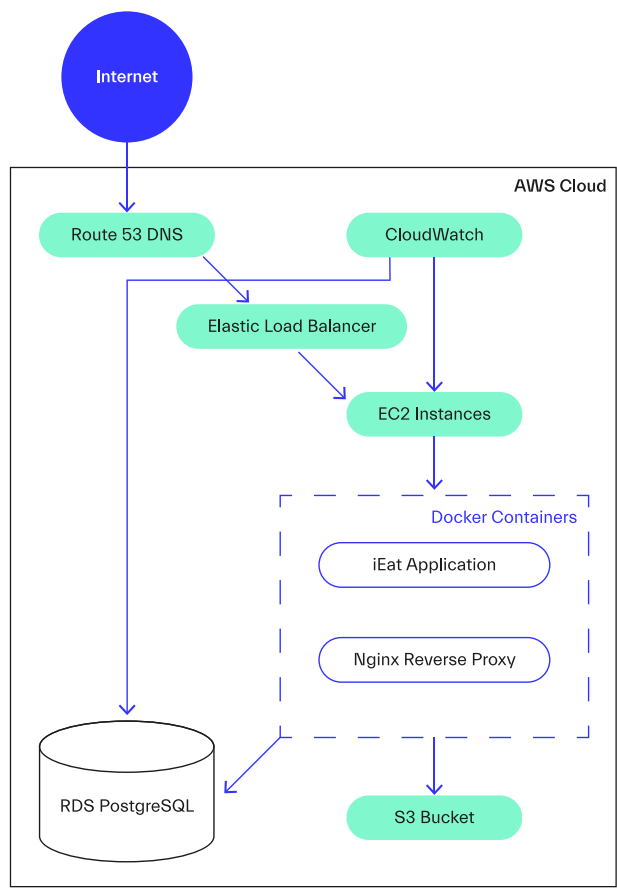
Diagrams and visual documentation

Creating visual representations of project architecture and control flow are often needed to understand, discuss and analyze the dynamics and dependencies among the different parts of your solution. Maintaining and authoring them to reflect the changes, especially in code-first -style projects is often tedious.

Luckily the AI is there to help as with them. For example, we can provide project descriptions or configuration files (such as Terraform scripts) to AI, which will generate e.g., Mermaid diagrams for us. Other popular formats, such as draw.io are typically supported, too.

In case you already had for example a hand-drawn diagram captured from the meeting room whiteboard, you may pass that as a starting point to AI and as for refinements and observations. For example:

Create a written description of our cloud infra based on this image.



NOTE: Our custom Claude Project was not used for this to demonstrate its visual capabilities. Results would be much better with the added context.



AWS Cloud Infrastructure Overview

Our application is hosted on Amazon Web Services (AWS) with the following architecture:

Domain Name System (DNS):

Route 53 handles DNS management, directing incoming traffic from the internet to our application.

Load Balancing:

An Elastic Load Balancer distributes incoming traffic across multiple EC2 instances for improved availability and fault tolerance.

Compute:

EC2 Instances host our application components.

Docker Containers run on these EC2 instances, encapsulating:

iEat Application: Our main application service.

Nginx Reverse Proxy: Handles incoming requests and forwards them to the appropriate services.

Monitoring and Metrics:

CloudWatch provides monitoring and observability for our infrastructure and application.

Database:

RDS PostgreSQL serves as our relational database management system.

Storage:

S3 Bucket is used for object storage, likely for static assets or data backups.

Networking:

The entire infrastructure is contained within the AWS Cloud environment. Internet traffic enters through Route 53 and is directed to the Elastic Load Balancer.

This architecture provides a scalable, resilient, and manageable environment for our iEat application. The use of Docker containers allows for consistent deployments and easier management of our application components.

Additional documentation use cases

In addition to the example above, do consider the following for further applications of AI in documentation:

- **Style guides:** By inputting linter rules and code examples, AI can generate clear and comprehensive style guides in markdown format.
- **Release notes:** AI can produce release notes by analyzing commit messages and code diffs for an upcoming release.
- **Dependency vulnerability reports:** AI can create informative reports by processing package manager files (e.g., package, json) and dependency scanner logs.
- **Troubleshooting guides:** AI can help in compiling troubleshooting guides based on resolved issues discussed in communication channels.

So, in addition to standard cloud & database diagrams, you may also create user guides, coding manuals and release notes again saving time and effort.

TIP: Use the coding guidelines and other conventions as input or context for your coding LLM to preserve the same look & feel across the codebase.

AI as a documentation interface

An often-overlooked advantage of AI in documentation is its ability to not only generate but also interpret and relay information. By ingesting project documentation, AI can serve as an interactive knowledge base, allowing team members to query specific information without the need to manually search through extensive documentation.

This capability is particularly beneficial for onboarding new developers. Rather than requiring new team members to read vast amounts of documentation, they can interact with an AI that has processed this information. This approach allows for more targeted and efficient knowledge transfer, enabling new developers to quickly access the specific information they need to become productive team members.

Testing and quality assurance

Unit tests

Test automation is a crucial building block to ensure the quality and robustness of your software. It is a key aspect to keep software maintainable, and often overlooked by the developers.

This is an area where AI shines. Making good test cases, also negative ones with decent coverage is slow and error-prone itself, and sometimes quite boring and repetitive.

You may use AI to but not limited to:

- Generate unit tests for your classes / modules / methods
- Generate mock interfaces for API's or databases
- Generate synthetic test data

For example, consider our method in iEat backend to book a table, which already had quite a bit of logic inside, including logic to prevent double booking by the same party or table, Boolean returns and inevitable null checks.

Let's see what IDE assistants can do for us by invoking:

```
/tests
```

Right off the bat, we got decent starting point.



```
...
describe('TableReservationService', () =>
{
  let service: TableReservationService;
  let reservation: Reservation;

  beforeEach(() => {
    service = new TableReservation-
Service();
    reservation = {
      id: '1',
      customerName: 'John Doe',
      tableId: 1,
      date: new Date('2023-10-10'),
      numberOfGuests: 4,
    };
    service.addReservation(reservation);
  });

  test('should check if a table is
available', () => {
    const date = new Date('2023-10-10');
    expect(service.isTableAvailable(date,
2)).toBe(true);
    expect(service.isTableAvailable(date,
1)).toBe(false);
  });
});
...
```

Good unit tests that capture the nuances are not only tedious to write but even more so when working with existing codebase with low to zero earlier test coverage. Using generative AI to inject tests afterwards is a good example of having the computer do the heavy lifting – in this case, injecting and inventing test cases.

Beyond unit testing

Testing is often a gradual process which begins at unit level (one or handful of classes/modules) and ends on the system level, where various technical aspects are tested against a fully functional, up & running system.

Above, we demonstrated a simple unit test being generated for a method. But what if you need to go beyond that? Think like testing your REST API when it's deployed to fulfill a scenario, like for iEat: seek available tables for a date, make and cancel a reservation?

Let's try this thing out. Let's imagine we have an OpenAPI spec available for iEat backend.

Paste the YAML that to your LLM and prompt

Create a node.js test cse that calls the API locally to seek reservations, inserting and cancelling a booking.

And we get a reservation_test.ts file that does exactly that:



```
describe('Table Reservation API', () => {
  let reservationId: string;

  it('should create a new reservation',
(done) => {
    const reservation = {
      id: '1',
      customerName: 'Jane Doe',
      tableId: 2,
      date: new Date().toISOString(),
      numberOfGuests: 3,
    };
  });
});
```

```

    chai.request(app)
      .post('/reservations')
      .send(reservation)
      .end((err, res) => {
        expect(res).to.have.status(201);
        expect(res.body).to.be.an('object');
        expect(res.body).to.have.property('success', true);
        expect(res.body).to.have.property('message', 'Reservation created successfully.');
```

reservationId = reservation.id;

```

        done();
      });
    });
  });
});

```

Taking testing to the next level

While we were satisfied with the above for the first steps in our testing journey, there are more possibilities just withing our reach. We will discuss them briefly in the next subchapters.

Creating and populating test databases

Generate test data and/or mocks to widen the scope of your testing.

For example, for iEat, we'd like to try out an SQLite database created on the fly from schema with test data:

Create a test SQLite database that is dynamically populated with feasible test data using this schema.
(upload your schema)

And literally in seconds, we got a python script to inject data using the schema we provided into an SQLite file we could read when we launch the tests.



```
...
# Sample data for restaurants
restaurants_data = [
    ("La Pizzeria", "123 Main St", 50),
    ("Sushi Place", "456 Elm St", 30),
    ("Burger Joint", "789 Oak St", 40)
]

# Insert restaurants
cursor.executemany('''
INSERT INTO restaurants (name, location,
capacity)
VALUES (?, ?, ?)
''', restaurants_data)
```

Mocking

Testing gets harder when the scope of the test gets wider. More modules, databases, pages or whatever you include will make automated test building harder.

A popular way to reasonably test modules without needing to run the entire application is mocking, and yet again unsurprisingly, this is something a LLM is well suited for you.

Consider the earlier example where we built a test database from scratch and could use that. It was a good approach to have real-life looking data without needing to mock anything, but what if we had, let's say a POS (Point of Sales, not the other meaning of this abbreviation) system that has an API which you need to call upon receiving a reservation.

And as is often the case, there is no test system for it.

Let's build a mock API for it with this lengthy prompt:

NOTE: Making prompts as short as possible usually gives better results.

Build a mock API implementation for our point of sales that I can call when I need to test my reservation system. The mock should not require launching additional processes for our jest tests but could be injected inside. We are using Axios as the REST API calling library. The first use of this is a POST method on URL /pos/reservations with json { "table": "tableid", "datetime": "date and time" }

Omitted the actual details for clarity, but the implementation was exactly what we wanted:



Explanation

- **Mocking Axios:** By placing a mock of Axios in the mocks directory, Jest automatically uses it when you require Axios in your tests. This mock simulates API behavior without making actual HTTP requests.
- **Mock API Logic:** The mocked POST method in axios.js simulates the /pos/reservations endpoint, checking if the request data is valid and returning appropriate responses.
- **Jest Test Suite:** The posReservation.test.js file contains test cases to validate the behavior of the makeReservation

function using the mocked API. It checks both successful and error scenarios.

- **No Additional Processes:** Since this mock operates purely within the Jest testing environment, there's no need for separate processes or servers, making it lightweight and easy to integrate.

Testing with Playwright and similar tools

Playwright is a powerful tool used primarily for automated web testing and its primary uses are end-to-end testing, cross-browser testing, automated interaction with web pages, performance testing, visual regression testing and API testing. All of these use cases are suitable for AI powering.

Automating old projects with Playwright

For projects that lack comprehensive browser tests, AI tools can be effectively utilized to automate test creation. Using Playwright, you can generate both positive and negative test cases for a sample page, such as the GlobalSQA sample page. While AI-generated tests can provide a solid starting point, they often require manual adjustments to fix minor issues, like discrepancies in expected page titles or form validation errors.

Example:

Could you create playwright test cases of this page <https://www.globalsqa.com/samplepagetest/>. Test cases should both contain positive and negative cases

ChatGPT will then yield a response like:



// Positive Test Case: Check page title

```
test('should have correct page title',
  async ({ page }) => {
    await page.goto('https://www.globalsqa.
com/samplepagetest/');
    await expect(page).toHaveTitle('Sample
Page Test - GlobalSQA');
  });
// Positive Test Case: Fill out and
submit the contact form
test('should submit contact form
successfully', async ({ page }) => {
  await page.goto('https://www.globalsqa.
com/samplepagetest/');
  await page.fill('in-
put[name="g2599-name"]', 'John Doe');
  await page.fill('in-
put[name="g2599-email"]', 'john.doe
@example.com');
```

This approach will give a good starting point for creating further tests.

Recording tests manually

Manual test recording in Playwright allows you to create reliable and accurate test cases by capturing real user interactions. This approach ensures that your tests reflect actual usage scenarios and can be used as a baseline for generating further negative or edge test cases.

Example would be pasting the pre-recorded test case to AI. Let's give it a try;


```
Could you give me negative test cases from this.
test('test', async ({ page }) => {
  await page.goto('https://www.globalsqa.com/
samplepagetest/');
  await page.getByLabel('Consent', { exact: true }).click();
  await page.getByLabel('Name(required)').click();
  await page.getByLabel('Name(required)').fill('testi testinen');
  ...
}
```

As a result, we got pretty good negative test cases:



Here are some possible negative test cases based on your provided script:

Test with Missing Required Fields

Description: Attempt to submit the form without filling in any of the required fields.

Expected Behavior: The form should not be submitted, and error messages should be displayed for the required fields.

```
test('test with missing required fields',
  async ({ page }) => {
    await page.goto('https://www.globalsqa.
com/samplepagetest/');
    await page.getByLabel('Consent', { exact:
true }).click();
    await page.getByRole('button', { name:
'Submit' }).click();
    const errorMessages = await page.locator('.error-message'); // Assuming error
messages have a class 'error-message'
    expect(await errorMessages.count()).to-
BeGreaterThan(0);
    .....
  }
}
```

All it takes for you is to copy and paste responses to the Playwright test suite.

Testing based on specifications

When developing test cases from specifications, you translate functional requirements into specific tests. For example, testing a restaurant booking application would involve creating test cases for viewing available restaurants, making reservations, handling cancellations, and preventing duplicate bookings.

While this method ensures broad coverage of critical functionality, it will require fine-tuning to align with the actual application behavior.

Using Playwright tests with AI

Integrating AI into Playwright test development can streamline the creation and maintenance of test cases. AI tools can assist in generating test scripts based on natural language descriptions, identifying edge cases, and even optimizing test execution. For instance:

Natural Language Processing (NLP): You can describe the desired tests in plain English, and AI tools will generate the corresponding Playwright scripts.

Automated Test Generation: AI can analyze your application's user interface and automatically generate a comprehensive suite of tests, including both typical user flows and potential edge cases.

Test Optimization: AI can help optimize the order and frequency of tests based on historical data, reducing test execution time and improving efficiency.

When using tools like Auto-Playwright or Zerostep the test cases will be like:

```
test("User registration", async
({ page }) => {
  await auto("go to registration page",
    { page, test });
  await auto("fill in name, email, and
    password", { page, test });
  await auto("submit the registration
    form", { page, test });
  const registrationSuccess = await
    auto("is there a 'Registration success-
    ful' message?", { page, test });
  expect(registrationSuccess).toBe(true);
});
```

As demonstrated above, there is no need for you to pre-record and try to figure out which name the selectors in the page are. AI can do the heavy lifting for you.

Expect the unexpected

Generating tests that should fail or what system should be able to tolerate is not easy. But there are tools out there that can help you with this. You've probably heard about errors in public systems where data you were not supposed to see was presented to you, like somebody else's bank or health records.

In the context of iEat a similar example would be to verify that reservations cannot be cancelled or viewed at all after they have been made, and that e.g. you should not be able to see the name of the customer in public view.

Using LLMs to help you discover this kind of non-business as usual scenarios often not discovered when doing the "happy day" scenarios is indeed advisable.

Learning and skill development

Despite having access to powerful AI tools that can augment your capabilities as a developer, understanding code and technology in general is still crucial – perhaps even more so than before.

Your role as a developer may shift from knowing the intricate details of specific frameworks to a more high-level perspective. Despite this you will remain responsible, which makes thorough review before implementation a necessary practice. Remember, the output of the LLMs is not necessarily correct or safe, even if it compiles correctly.

The AI-driven development approach introduces new dynamics to personal learning and skill development. While there may be an increased volume of knowledge to get, AI also offers the advantage of providing a tireless personal coach to assist with questions and troubleshoot failing demonstrations.

We posit that certain certificate-oriented trainings focused on rote memorization of details may become obsolete. In their place, continuous, on-demand learning facilitated by Large Language Model (LLM) coaches are expected to become the predominant method of skill acquisition and development.

AI as a learning accelerator

AI proves exceptionally helpful in learning about unfamiliar technologies and concepts. This manifests in two primary ways:

- **Ad hoc usage:** Developers require less foundational understanding to work effectively with modern technologies, as AI can offer context-specific guidance.
- **Conceptual comprehension:** AI facilitates faster attainment of key insights by tailoring explanations to an individual's background and learning style. For instance, an AI can elucidate the principles of a state management library like Zustand by drawing parallels to a developer's existing knowledge of Redux, thereby accelerating the learning process.

I've worked previously with (modern) redux extensively, but not Zustand. Could you give me the rundown on how it works. No need for code examples & keep it brief. Also, how do I do all the cool redux stuff I've gotten used to, like time travel.

And the comparison between these two technologies was right on the mark:



Zustand is a lightweight state management library for React that aims to simplify global state management compared to Redux. Here's a brief rundown:

- **Store creation:** Create a store using a hook-like syntax.
- **State and actions:** Define both state and actions in the same place.
- **Usage:** Use the store directly in components without wrappers or providers.

- **Selective updates:** Components only re-render when their subscribed state changes.
- **Middleware support:** Allows adding middleware for logging, persistence, etc.

For Redux-like features:

- **Time travel:** Zustand doesn't have built-in time travel, but you can implement it using middleware to track state changes.
- **DevTools:** Zustand supports Redux DevTools out of the box for debugging.
- **Middleware:** You can add custom middleware for logging, async actions, etc.
- **Slices:** While not a native concept, you can organize state into slices using separate stores or nested objects.
- **Selectors:** Use built-in selectors for efficient state derivation.

Overall, Zustand provides a simpler API while still allowing for powerful state management patterns, though some advanced Redux features may require custom implementations.

Let's dig in a bit deeper:

Give me some code examples on how it's supposed to simplify things vs redux toolkit

And we got a more detailed explanation.



Certainly, I'd be happy to provide some code examples to illustrate how Zustand simplifies state management compared to Redux Toolkit. Let's create a simple counter application in both libraries to showcase the differences.

// Zustand Example

```
import create from 'zustand'

// Create store
const useStore = create((set) => ({
  count: 0,
  increment: () => set((state) => ({
    count: state.count + 1 })),
  decrement: () => set((state) => ({
    count: state.count - 1 })),
}))

// Use in component
function Counter() {
  const { count, increment, decrement } =
    useStore()
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={increment}>Increment</button>
      <button onClick={decrement}>Decrement</button>
    </div>
  )
}
```

// Redux Toolkit Example

```

import { createSlice, configureStore }
from '@reduxjs/toolkit'
import { useSelector, useDispatch, Provider } from 'react-redux'

// Create slice
const counterSlice = createSlice({
  name: 'counter',
  initialState: { count: 0 },
  reducers: {
    increment: (state) => { state.count
+= 1 },
    decrement: (state) => { state.count
-= 1 },
  },
})

// Create store
const store = configureStore({
  reducer: { counter: counterSlice.reducer },
})

// Use in component
function Counter() {
  const count = useSelector((state) =>
state.counter.count)
  const dispatch = useDispatch()
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => dispatch(counterSlice.actions.increment())}>Increment</button>
      <button onClick={() => dispatch(

```



```
patch(counterSlice.actions.decrement()))>Decrement</button>
    </div>
  )
}
```

...

Here's a breakdown of how Zustand simplifies things compared to Redux Toolkit:

Setup: Zustand requires less boilerplate. You create a store with a single function call, while Redux Toolkit needs slice creation, store configuration, and a provider wrapper.

State and actions: In Zustand, you define state and actions in one place. Redux Toolkit separates them into state and reducers.

Usage in components: Zustand allows direct use of the store in components. Redux Toolkit requires useSelector and useDispatch hooks.

No need for action creators: Zustand actions are just functions. Redux Toolkit still uses action creators, although they're generated automatically.

No provider needed: Zustand doesn't require wrapping your app in a provider component.

Simpler updates: Zustand allows direct state mutations in actions. Redux Toolkit uses Immer under the hood to allow this in reducers, but the mental model is still based on immutability.

Overall, Zustand provides a more straightforward, less ceremonial approach to state management, which can be

especially beneficial for smaller to medium-sized applications or when you want to avoid the complexity of Redux.

Fostering generalist capabilities

AI empowers developers to rapidly get information across diverse domains, even those outside their primary expertise. This capability enables professionals to gain high-level understanding of complex topics quickly.

For example, let us imagine that our iEat has grown big and is planning to expand into food delivery. Armed with loads of VC money, we are planning to start operations in Singapore, AI can provide a concise overview of relevant liability laws concerning allergies and compare the legal framework to those of the European Union and United States.

Our app is launching in Singapore. walk me through the local liability laws that are relevant to our app. I am completely unfamiliar with their legal system and not a lawyer.

For example, if we screw up with allergen information, how would the liability stuff go in general.

While legal advisory will require humans in the loop, the response had both references to local law, and even speculated with the implications if we served seafood to somebody that told us to being allergic to them.



Based on Singapore's legal framework, here are key points relevant to iEat's liability, particularly regarding allergen information:

Consumer Protection (Fair Trading) Act:

Protects consumers against unfair practices.

iEat could be liable if it misrepresents allergen information.

...

If iEat provides incorrect allergen information:

- Primary liability falls on the restaurant
- iEat could be secondarily liable if negligent in transmitting information
- Damages could include medical expenses, lost wages, and pain and suffering

To mitigate risks:

- Implement robust allergen information verification processes
- Clearly state in terms that restaurants are responsible for allergen accuracy
- Maintain adequate liability insurance

Consult a Singaporean lawyer for specific legal advice and strategy.

The first recommendation was a bit off. We are not actually the party who delivers the food, but just an intermediary. So let us just stop here: this is clearly a domain where we can go wrong and should not trust the AI.

So, while AI can provide insights into domains unfamiliar to you, it may and often will produce confident but incorrect information. Do not rely solely on AI-generated information for critical decisions in areas beyond your ability! Verification from authoritative sources and expert consultation remain essential practices, especially when dealing with consequential matters such as legal compliance or safety regulations.

And as a general note, avoid any legal issues to the best of your ability.

Brownfield considerations

This guide and use cases focus on building entirely new things. The traditional wisdom tells us, however, that roughly two-thirds of software's TCO come from the maintenance phase. The good news for AI is that having an existing codebase and infrastructure improves the changes of AI to provide you with helpful answers.

Most of the AI use cases listed in this section can very well be applied in the brownfield settings, such as finding errors in the code. Depending on the context, you might find these approaches as easy starts for an existing project:

Gaining a faster understanding on the codebase

Use the AI as your mentor to guide you through the codebase or explain difficult parts of it. Think prompts like:

How are the Customers and Orders related in the codebase?

or

Please explain the generic structure of the frontend application under the UI for me.

Locating root causes for errors

Finding errors in contexts and projects you are unfamiliar with can be daunting at times, but trying out AI to figure out what went wrong might save you a lot of time.

An obvious sample would be to copy and paste the error message into AI that has the relevant codebase in the context and see what it thinks about it. Only after failing that, try something else.

Refactoring

Improving or changing existing code, especially something you have not written yourself and may not be very maintenance friendly is something AI can help you with.

Consider for example a codebase with deep call hierarchies:

- Data is read from database, like PERSON.NAME.
- It is converted into a value object. `Person { string name { get; set; } ...}`
- Results are passed on the API service. `{ name: "User Name", ... }`
- Frontend converts it into it's internal state/value object. `{ currentUser_name: "User Name", ... }`

Let's imagine we'd like to add a "User ID" number to all of these points? That's right, in a bad case that's three points to modify.

Now with AI, make sure you have the codebase ready, and prompt

Add a property "UserID" for everything that reads and uses the Person data.

You are likely to get a decent first guess to build.

Modernization

Updating or changing dependencies, such as frontend libraries can be a major effort. Finding all the places that you need to change takes time and is usually involves updating the same thing, such as adding a parameter to a function, in several places.

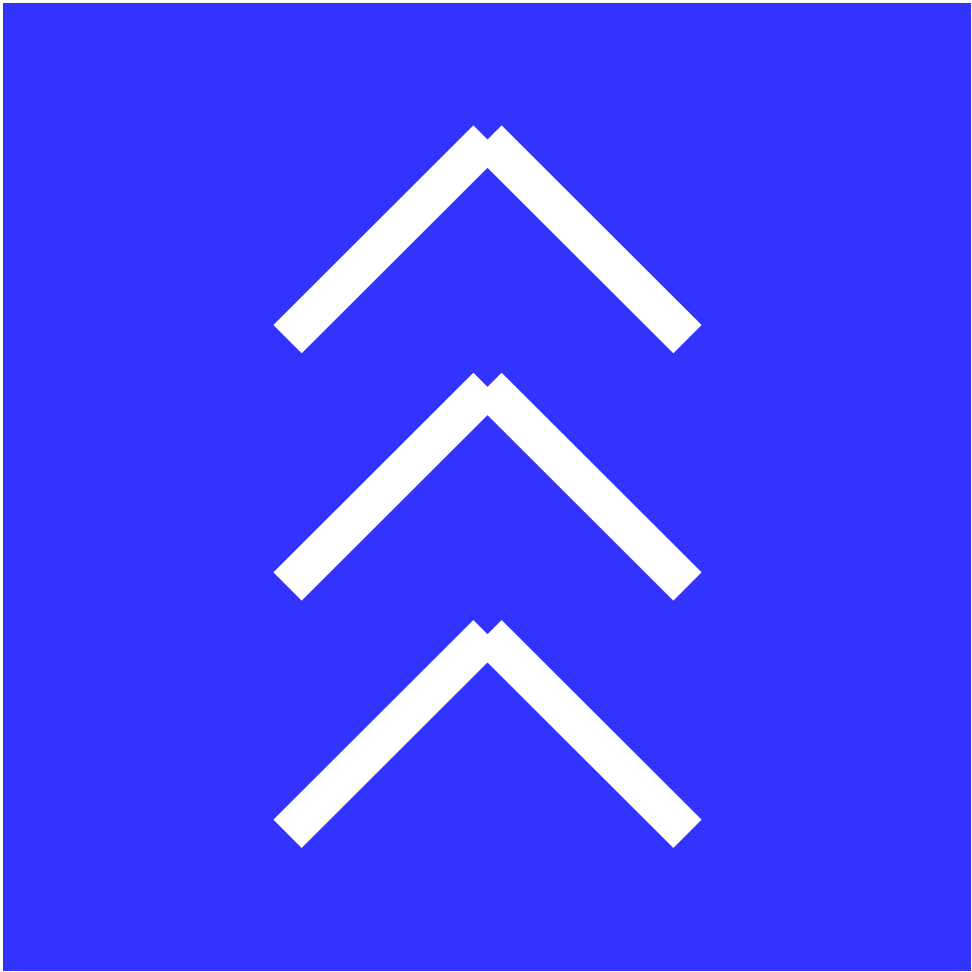
This is again something you could ask the AI, like GitHub Copilot for help:

Our API library has been updated and does not provide non-async methods anymore. Find and update calls to async in the @codebase.



PART 3 •

Road ahead





Future trends and how to prepare

The final part of this book is dedicated to looking at the road ahead. We approach this from three angles:

1. Decision making in the AI era

How does the new era of AI-powered development affect the decision-making process? Will the increased productivity lower the threshold to develop software?

2. Trends in AI-powered software engineering

What kind of trends we anticipate regarding AI-assisted software engineering tools? What kind of possibilities agents – AI-powered development assistants – and chaining them offer?

3. How to become AI-powered

How will all this affect the ways we create software and what kind of steps organizations should take?

Finally, we conclude the book with discussion on the possible outcomes.

Decision-making in the AI era

In recent years, low-code and no-code platforms have gained traction as fast, cost-efficient ways to build applications and services. These platforms allowed businesses to deploy feature-rich systems without requiring extensive technical knowledge or custom programming.

Traditionally, organizations faced a choice between buying or building software. However, low code and no code introduced a new alternative: configuring it. This meant that if a packaged CRM or commercial platform like Shopify did not meet your needs, but you had reasons to hesitate on a custom software project, low-code or no-code solutions became an attractive option.

The rapidly advancing capabilities of AI offer yet another powerful choice: **generate it**.

The first benefit of the new Generate it approach saves us from inevitable vendor lock-in associated with Configure it strategy: the app you built onto Outsystems or Microsoft automation tools is never going to leave those ecosystems. Other benefits of the approach include faster time to market, full control of the code, outlook, and life span and finally, less risk due to reduced effort.

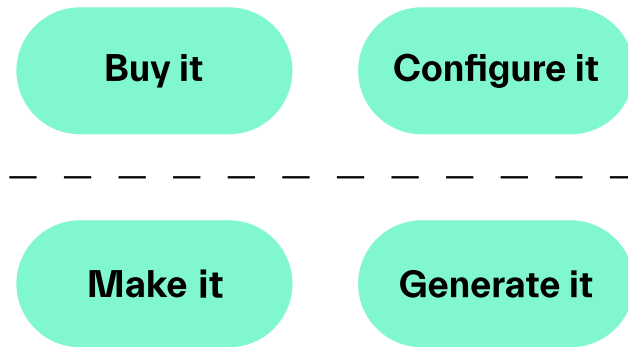


Figure 24. The four key approaches to software solutions in the AI era: Buy, Configure, Make, or Generate.

Choosing the right approach

Let us explore how AI impacts the following key decision-making criteria:

- **Customizability:** The ability to brand, differentiate, and build all the features you need. AI-powered tools offer unprecedented levels of customization by generating code, content, and models tailored to specific business needs.
- **Time to market:** AI can significantly reduce the lead time from idea to rollout by automating tasks that traditionally needed manual intervention, such as coding, testing, and deployment.
- **Cost predictability:** AI-driven solutions can provide better cost predictability by automating estimation processes, monitoring costs in real-time, and optimizing resource allocation. This is distinct from simply being cost-effective; it is about knowing what to expect upfront.

- **Vendor dependence:** AI tools can help reduce vendor dependence by enabling businesses to generate and keep their solutions in-house, avoiding lock-in with specific vendors or platforms.
- **Shared responsibility:** AI allows businesses to use shared datasets, models, and frameworks, benefiting from collective learning and improvements without compromising on customization.
- **IT skill requirements:** AI is changing the skill sets required to develop and maintain solutions. While traditional IT skills like coding and infrastructure management are still important, AI tools are lowering the barriers, allowing non-technical users to take part in the creation and deployment of solutions.

The list above is by no means complete but captures the key aspects for decision making on the strategy to, for example, deliver the new app the marketing department has been asking for ad nauseum.

The decision-making spectrum

So how to evaluate the promises of AI-powered development paradigm, generate it when the guy from the marketing is breathing down your neck?

Let us try to apply the criteria we presented above to compare the traditional buy it, make it, or configure it to the newly introduced AI-powered generate it approach. We have used classification of high-medium-low as a typical average.

NOTE: This classification is generic and you need to apply it in your context carefully. For example, you might conclude that standard office tools, such as Forms or Power Automate or Forms are good enough for the Marketing Department – without any need for you to build anything.

Paradigm	Customizability	Time to Market	Cost predictability	Vendor Dependency	Shared Responsibility	IT Skill Requirements
Buy it	Low	Fast*	High	High	High**	Low
Make it	High	Slow	Low	Low	Low	High
Configure it	Medium	Medium	Medium	High	High**	Low
Generate it	High	Medium	Medium	Low	Low	Medium

Figure 25. (*) Anyone involved in an ERP project might disagree. (**) AKA “Buy IBM”. Large customer base does not always translate into cheaper prices, easy updates, and constant flow of new features – sometimes quite the opposite.



The introduction of AI to translate requirements into working solutions is yet another level of automation where technology is pushed further into the background. This allows the AI-powered experts and their customers to focus much more on the content than the technical details.



Trends in AI-powered software engineering

In this chapter, we take a brief look at what we expected to emerge in near and medium term in the software engineering field.

We begin the evaluation with expectations for LLM evolution, and as an example, study the recent release from OpenAI. Further Next, we discuss the specialized **agents** how these **agents would interact together** to cover most of the current development cycle?

Evolution of LLMs

The LLMs of today will pale in comparison to what we are going to see in the coming years. Larger context sizes, better and perhaps more focused training, enhanced reasoning capabilities, and even specialized hardware and optimization/partitioning techniques are expected to increase the capabilities significantly.

While this all sounds wonderful, there are still caveats and we don't expect significant breakthroughs, but rather incremental process on the medium- and long term.

There are certain hurdles to cross which we will discuss briefly in the next.

For instance, model performance increases with scale, and in theory,

a model trained on more data would outperform even without further improvements. However, **we're approaching limits in model scaling**. We're already consuming most available high-quality data, and opportunities to increase capacity tenfold become scarce. Furthermore, laws of statistics and other limitations well known by the more traditional machine learning community still apply.

Cost will also remain a factor. Running the advanced models is already expensive, and **more capable models might turn out to be prohibitively costly for day-to-day** use. One solution is to choose a model that fits the purpose and delegate only the most difficult requests to the newer, more expensive models. This might lead to emergence of smaller, focused or purpose-built models instead of large, multi-purpose models.

While approaches like OpenAI's o1 can help with both data quality and quantity in training future models, we'll still need human-generated data for knowledge.

All in all, we might face **diminishing returns as models improve**. This doesn't mean they're not becoming more intelligent, but that increased intelligence may not significantly boost their utility, as model intelligence is no longer the main bottleneck in many applications.

Advanced reasoning with OpenAI o1

As an example of recent development, we explore the OpenAI's new o1 model. It represents a significant leap in AI capabilities, particularly in complex reasoning tasks.

Understanding OpenAI's o1 model requires grasping the concept of slow versus fast thinking in AI. Traditional large language models (LLMs) excel at fast thinking - rapid, intuitive responses based on pattern recognition. The o1 model, however, introduces a slow thinking capability, enabling more deliberate, analytical approaches to complex problems.

This slow thinking manifests as a hidden chain-of-thought (CoT) process, where the model thinks before answering. While the specifics

remain undisclosed, o1 appears to use a model fine-tuned for CoT, allowing it to break down complex tasks into steps and reason through them methodically.

This approach, proven effective with standard LLM prompting, is still not the end of the road. Research suggests future iterations might evolve towards a “graph-of-thought” concept - essentially a dynamic mind map the AI constructs and refines. This would enable even more nuanced problem-solving strategies and cognitive capabilities.

One of o1’s key innovations lies in its scalability. Its performance improves not just with more training data, but also with increased inference time. In other words, o1 gets better results both by learning how to think more effectively (during training) and by spending more time thinking (during use).

The current version of o1 is heavily geared towards math and science. However, the cognitive processes in coding, while similar, have distinct characteristics. An o1-like model specifically tailored for coding tasks could potentially yield even better results in software development contexts.

The o1’s “secret sauce” lies in its use of reinforcement learning (RL). By increasing the language model’s temperature to generate diverse trains of thought and reinforcing those that lead to correct solutions in complex reasoning problems, o1 goes beyond simple CoT prompting. Applying this approach to coding-specific tasks - such as tool usage, accessing relevant code parts, error recovery, and context management - could potentially yield models with human-level or even superhuman autonomous coding capabilities.

What we’ve seen so far is just o1-preview, without the ability to control thinking duration. Benchmarks suggest that the full o1 model is significantly more capable. Looking ahead, how might an o3 model perform when allowed to ruminate on a coding problem for an hour, with built-in code access, linters, and memory capabilities?

Economics remains a challenge, but as models become commoditized and more efficient, costs will decrease. Given the high value of development work, the potential benefits may outweigh the costs, though this factor may slow adoption to some degree.

The question of whether we've achieved Artificial General Intelligence (AGI) depends entirely on one's definition. It's clear that the goalposts have shifted dramatically - what we have today might well be considered AGI by standards from the 1990s. We've already achieved superhuman performance in domains once considered nearly impossible for machines, such as chess, Go, and poker. With AI now achieving gold medal results in mathematics Olympiads, what's to stop it from reaching superhuman coding abilities or even superhuman general intelligence?

Agents to the rescue

Current language models possess much of the intelligence required for simple coding tasks. The primary challenge, however, lies in their shortcomings in navigating and understanding large codebases effectively. Two main obstacles persist: identifying which files to modify for a given task and optimizing the infrastructure between LLM calls.

Vector-based search on queries provides a starting point, but it's insufficient for agents that require comprehensive relevance. A more effective approach involves feeding method signatures and LLM-generated comments to the AI along with the objective, allowing it to identify pertinent elements. This "Agentless" method has shown promising results on benchmarks like SWE-bench.

For more complex scenarios, building a graph of the repository and using it to traverse relevant files may be necessary. Smaller language models could serve as efficient filters to quickly determine code relevance.

While it's technically feasible to input entire codebases into a large

context window, the economic implications make this approach impractical. Even with context caching, the cost remains prohibitive given the high volume of requests an agent makes.

Is a hybrid approach the way forward? It seems that way. Initial context could be built by LLMs selecting relevant parts based on code structures, using repository graphs to trace dependencies, with smaller language models determining snippet relevance. A crucial feature will be a feedback mechanism allowing agents to request additional context when necessary. Context, however, is only part of the solution. These agents require appropriate tools. Current agents often struggle with code edits via command-line interfaces, which doesn't reflect typical human coding practices. Moreover, integrating IDE functionalities such as imports and refactoring into these agents' workflows is essential.

Implementing predefined workflows, such as writing and running tests, could enhance efficiency. So instead of devising new processes each time, agents could follow established patterns: updating tests, inserting new tests, writing code, adding imports, running linters, fixing errors, and executing tests.

The path forward involves extensive coding and refinement. While there are no insurmountable obstacles, we are still in the early stages of development. We expect that delegating certain development tasks to AI agents will probably become standard practice in near future. And this marks just the beginning; with the scope of tasks these agents can handle is expected to steadily increase.

Finally, the teams must adapt their workflows to incorporate these new AI collaborators. The decision to begin this integration depends on an organization's willingness to be at the forefront of this technological shift.

Making agents work together

As the task-oriented agents get better, the obvious development is to chain them to cover a wider array of tasks, perhaps even autonomously. In a way this is a classical engineering solution: instead of one big machine, have smaller but nimbler machines to produce something, such as code in steps.

As outlined earlier in this book, we are going towards a seamless blend of AI agents and IDE assistants taking on various tasks. Developers will collaborate with AI on current tasks, while also overseeing and assisting AI agents working independently on other projects. The IDE will serve as the natural nexus for these interactions, providing a familiar interface for human-AI collaboration.

Also, deeper integration with platforms like GitHub is on the horizon. Assigning work to AI agents could become as straightforward as delegating a GitHub issue. GitHub Copilot Workspaces shows promise in this direction, potentially offering a comprehensive environment for AI-assisted development.

Microsoft's strong position in the developer tools market, coupled with their aggressive AI strategy, makes their ecosystem a frontrunner in these advancements. Their ownership of GitHub and substantial resources position them well to drive innovation in this space.

The autonomous software engineering agent Devin, despite maintaining a low profile since early 2024, remains a project of interest. Now under Microsoft's umbrella, significant developments are underway. A production-ready version of Devin could emerge soon, potentially with tight integration into Visual Studio Code and GitHub, leveraging Microsoft's existing developer tool ecosystem.

GitHub Copilot Extensions represent another interesting development in this space. By allowing developers to create extensions on top of Copilot, GitHub is paving the way for a platform where users can release custom agent or assistant capabilities. This opens possibilities for

integrations with third-party systems, potentially creating a rich ecosystem of AI-powered development tools.

How to become AI-powered?

The AI revolution in software development transcends mere tool selection. It's about reimagining our work processes and organizational mindsets. While choosing the right AI assistants matters, the true catalysts for change lie in how we integrate these tools and adapt our workflows.

Cutting through the red tape

Many organizations approach AI with caution, erecting barriers driven by uncertainty and fear. This approach, though understandable, frequently hinders more than it helps. The reality is that AI in development typically yields substantial productivity and quality benefits, with many perceived risks being largely unfounded.

Security and confidentiality there's certain resemblance to early discussions about security risks of open-source code or cloud computing. In AI-powered development, we are not sharing the personnel database of your company with AI, but selected fragments of the program code being developed. It has only very limited value outside your own domain and very little to no risk if shared with an AI tools even in its entirety. Of course, there might be IPR or other factors that need to be considered, but let's be honest here: but most of the code we produce is not like that.

When it comes to **sensitive data** more caution is naturally warranted.

To harness AI's full potential, **organizations must reassess blanket prohibitions and develop clear, sensible guidelines for AI use.** Educating decision-makers on the actual risks and benefits is crucial in dispelling common misconceptions. The costs associated with most AI development tools are negligible compared to the productivity gains they offer.

The first critical step towards building fully AI-enabled teams is simply allowing and encouraging the use of these powerful tools. By removing unnecessary restrictions, companies can unlock significant improvements in both efficiency and code quality.

Fostering an AI-first mindset

Leveraging AI effectively requires a shift from viewing it as a last resort to making it our first port of call. This means starting any knowledge work by establishing a baseline with AI, then refining it with human intelligence. Developers should be encouraged to view AI assistants as collaborative partners in every aspect of their work, from ideation to debugging.

This AI-first approach doesn't diminish the role of human developers. Instead, it amplifies their capabilities, allowing them to focus on higher-level thinking, creativity, and the nuanced aspects of software design that AI still struggles with. By reframing how we approach problem-solving, teams can use AI to explore multiple solutions quickly before diving deep into implementation.

Building an AI-ready knowledge base

Successful AI integration heavily relies on the quality and accessibility of our knowledge bases. AI, for all its power, cannot access silent knowledge or face-to-face communications. To maximize AI effectiveness, organizations should double down on solid documentation practices, maintain up-to-date Architectural Decision Records (ADRs), and consider enabling and storing Teams transcripts.

These practices not only feed our AI assistants but also create a more transparent, collaborative environment for all team members. By building robust, accessible knowledge bases, we're investing in both current productivity and future capabilities.

Embracing remote and asynchronous work

The integration of AI “team members” into our workflows aligns naturally with remote and asynchronous work models. This shift challenges the notion of returning to pre-COVID work environments where physical presence was prioritized over outcomes.

Organizations should focus on productivity rather than synchronous work hours, developing strategies for effective collaboration between human developers and AI tools across different time zones. This approach not only accommodates the global nature of many development teams but also leverages the 24/7 availability of AI assistants, allowing for more flexible and efficient workflows.

By addressing these aspects—reducing bureaucratic barriers, fostering an AI-first mindset, building comprehensive knowledge bases, and embracing flexible work arrangements—organizations can create an environment where AI truly enhances software development. The key is to view AI not as a threat or a mere tool, but as a collaborative partner in the pursuit of better, more efficient software creation.

Software engineering agents

While software engineering agents haven't yet achieved widespread practical adoption, they're on the brink of transforming how we approach development work. The integration of these agents into our workflows isn't just a possibility—it's an inevitability we need to prepare for.

Current state

Today's software engineering agents are primarily research projects,

often hampered by clunky user experiences. Attempting to use them now might feel awkward and inefficient. However, this awkwardness shouldn't deter forward-thinking teams from experimenting with them. These agents are at their worst now—they'll only improve from here.

It's crucial to understand that the evolution of these agents won't be a sudden leap from barely usable to solving all software engineering tasks. Instead, we'll see a gradual expansion of their capabilities. Even if they can only handle a fraction of the simplest tasks at an acceptable cost, that's a valuable starting point.

How to prepare for the agent-assisted future

As these agents breach the threshold of real-world usability, development processes will need to adapt. One clear change on the horizon: "Can an agent handle this?" should become a standard question in planning routines. This shift requires technological readiness and the willingness to question and reimagine established workflows.

It's worth noting that the mere existence of useful agents isn't enough. Teams and organizations must actively incorporate them into their processes to reap the benefits. Deliberate effort will be needed to overcome inertia in established practices.

Current recommendations

Do we recommend wholesale adoption of software engineering agents today? Not necessarily. However, some specialized agents, particularly in code review, are maturing rapidly and may already offer tangible benefits.

For teams willing to lean into the future—those ready to grapple with poor user experiences, frequent hiccups, and limited capabilities—experimenting with these agents is a viable strategy. Early adopters will face challenges but will also be best positioned to capitalize on advancements as they occur.

Looking ahead

The potential of software engineering agents is vast. As their capabilities expand, they'll take on increasingly complex and demanding work. This progression will reshape roles within development teams, potentially freeing up human developers to focus on higher-level design, creative problem-solving, and interpersonal aspects of software development.

While we can't predict exactly how quickly or dramatically these changes will unfold, one thing is clear: the era of agent-assisted software engineering is approaching. Forward-thinking teams and organizations should start preparing now, experimenting with available tools and considering how their processes might evolve in an agent-augmented development environment.

In the future, it is expected that software engineering will be a blend of human creativity and machine efficiency.

Towards AI-powered development teams

When one cog in the organizational machinery - the development team - starts to rotate faster due to AI assistance, other parts of the organization will need to adapt. This ripple effect may require adjustments in product management, quality assurance, and even customer support to keep pace with the increased output from the development team.

While AI can significantly boost productivity, the human "context window" - our ability to manage complexity and scope - won't increase at the same rate. If productivity increases fivefold, we can't expect developers to handle five times the scope. This suggests that even **smaller, more efficient teams producing higher output** could be desirable, as already promoted by various Agile efforts over the years.

The introduction of useful AI agents will **necessitate changes in agile processes**. Deciding whether an agent should attempt a task will become a standard part of the workflow. Unlike human-driven prioritization,

agents can tackle any task they're capable of immediately, potentially clearing backlogs of lower-priority or trivial issues.

We expect agents to take on more and more tasks currently done over time, with human developers evolving into “agent herders”. These modern cowboys will manage AI interactions, handle the most complex issues, and liaise with stakeholders. In a way, this allows us to **focus more on the problem than the solution**.

Accurate functional specifications will become crucial in this new paradigm. AI can't rely on the tacit or implicit knowledge – or even common sense – that human teams often depend on. **This shift towards comprehensive documentation** also supports arguments for remote work, as it reduces the need for in-person knowledge transfer. Similarly, the quality of task tickets will need to improve. This change will benefit human developers as well, reducing ambiguity and miscommunication.

Staying ahead with AI

Generative AI, as we see it today, is still a relatively new phenomenon. Reasonably capable models have only been around for a few years, and we're just beginning to understand their full potential and how they might impact white-collar jobs. This goes beyond software development alone.

The current capabilities of AI in software development are already impressive, delivering significant productivity gains. However, fully integrating these tools into development workflows is a gradual process, requiring time, effort, and adaptability. In Part 1 of this book, we presented a roadmap to begin this journey.

Another key factor in productivity improvements lies in decision-making. Do the benefits of AI-powered development outweigh the risks and costs associated with proprietary software compared to commercial alternatives or no-code/low-code solutions?

While large language models (LLMs) have primarily been limited to mimicking human capabilities, emerging approaches are pushing beyond these boundaries. New advancements are leading to smarter models with enhanced reasoning abilities, potentially transforming how we handle software development tasks. We believe that some of the current limitations of language-based models can be addressed by preprocessing and postprocessing prompts and responses, combining them with more deterministic tools, and using specialized agents for specific tasks.

Jump on the AI train before it leaves the station!

That said, achieving human-level performance from AI agents in software engineering still requires substantial progress, particularly in areas like tooling and context retrieval. Nevertheless, the gap between AI-assisted and fully autonomous development is expected to narrow.

The elephant in the room is the potential job displacement within the development field. While it's tempting to dismiss these concerns with promises of reskilling or AI merely augmenting human work, the reality might be more complicated. Lowering the cost of software development could lead to more software being created, but it might also reduce demand for lower-skill, manual tasks. It's even possible we could reach a point where fully autonomous AI agents handle software engineering with minimal human oversight.

Whether this scenario will fully materialize remains uncertain, but one thing is clear:

Jump on the AI train before it leaves the station!

Embracing and mastering AI tools now is the best way to stay relevant and valuable in an increasingly AI-driven industry.

